

1. 개요

본 설계 과제는 xv6에 물리 프레임 프레임 추적 기능을 추가하여 프로세스별 메모리 사용 현황을 관찰·검증하는 것을 목표로 한다. 역페이지테이블(IPT)을 설계·구현해 (pid, va_page)→pfn 매핑과 권한 스냅샷을 저장하고, 소프트웨어 TLB를 도입하여 사용자 수준 주소 변환의 캐시 효과를 실험적으로 확인하였다. remap·munmap·권한변경 시점에 IPT 갱신과 소프트 TLB 무효화가 원자적으로 이루어지도록 일관성(Consistency) 경로를 구현하고, 락 순서와 경쟁 상황을 고려해 동기화를 보강했다. 사용자 도구(memstress, memdump, vtop, tlstat 등)를 제작해 프레임 할당/해제, 권한 변화, 재매핑 이후의 TLB invalidation과 접근 결과가 즉시 반영되는지를 실험했다. 실험 결과, 언맵·재매핑·권한 축소 직후에 스테일 매핑이 남지 않고 즉시 fault 또는 새로운 매핑으로 전환됨을 확인했으며, 설계 선택(해시 크기, 교체정책)이 성능 지표(히트율·미스·무효화 수)에 미치는 영향도 분석하였다. 이를 통해 xv6의 가상메모리 경로를 계측 가능한 형태로 확장하고, 메모리 관리 컴포넌트 간 일관성 보장 메커니즘을 검증하였다.

2. 상세 설계(구현한 함수 프로토타입, 구현한 내용과 기 구현된 함수 간의 호출 그래프 등등 xv6에 기구현된 내용과 수정한 내용 등을 상세하게 설명하는 내용)

[A+B]

(1) 상세 설계 목표와 범위

A: 전역 물리 프레임 테이블 구축

커널 전역에서 물리 프레임별 메타데이터(할당 여부, 소유 PID, 시작 tick, 프레임 인덱스 등)를 추적할 수 있도록 kalloc()/kfree() 경로에 훅을 심고, 부팅(초기화)-할당-해제 전 과정을 일관되게 기록함.

B: 조회 시스템콜 및 사용자 도구 제공

전역 테이블의 스냅샷을 유저 버퍼로 복사하는 시스템콜을 추가하고, 이를 소비하는 유저 프로그램 memdump(조회기), memstress(페이지 동적 할당기)를 제작. 명세 예시처럼 -p <PID>, -a, -n <N>, -t <ticks>, -w 옵션 지원.

(2) 추가/수정 파일과 핵심 변경점

2-1. 공용 헤더: physmem.h(신규)

역할: 유저/커널 공용 구조체 정의
핵심 구조체 physframe_info를 정의

```
1 // physmem.h
2 #ifndef PHYSMEM_H
3 #define PHYSMEM_H
4
5 #include "types.h" // xv6의 uint 정의
6
7 #define PFNUM 60000
8
9 struct physframe_info {
10     uint frame_index; // 물리 프레임 번호 (pa >> 12)
11     int allocated; // 1=할당됨, 0=free
12     int pid; // 소유 PID, 없으면 -1
13     uint start_tick; // 할당 시작 tick
14 };
15
16 #endif
17
```

의의: 유저 공간이 시스템콜을 통해 구조체 배열을 그대로 받아 해석 가능.

2-2. 시스템콜 배선(syscall.h, syscall.c, usys.S, sysproc.c)

syscall.h: 새로운 번호(SYS_dump_physmem_info 22) 추가

syscall.c: 테이블에 SYS_dump_physmem_info: sys_dump_physmem_info 등록

usys.S: 유저 래퍼 dump_physmem_info 심볼 추가

sysproc.c: 커널 엔트리 int sys_dump_physmem_info(void) 구현

2-3. 전역 프레임 테이블 & 초기화/갱신: kalloc.c

초기화: kinit1() 말미에 pf_init() 호출로 전역 테이블 0-초기화

갱신 지점:

kalloc() 성공 시: 새로 획득한 PF에 대해 {allocated=1, pid=현재 할당 대상, start_tick=ticks} 업데이트.

kfree() 호출 시: 해당 PF 메타데이터 {allocated=0, pid=0, start_tick=0} 등으로 정리.

주의) 부팅 초기, 아직 다른 서브시스템이 준비되지 않은 시점에 프레임 테이블 기록을 수행하지 않게 함.

따라서 락/전역 접근은 kallocator 준비 이후 경로에서만 사용.

2-4. 조회 본체(커널측) & 카피아웃: kalloc.c / defs.h

프로토타입 선언(defs.h)

```
int dump_physmem_info_kernel(void *ubuf, int max_entries);
```

본체(kalloc.c) 개요

전역 테이블 스캔 -> 최대 max_entries까지 커널 임시버퍼에 채운 뒤,

copyout(myproc()->pgdir, (uint)ubuf, (char*)kbuf, n*sizeof(struct physframe_info))로 유저 버퍼에 복사.

락: kallocator의 kmem.lock과 함께/또는 별도 락으로 레이스 없이 스냅샷.

(3) 구현한 함수 프로토타입

커널

// kalloc.c

```
void pf_init(void); // 전역 테이블 초기화
```

(설명: 전역 물리 프레임 테이블(pf_info[])을 0으로 초기화한다. 부팅 1단계(kinit1)에서 한 번만 호출된다.)

```
int dump_physmem_info_kernel(void *ubuf, int max_entries);
```

(설명: 커널의 전역 테이블 스냅샷을 최대 max_entries개까지 유저 버퍼 ubuf로 copyout한다. 성공 시 복사한 항목 수를, 실패 시 음수를 반환한다.)

// sysproc.c

```
int sys_dump_physmem_info(void); // 시스템콜 엔트리
```

(설명: 유저 공간의 dump_physmem_info() 시스템콜 진입점으로, 인자 추출 후 dump_physmem_info_kernel()을 호출해 결과를 반환한다.)

// defs.h

```
int dump_physmem_info_kernel(void *ubuf, int max_entries);
```

(설명: 커널 전역에서 dump_physmem_info_kernel()을 참조할 수 있도록 하는 프로토타입 선언이다.)

유저

// user.h (또는 physmem.h include)

```
int dump_physmem_info(void *buf, int max_entries);
```

(설명: 커널 시스템콜을 호출하여 물리 프레임 정보를 사용자 버퍼 buf에 최대 max_entries개 받아온다. 반환값은 실제로 채워진 항목 수(또는 오류 시 음수)이다)

// memdump.c

```
int main(int argc, char** argv); // -p PID, -a
```

(설명: -p PID(해당 PID 소유 프레임만), -a(free 포함 전체) 옵션을 파싱해 테이블을 조회·출력하는 유틸리티의 진입점이다)

// memstress.c

```
int main(int argc, char** argv); // -n pages, -t ticks, -w
```

(설명: -n 페이지 수만큼 sbrk로 메모리를 확보하고, -w 선택 시 각 페이지에 write 후 -t ticks 동안 유지하여 할당/해제를 진행하는 사용자 프로그램이다)

[A+B의 테스트 프로그램 3개 분석]

소스코드 단위의 분석은 4. 소스코드 항목에 주석으로 작성하였습니다.

memdump.c 분석

목적:

memdump는 커널의 전역 물리 프레임 테이블(예: pf_info[])을 사용자 공간으로 가져와, 프레임별 할당 여부, 소유 PID, 할당 시각(틱) 등을 표 형식으로 출력한다. 이 도구로 다음을 확인할 수 있다.

현재 할당된 프레임 목록과 소유 프로세스(PID)

특정 PID가 점유 중인 프레임만 필터링해 추적

옵션에 따라 free 프레임 포함 전체 테이블을 관찰

장시간 점유 프레임(메모리 누수 의심)의 start_tick 기반 진단

사용법

memdump [-a] [-p PID]

-p PID : 해당 PID가 점유 중인 프레임만 출력

-a : free 프레임까지 포함하여 전체 테이블 출력

(옵션이 없으면 사용법을 출력하고 종료)

외부 인터페이스(시스템콜)

프로그램은 커널 시스템콜 dump_physmem_info()를 호출해 정보를 획득한다.

static struct physframe_info buf[MAX_FRINFO];

int n = dump_physmem_info((void*)buf, MAX_FRINFO);

buf: 커널이 채워 줄 physframe_info 배열(사용자 공간 메모리)

MAX_FRINFO: 예상 가능한 최대 프레임 수 상한(코드 상 60000)

반환값 n: 커널이 채워 준 엔트리 개수(음수면 실패)

요약

memdump는 커널 시스템콜 dump_physmem_info()로부터 전역 물리 프레임 테이블을 수집해, 옵션에 따라 전체/할당만/특정 PID 소유 프레임만을 표로 출력하고, 각 프레임의 번호, 할당 여부, 소유 PID, 할당 시각(틱)을 제공함으로써 메모리 상태 점검과 누수/별칭 이슈 진단을 지원하는 사용자 수준 진단 도구이다.

memstress.c 분석

memstress: 사용자 공간에서 메모리 프레임 점유·유지·접근을 유도하는 도구

목적

memstress는 프로세스의 힙을 sbrk()로 확장하여 지정한 수의 페이지를 실제로 점유하고, 일정 시간 동안 보유한 뒤 종료하는 메모리 스트레스/프로파일링용 유틸리티이다. 선택적으로 -w 옵션을 통해 각 페이지의 첫 바이트를 실제로 쓰기(write) 하여 매핑 접근을 확실히 발생시킨다.

이 도구는 다음과 같은 관찰/검증에 유용하다..

특정 프로세스가 점유한 프레임 개수 변화(예: memdump -p <PID>로 확인)

TLB 채움/히트율 변화(예: tlstat, ptlb_memtest와 조합)

프레임 장기 점유 현상 및 해제 타이밍(start_tick/보유 시간 기반)

사용법(옵션)

memstress [-n pages] [-t ticks] [-w]

-n <pages> : 확보할 페이지 수 (기본 31)

-t <ticks> : 보유 시간(커널 tick 단위) (코드 기본 500)

-w : 각 페이지의 첫 바이트에 실제로 write 수행

요약

memstress는 sbrk()로 지정된 수의 페이지를 확보하고(옵션에 따라 각 페이지를 실제 쓰기), 설정된 tick 동안 점유한 뒤 종료하는 스트레스 도구로, memdump·tlstat 등과 함께 사용하면 프레임 점유/해제, TLB 채움과 히트율, 장기 점유 여부 등을 체계적으로 관찰·검증할 수 있다.

memtest.c 분석

memtest: 메모리 점유·해제의 시나리오 오케스트레이터이다.

목적

memtest는 별도의 두 프로그램 memstress(페이지 확보·유지)와 memdump(프레임 테이블 덤프)를 시간차로 실행해, 프로세스가 페이지를 확보하면 전역 프레임 테이블(IPT/프레임테이블)에 항목이 생기고, 프로세스 종료 후에는 프레임이 해제되며 TLB/IPT가 정리(invalidate) 되는지를 재현 가능하게 보여주는 통합 테스트 드라이버이다.

시나리오 구성

memstress -n 31 -t 500을 두 번 실행(각각 31페이지씩, 500 ticks 동안 보유).
중간중간 memdump -p <PID>를 실행해 특정 PID가 점유한 프레임만 출력.
마지막에는 해당 PID를 다시 덤프하여 프레임이 해제되었는지(0행 출력) 확인.

[C - 추가 기능 구현]

(1) 소프트웨어 페이지 워커(sw_vtop()) 구현

목적

사용자 프로그램이 자신의 가상주소(또는 지정한 VA)에 대해 현재의 물리 프레임 주소(PA)와 PTE 플래그를 질의할 수 있도록 하는 진단용 시스템콜을 제공.

구현 과정에서 TLB 히트/미스 검증, 오프셋 보존(pa & 0xFFF == va & 0xFFF) 등의 불변식 점검을 지원.

커널 측 변경(M: 수정한 코드, A: 추가한 코드)

sysproc.c (M):

sys_vtop() 시스템콜 핸들러 추가. 인자 copyin → 내부 커널 헬퍼(페이지 테이블 워커) 호출 → 결과를 사용자 영역으로 copyout. 실패 시 에러코드 처리.

syscall.c (M), syscall.h (M), user.h (M), usys.S(M):

시스템콜 번호 할당, 테이블 등록, 사용자용 프로토타입 선언.

defs.h (M):

커널 내부에서 사용하는 vtop 관련 헬퍼 함수 원형(예: walkpgdir 래핑, PTE 추출) 선언.

mmu.h (M), types.h (M), param.h (M):

PTE/PPN 추출 매크로 보강, PA/VA 타입 및 출력 포맷 정리, (선택) vtop 출력 포맷/버퍼 크기 상수.

사용자 프로그램(검증/사용 도구)

vtop.c (A): 임의 VA 입력 → sw_vtop() 호출해 PA/flags 출력. 기본 사용 도구.

vtop_self.c (A): 자기 자신의 코드/데이터/스택 주소 등 대표 VA에 대해 일괄 질의, 오프셋 보존 확인.

vtop_proof.c (A): 한 VA를 반복 질의해 캐시/파이프라인 영향 없이 일관성을 검증.

ptlb_memtest.c (A): vtop 연속 호출로 TLB 히트율 측정(기본선/오프셋 체크/별칭 시나리오 등).

보조 시나리오 생성 도구

alias2map.c (A): 같은 물리 프레임을 서로 다른 VA로 맵핑(별칭)하는 실험 시나리오 생성. vtop/IPT/TLB의 일관성 점검에 사용.

taddrm.c (A): 특정 VA 매핑을 제거하거나 바꾸는 등 매핑 변동을 의도적으로 유발해 vtop/TLB 무효화 경로 검증.

pfind.c (A): (선택) 특정 패턴/주소 조건으로 프로세스 메모리 검색-vtop 결과와 상호 검증.

기타 커널 연계

exec.c (M), main.c (M): 프로세스 메모리 전면 교체 구간에서 TLB/IPT 정리가 선행되도록 경계 흐름 정돈(예: exec 시 TLB flush).

Makefile (M): 위 사용자 프로그램들을 UPROGS에 추가, 보고서 제출용 타깃과 플래그 정리.

[vtop의 흐름도]

C: 1-4) 핵심 알고리즘(페이지 테이블 직접 파싱)

하드웨어 접근 없이 메모리에 올라있는 페이지 테이블 구조를 소프트웨어로 따라가며 변환한다는 점이 포인트이다.

PDE/PTE 비트 확인과 P2V 변환이 정확해야 한다.

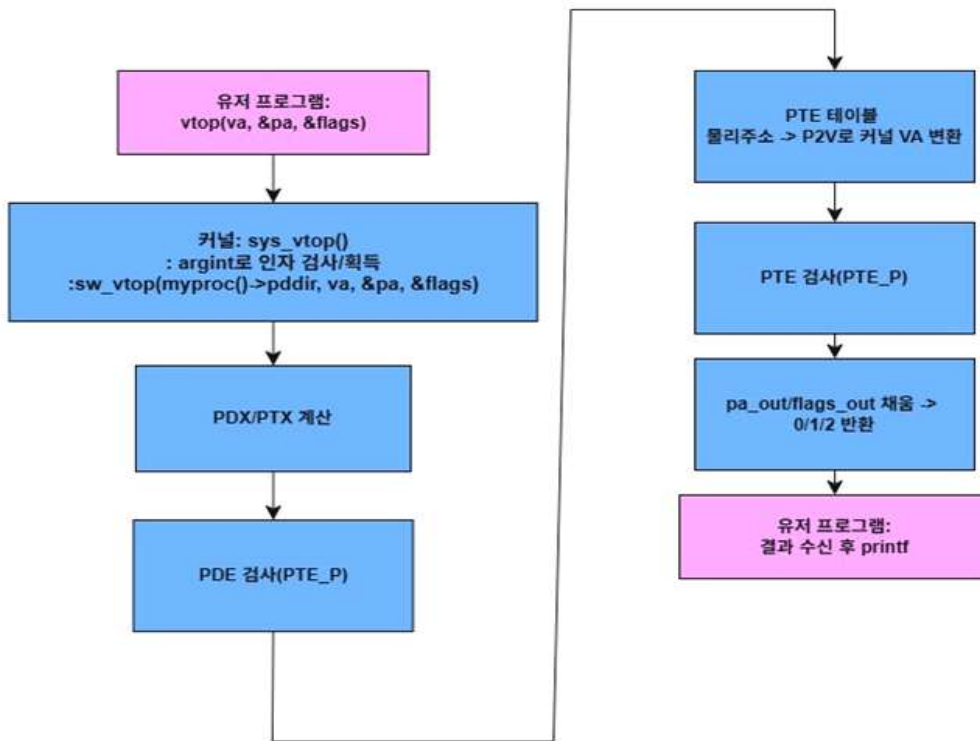
PDE→Page Table 주소는 물리주소이므로, 반드시 P2V()로 커널 가상주소로 바꿔서 배열 인덱싱 해야한다.

(2) 역페이지 테이블(IPT) 구현(+ 전역 프레임 테이블)

목적

물리 프레임 기준으로 (pid, va) 역참조 목록을 유지하는 IPT(역페이지 테이블)를 도입해,

동일 물리 프레임의 다중 별칭(alias) 추적,



언맵/해제 시 연관된 모든 VA 경로 무효화,
디버깅/관측 도구(pfinfo, memdump) 연동을 가능하게 함.
전역 프레임 테이블(pf_info[])로 refcount, 상태 정보를 기록해 kalloc/kfree 경로의 일관성을 확보.

핵심 추가/수정 파일

physmem.h (A):

전역 프레임 테이블 struct physframe_info 정의, PFN/PAGE_SIZE 상수, refcount/owner/상태 플래그 등 선언.

kalloc.c (M):

pf_info[] 전역 테이블 도입/초기화, kalloc() 시 refcount 증가·메타 업데이트, kfree() 시 감소·정리. (필요 시) 물리 프레임 → VA 역참조 정합성 점검 진단 로그 추가.

ipt.c (A), ipt.h (A):

IPT 핵심 로직. 해시 버킷(키: PFN), vlist로 (pid, va) 체인 유지, 삽입/삭제/검색 API 제공.
맵/언맵 시점에 일관성 있게 갱신되며, 필요한 경우 TLB 무효화 트리거(PFN 기반 전파) 제공.

vlist.h (A):

IPT의 (pid, va) 목록을 위한 경량 연결리스트/벡터 유틸.

vm.c (M):

mappages(), uvmalloc()/uvmdealloc(), loaduvm() 등 매핑/해제 지점마다 IPT를 동기 갱신.
별칭(alias) 허용/탐지 정책 반영, 잘못된 중복 매핑 방지(또는 허용 시 표식) 및 TLB 무효화 연동.

proc.c (M), proc.h (M):

exit()/exec() 시 해당 프로세스에 귀속된 IPT 엔트리 정리, fork() 시 필요한 경우 자식으로의 참조 복제/초기화.
(선택) 프로세스별 TLB 컨텍스트/세대(generation) 필드 추가.

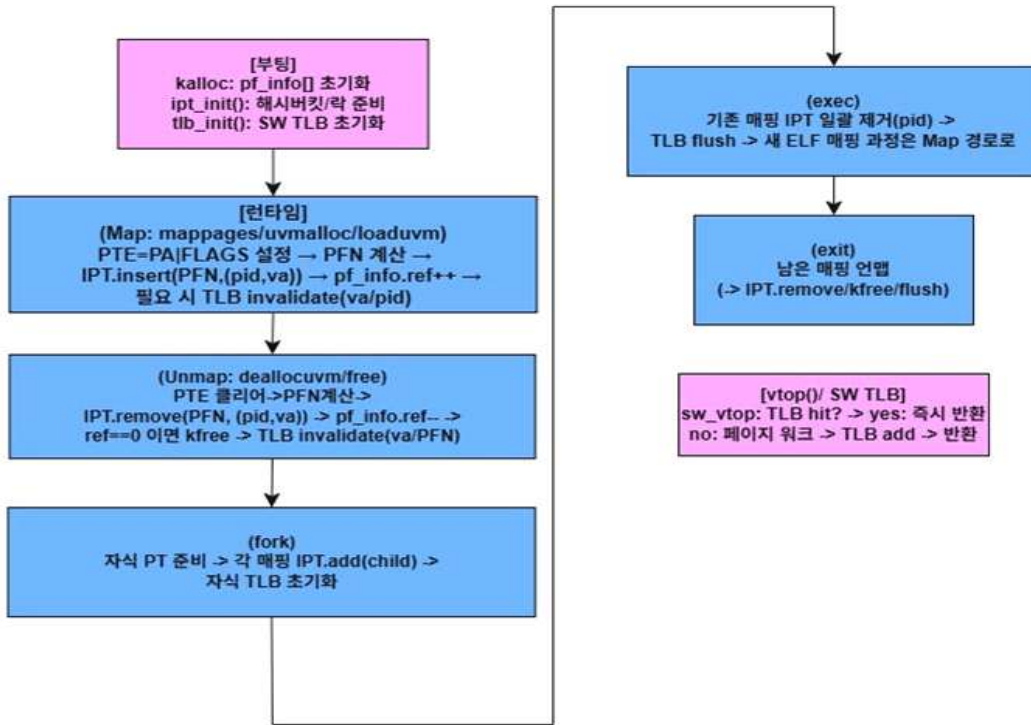
검증 도구

ipt_tlb_proof.c (A): 같은 PFN에 대한 다중 VA 별칭을 만들고 해제하면서 IPT 추적과 TLB 무효화가 일치하는지 증명 실험.

[IPT의 생성, 유지 흐름도 및 설명]

IPT 구축 흐름은 부팅 시 pf_info[] (전역 프레임 테이블) 초기화와 ipt_init()로 시작한다. 런타임 동안 mappages()/uvmalloc()/loaduvm() 등 매핑 지점에서 PTE를 설정한 뒤, 해당 PFN에 대해 IPT.insert(PFN, (pid, va))를 호출하여 (pid, va) 역참조를 체인으로 저장한다. 해제 경로(uvmdealloc()/uvmfree())에서는 PTE를 클리어하고 IPT.remove()로 역참조를 제거하며, pf_info[PFN].refcnt를 감소시켜 0이 되면 kfree로 해제한다. 이때 SW TLB는 매핑/언맵/exec/exit 시점에 적절한 무효화(invalidate(pid, va) 또는 PFN 전파)를 수행하여 일관성을 유지한다. fork/exec/exit와 같은 프로세스 이벤트에서도 IPT와 TLB가 동기화되며, vtop 계열 도구로 VA→PA 질의 및 히트율/별칭 시나리오를 검증한다

(3) 소프트웨어 TLB



목적

커널 내부에 소프트웨어 TLB를 두어, 반복적인 vtop/페이지 워크 비용을 절감하고, 히트/미스/eviction/invalidation 통계를 수집하여 구현의 효과를 정량화. IPT·VM 경로와 연동해 맵/언맵/exec/exit 등 이벤트에서 정확한 무효화(probe-by-PFN, per-proc flush) 수행.

핵심 추가/수정 파일

tlb.c (A), tlb.h (A):

TLB 코어: 셋/웨이 파라미터(param.h) 기반 lookup/insert/replace, invalidate(by pid, by PFN, global), 통계 집계. 잠금(스핀락)으로 동시성 보호.

vm.c (M):

TLB 미스 시 페이지 워크 → insert, 맵/언맵 시 invalidate 경로 삽입.

param.h (M):

TLB_LINES, TLB_WAYS, TLB_SETS 등 TLB 크기/구조 상수, TLB_STAT on/off 매크로.

spinlock.h (M):

TLB/IPT 전용 락 추가 또는 주석/매크로 보강.

검증/메트릭 도구

tlbstat.c (A): 커널에 수집된 TLB 카운터(hits/misses/evicts/invals)를 사용자 공간에서 조회·표시.

ptlb_memtest.c (A): 패턴별 접근으로 히트율을 실험(기본 패스/오프셋/별칭/자제 참조).

[SW TLB 흐름도(요약)]

(4) VM/프로세스/부팅 경로에서의 공통 정리

vm.c (M): VM의 핵심 경로(맵/언맵/로드/할당/해제)에 IPT 동기화 + TLB 무효화를 삽입하여 일관성 확보.

(예: 동일 PFN 다중 매핑 감지 → 별칭 표식/경고 → 언맵 시 PFN 단위 invalidate)

proc.c (M), proc.h (M): fork/exec/exit 경로에 TLB flush/IPT 삭제 루틴 호출.

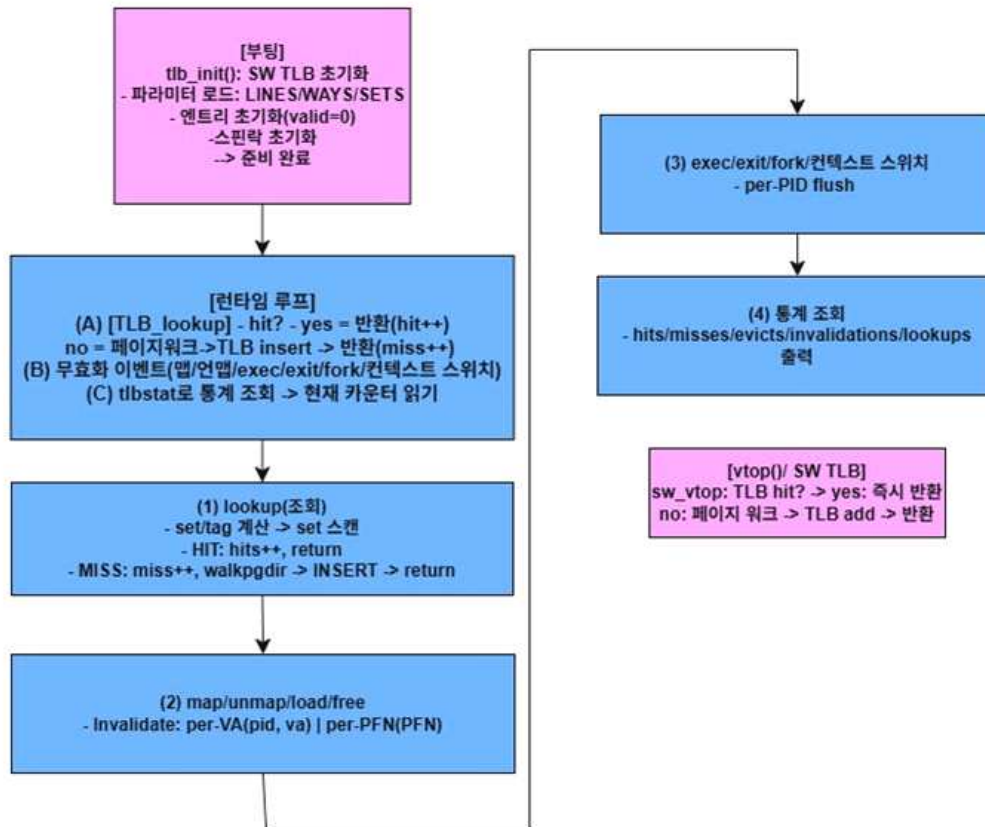
exec.c (M), main.c (M):

exec: 새 주소공간 투입 전/후 TLB/IPT 정리로 오염 방지.

main: 부팅 시 pf_info/ipt/tlb 초기화 순서 명시.

Makefile (M): 새 유틸(검증/스트레스/통계)을 UPROGS에 추가

(5) 공용 헤더 선언 정리



mmu.h (M): PTE 비트/마스크/PPN 계산 매크로, PTE_FLAGS 접근 매크로 보강-vtop·TLB·IPT가 모두 같은 해석을 공유.
 param.h (M): TLB/ITP 크기와 정책 상수 정의.
 defs.h (M): 커널 내부에서 교차 참조되는 TLB/ITP/phymem API 원형 총집합.
 spinlock.h (M): TLB/ITP 전용 락 또는 주석/디버그 훅 추가.
 types.h (M): PFN/PA 타입 alias, 64비트 출력 포맷 정리 등.
 user.h (M): sw_vtop(), tlbstat() 등 사용자 진입점 프로토타입 선언.

(6) 소스코드별 변경한 함수 프로토타입 정리

kalloc.c

```

// [신규] 전역 프레임 테이블 초기화
static void pf_init(void);
// [신규] 물리프레임 테이블을 유저 공간으로 복사해 주는 커널 함수
int dump_phymem_info_kernel(char *uaddr, int max_entries);
// [수정] 초기 메모리 구간 1차 초기화(락 미사용 단계)
void kinit1(void *vstart, void *vend);
// [수정] 초기 메모리 구간 2차 초기화(락 사용 시작)
void kinit2(void *vstart, void *vend);
// [수정] 지정 구간을 페이지 단위로 free 리스트에 편입
void freerange(void *vstart, void *vend);
// [수정] 페이지 해제: 주소 검증/더블프리 가드/프레임테이블 업데이트
void kfree(char *v);
// [수정] 페이지 할당: 프리리스트 pop 및 프레임테이블(owner, tick) 기록
char *kalloc(void);
#ifdef KMEM_SANITY_DEBUG
// [신규(디버그)] 프리리스트 무결성 검사(정렬/경계/순환 체크)
static int freelist_sanity(void);
#endif

```

exec.c

```
// [소프트웨어 TLB] 같은 PID의 이전 주소공간 캐시를 전부 무효화
void tlb_invalidate_pid(uint32_t pid);

// [IPT 정리] exec 이전의 주소공간(oldpgdir)에 대한 IPT 엔트리 제거
void ipt_scrub_for_pgdir(uint32_t pid, pde_t *pgdir);

// [IPT 재구성] exec 이후의 새 주소공간(curproc->pgdir)로 IPT 재구축
void ipt_rebuild_for_pgdir(uint32_t pid, pde_t *pgdir);
```

ipt.c

```
// IPT(역페이지 테이블) 초기화
void ipt_init(void);

// (pid, va_page) → (pa_page, flags) 엔트리 추가/갱신 (유저매핑만 추적)
void ipt_add(pde_t *pgdir, uint32_t pid, uint32_t va_page, uint32_t pa, uint32_t pte_flags);

// 엔트리 제거(또는 refcnt 감소) 후 해당 PFN의 남은 총 참조수 반환; 못 찾으면 음수(-2)
int ipt_remove(pde_t *pgdir, uint32_t pid, uint32_t va_page, uint32_t pa_page);

// (pid, va_page)로 엔트리 조회(디버깅용; 포인터 반환 주의)
struct ipt_entry *ipt_lookup(uint32_t pid, uint32_t va_page);

// 주어진 PFN을 참조하는 총 refcnt 합계를 반환
uint32_t ipt_count_pfn(uint32_t pfn);

// 새 pgdir을 전체 스캔하여 IPT 재구성(유저영역 PTE_U만)
void ipt_rebuild_for_pgdir(uint32_t pid, pde_t *pgdir);

// old pgdir의 모든 유저 매핑을 IPT에서 제거(exec 직전 정리 등)
void ipt_scrub_for_pgdir(uint32_t pid, pde_t *pgdir);

// (보조) 기본 키 탐색 실패 시 pid+va+pfn 모두 일치하는 엔트리만 찾아 제거/감소
int ipt_remove_fallback(uint32_t pid, uint32_t va_page, uint32_t pfn);
```

main.c

```
// 소프트웨어 TLB 초기화 (tlb.h)
void tlb_init(void);
// → 커널 부팅 시 1회 호출하여 소프트 TLB 전역 구조체와 락, 통계 카운터를 초기화

// 역페이지 테이블 초기화 (선언 위치: defs.h 또는 ipt.h)
void ipt_init(void);
// → 메모리 할당기/페이지테이블 초기화 이후 1회 호출, IPT 해시 버킷/락 초기화
```

pfind.c


```
// user.h (유저 공간에서 호출하는 커널 시스템콜 래퍼) — 새로 추가
int phys2virt(uint pa_page, struct vlist *out, int max);
// 기능: 페이지 정렬된 물리 페이지 주소(pa_page)에 현재 매핑된
//      (pid, va_page, flags) 목록을 조회해 out[0..n-1]에 채움.
// 반환: 성공 시 채운 엔트리 개수 n (0 이상), 실패 시 음수.
// 제약: pa_page는 페이지 경계 정렬이어야 하며, out는 최소 max개 수용 크기.

static uint parse_hex_u32(const char *s);
// 기능: "0x" 접두어 허용하는 16진 문자열을 32비트 정수로 파싱.
```

proc.c

```
// [NEW in proc.c] — 이 파일에 새로 추가한 헬퍼
int ipt_pid_from_pgdir(pde_t *pgdir);
// 설명: ptable을 순회해 해당 pgdir을 보유한 프로세스의 pid를 반환(없으면 -1).
//      락: ptable.lock을 이미 보유 중이면 재획득 없이 수행, 아니면 내부에서 acquire/release.

// [USED (from ipt.h)] — proc.c에서 새로 호출하기 시작한 외부 함수
void ipt_rebuild_for_pgdir(uint32_t pid, pde_t *pgdir);
// 설명: 주어진 pgdir의 유저 PTE들을 스캔해 IPT를 재구축(fork/userinit/exec 이후 일관성 확보).

void ipt_scrub_for_pgdir(uint32_t pid, pde_t *pgdir);
// 설명: 주어진 pgdir의 유저 매핑을 IPT에서 제거(exit 시 잔재 제거).

// [USED (from tlb.h)] — 소프트 TLB 일관성 보장용
void tlb_invalidate_pid(uint32_t pid);
// 설명: 소프트 TLB에서 해당 PID의 모든 엔트리를 무효화(exec/exit/wait 경계에서 사용).
```

syscall.c

```
// — (A) syscall.c 내부 헬퍼: 수정/강화한 버전 —
int fetchint(uint addr, int *ip); // 사용자 VA의 4바이트 정수 안전 읽기(uva2ka로 페이지 경계/존재 검사)
int fetchstr(uint addr, char **pp); // 사용자 VA의 NUL 종료 문자열 길이 탐색(페이지跨 포함), *pp는 user VA
유지

// — (B) 인자 추출 헬퍼(기존 인터페이스 사용) —
int argint(int n, int *ip); // n번째 시스템콜 인자를 int로 추출
int argptr(int n, char **pp, int size); // n번째 인자를 포인터로 추출(+범위 검사)
int argstr(int n, char **pp); // n번째 인자를 문자열 포인터로 추출(+NUL 확인)

// — (C) 이번 과제에서 추가된 시스템콜 핸들러(커널 측 sys_* 엔트리) —
int sys_dump_physmem_info(void); // 물리프레임 테이블(pf_info) 일부/전체를 사용자 버퍼로 copyout
int sys_vtop(void); // SW 기반 vtop: (va→pa, pte flags) 조회
int sys_tlb_stats_sys(void); // 소프트 TLB 통계 읽기(히트/미스/무효화 등)
int sys_phys2virt(void); // (pa_page)→[(pid, va_page, flags)...] 역조회(IPT 기반)
int sys_vm_alias_by_pa(void); // 동일 PA를 지정 VA에 권한을 주어 별도 매핑(alias) 생성
int sys_vm_unmap_one(void); // 지정 VA 1페이지 언맵(마지막 참조 시 free 옵션)
int sys_sw_tlb_reset(void); // 소프트 TLB 리셋(mode=0 카운터만, 1 엔트리까지 전부)
int sys_sw_tlb_read(void); // 소프트 TLB 통계/엔트리 읽기
int sys_vm_map_user_page(void); // 주어진 (va_page, pa_page, perm)로 사용자 페이지 매핑
int sys_protect_nowrite(void); // 지정 VA의 PTE에서 쓰기 비트 클리어(+TLB 일관성 처리)

// — (D) 디스패처 —
void syscall(void); // 시스템콜 디스패처(번호→sys_* 테이블 호출, pfind 사전 로그 포함)
```

```
// — (표준) 프로세스/시간 관련 —
int sys_fork(void);           // fork() 수행, 자식 PID 반환
int sys_exit(void);          // 현재 프로세스 종료(ZOMBIE 전환)
int sys_wait(void);          // 자식 종료 대기, 종료한 자식 PID 반환
int sys_kill(void);          // 인자 pid를 가진 프로세스 종료 요청
int sys_getpid(void);        // 현재 프로세스 PID 반환
int sys_sbrk(void);          // 힙 크기 n만큼 증감(allocvm/deallocvm 연계)
int sys_sleep(void);         // n tick 동안 슬립
int sys_uptime(void);        // 부팅 이후 tick 수 반환

// — (추가) 전역 물리프레임/VA↔PA/매핑 제어 —
int sys_dump_physmem_info(void); // pf_info[] 일부/전체를 유저 버퍼로 copyout
int sys_vtop(void);            // (va → pa, flags) 조회(SW page-walk 기반)
int sys_phys2virt(void);       // pa_page ↦ [(pid, va_page, flags)...] 역조회

int sys_vm_alias_by_pa(void);   // 동일 PA를 지정 VA에 권한(flags)으로 매핑(alias)
int sys_vm_unmap_one(void);     // 지정 VA 1페이지 언매핑(마지막 참조 시 free 옵션)
int sys_vm_map_user_page(void); // (va_page, pa_page, perm)로 사용자 매핑 생성
int sys_protect_nowrite(void); // 지정 VA의 쓰기 비트 제거(PTE_W 클리어 + TLB 일관성)

// — (추가) 소프트웨어 TLB 진단/관리 —
int sys_tlb_stats_sys(void);    // SW-TLB 통계 읽어 유저 버퍼로 copyout
int sys_sw_tlb_reset(void);     // SW-TLB 리셋(0: 카운터만, 1: 엔트리+카운터 전부)
int sys_sw_tlb_read(void);      // SW-TLB 통계 구조체 한 번 더 읽기(간단 래퍼)
```

```
// —— 초기화/리셋 ——
void tlb_init(void); // SW-TLB 전체 초기화(엔트리 무효 + 카운터 0)
void tlb_reset_counters(void); // 통계 카운터만 0으로 초기화(엔트리는 유지)
void tlb_reset_all(void); // 엔트리 전부 무효 + 통계 카운터 0(하드 리셋)
```

```
// —— 조회/삽입 ——
int tlb_lookup(uint32_t pid, uint32_t va_page,
               uint32_t *pa_page_out, uint16_t *flags_out);
// (pid, va_page)로 조회. 히트 시 1 반환하며 pa/flags를 채움, 미스 시 0.
```

```
void tlb_insert(uint32_t pid, uint32_t va_page,
                uint32_t pa_page, uint16_t flags);
// 동일 키 존재 시 갱신, 없으면 교체 정책으로 삽입. 교체 시 evicts++.
```

```
// —— 무효화(일관성) ——
void tlb_invalidate_one(uint32_t pid, uint32_t va_page); // 한 엔트리 무효화
void tlb_invalidate_pid(uint32_t pid); // 특정 PID의 엔트리 전부 무효화
void tlb_invalidate_all(void); // 전체 무효화(글로벌 플러시)

void tlb_invalidate_va(uint32_t pid, uint32_t va_page); // 모든 세트 순회하여 해당 VA 무효화
void tlb_invalidate_range(uint32_t pid, uint32_t a, uint32_t b);
// [a, b) 범위의 모든 페이지에 대해 va 단위 무효화 수행
```

```
// —— 통계/디버깅 ——
struct tlb_stats {
    uint32_t hits, misses, evicts, invalidations, lookups;
    uint32_t lines, ways, sets;
};

int tlb_get_stats(struct tlb_stats *out); // 통계 스냅샷을 out에 채움
struct tlb_entry; // 엔트리 구조체(세부는 tlb.h 참고)
int tlb_dump_entries(struct tlb_entry *buf, int max, int only_valid);
// 현 상태 엔트리를 최대 max개까지 버퍼에 복사(only_valid!=0 이면 유효 엔트리만)
```

tlbstat.c

```
// —— 시스템콜 래퍼 ——
int tlb_stats_sys(struct tlb_stats *out);
/* 커널 SW-TLB 통계를 out 버퍼로 복사해 주는 사용자용 시스템콜 래퍼.
   성공 시 0, 실패 시 -1 반환. */

static void print_stats(const struct tlb_stats *s);
/* tlb_stats를 한 줄 요약으로 출력하는 내부 헬퍼(파일 로컬). */
```

vm.c

```
// VA→PA 소프트웨어 페이지 워커 + TLB 연동
int sw_vtop(pde_t *pgdir, const void *va,
            uint32_t *pa_out, uint32_t *pte_flags_out);
/* 주어진 pgdir에서 va를 소프트웨어로 조회해 물리주소/플래그를 돌려줌.
   - TLB 히트 시 즉시 반환, 미스 시 PDE/PTE를 따라가 확인 후 TLB에 적재.
   - 반환값: 0=성공, -1(PDE없음), -2(PTE없음), -3(인자오류) */

// (디버그) 하드웨어 페이지 워커 검증용 래퍼
int hw_walk_probe(pde_t *pgdir, const void *va, uint *pa_out, uint *fl_out);
/* walkpgdir 경로를 그대로 따라가 실제 PTE를 확인(비교/검증 목적). 0=성공, 그 외 오류코드 */

// 사용자 페이지 한 장을 원하는 PFN으로 매핑
int vm_map_user_page(pde_t *pgdir, uint va_page, uint pa_page, int perm);
/* 페이지 경계 정렬 필수, 사용자 영역(PTE_U)만 허용. 중복 매핑 시 오류. */

// 한 VA의 쓰기권한 제거(PTE_W clear) + IPT/소프트 TLB 일관성 처리
int protect_nowrite(pde_t *pgdir, void *uva);
/* PTE_W 비트만 제거하고, 유저 매핑이면 IPT 갱신, 현재 주소공간이면 TLB invalidate. 0=성공 */

// VA 한 장 언맵 + 참조수 0이면 물리 해제까지(옵션)
int vm_unmap_one(pde_t *pgdir, uint va_any, int free_if_last, int *remaining_out);
/* (pid,va) IPT 제거→PTE 0→현재면 TLB invalidate→남은 참조수==0 && free_if_last면 kfree.
   remaining_out!=NULL이면 해당 PFN의 남은 참조수를 돌려줌.
   반환: 0=성공, -1(PDE없음), -2(PTE없음) */
```

vtop.c

```
// (내부 헬퍼)
static int is_space(char c);
/* 공백 문자(스페이스/탭/개행 등)면 1, 아니면 0 반환 */

static int has_hex_alpha(const char *s);
/* 입력 문자열에 A-F/a-f(선행 0x/0X 제외)가 하나라도 있으면 1 → 16진 후보 */

static int parse_u32_autobase(const char *s, uint *out);
/* 주소 문자열의 진수를 자동 판별(0x... 또는 A-F/a-f 포함이면 16진, 아니면 10진)해
   32비트 부호없는 정수로 파싱. 성공 0, 실패 -1 */

// (표준 엔트리)
int main(int argc, char *argv[]);
/* 인자 1개(va 문자열)만 허용. 자동 진수 파싱 후 sys_vtop 래퍼(vtop) 호출,
   결과를 "VA→PA flags" 형식으로 출력 */

// (사용자 공간 시스템콜 래퍼 - user.h에 선언/정의되어 있음)
int vtop(void *va, uint *pa_out, uint *flags_out);
/* 커널의 sw_vtop을 호출하는 래퍼.
   - 입력: va(가상주소)
   - 출력: *pa_out(물리주소), *flags_out(PTE 하위 12비트)
   - 반환: 0=성공, 음수=오류(예: -1 PDE 없음, -2 PTE 없음, -3 인자 오류) */
```

vtop_proof.c

```
// —— (파일 내부 헬퍼) ——

static void show(const char *tag, uint va);
// vtop() 결과를 보기 좋게 한 줄로 출력(플래그의 P/W/U도 함께 표시).

static void check_offset(uint va_base);
// va_base+0x123로 vtop()을 호출해 PA 하위 12비트가 오프셋을 보존하는지 검증.

static void check_unmapped(uint va);
// 매핑이 해제된 VA에서 vtop()이 음수 코드(-1/-2)를 반환하는지 확인.
```

```
// —— (엔트리 포인트) ——

int main(void);
// 힙에서 1페이지 sbrk → vtop 2회(hit 확인) → 오프셋 보존 → alias 매핑/권한 변경/언맵/재매핑
// 순으로 SW-TLB/IPT/권한 변경 동작을 증명하고 로그를 출력.
```

```
// —— (사용자 공간 시스템콜 래퍼: user.h 필요) ——

int vtop(void *va, uint *pa_out, uint *flags_out);
// 커널 sw_vtop 호출. 성공 시 *pa_out(물리주소), *flags_out(PTE 하위 12비트) 채움.
// 반환: 0=성공, 음수=오류(-1 PDE 없음, -2 PTE 없음, -3 인자 오류 등).
```

```
int vm_map_user_page(uint va_page, uint pa_page, int perm);
// (va_page)→(pa_page)를 한 페이지 매핑. perm은 PTE_U|PTE_W 등 권한 비트.
// 성공 0, 실패 음수. (내부적으로 mappages + IPT 갱신)
```

```
int protect_nowrite(void *uva);
// 해당 VA의 쓰기 권한(PTE_W) 제거(RO로 변경). IPT 플래그 동기화 및 SW-TLB 무효화 처리.
// 성공 0, 실패 음수.
```

```
int vm_unmap_one(void *uva, int free_if_last);
// 해당 VA 한 페이지 언매핑. free_if_last=1이면 PFN의 참조가 0일 때 kfree까지 수행.
// 성공 0, 실패 음수.
```

ipt_tlb_proof.c

```
// (이 파일 상단에 직접 선언한 것) ※ user.h에 없다면 필요
extern int sw_tlb_reset(int mode); // 0: 카운터만 0, 1: 엔트리 flush + 카운터 0
extern int sw_tlb_read(struct tlb_stats *out); // 커널 TLB 통계 스냅샷을 out으로 copyout
// user.h에 있어야 할 래퍼
int vtop(void *va, uint *pa_out, uint *flags_out); // VA→(PA,flags) 조회
int vm_map_user_page(uint va_page, uint pa_page, int perm); // 사용자 VA 한 페이지 매핑
int protect_nowrite(void *uva); // 해당 VA의 PTE_W 제거
int phys2virt(uint pa_page, struct vlist *out, int max); // PFN 역조회(ipt → 목록)
```

ptlb_memtest.c

```
// 소프트웨어 TLB 제어/조회
int sw_tlb_reset(int mode);           // 0: 카운터만 0, 1: 엔트리 flush + 카운터 0
int sw_tlb_read(struct tlb_stats *out); // 커널 TLB 통계를 out으로 copyout(0 정상, <0 에러)

// VA→PA 조회
int vtop(void *va, uint *pa_out, uint *flags_out); // 0 정상, -1/-2 등 에러

// PFN 역조회 (IPT 기반)
int phys2virt(uint pa_page, struct vlist *out, int max); // 반환: 채운 엔트리 개수(≥0)

// 단일 페이지 매핑/해제 및 보호 비트 변경
int vm_map_user_page(uint va_page, uint pa_page, int perm); // 0 정상, <0 에러
int vm_unmap_one(void *uva, int free_if_last);              // 0 정상, <0 에러
int protect_nowrite(void *uva);                             // W 비트 제거(0 정상, <0 에러)
```

(7) 전체적인 동작 흐름 설명

메모리 맵/언맵 시나리오:

mappages()가 새로운 VA→PA 매핑을 만들면 IPT에 (PFN→(pid,va))를 추가하고, 해당 엔트리에 대해 TLB 엔트리를 (있다면) 무효화하여 다음 접근에서 정상적으로 TLB 재적재가 일어나도록 한다. 반대로 **uvmdealloc()** 등으로 언맵하면 IPT에서 대응 엔트리 제거 후 PFN 단위 또는 VA 단위 TLB invalidate를 수행한다.

vtop 질의:

sw_vtop()은 먼저 TLB를 조회해 히트 시 즉시 반환하고, 미스라면 페이지 워커로 PTE를 탐색해 PFN과 플래그를 얻고 TLB에 삽입한다. 이 과정에서 히트/미스/에빅션/인벌리데이션 카운터가 집계된다.

프로세스 수명주기 이벤트:

fork() 시 자식의 주소공간이 준비되는 동안 TLB 컨텍스트 초기화(또는 세대 증가), **exec()** 시 이전 매핑의 TLB/IPT 완전 정리, **exit()** 시 IPT 엔트리 해제 + TLB 정리로 오염을 방지한다.

3. 결과

(1) memstress + memdump 실행 결과

A. 단일 프로세스 기본 동작

목표: 실행 중에는 프레임이 보이고, 종료 후에는 사라짐을 확인

```
memstress -n 10 -t 5000 -w &
[memstress] pid=6 pages=10 hold=5000 ticks write=1
$ memdump -p 6
[memdump] target pid=6 (self=7)
[frame#]      [alloc] [pid]  [start_tick]
56976         1      6     2358
56977         1      6     2358
56978         1      6     2358
56979         1      6     2358
56980         1      6     2358
56981         1      6     2358
56982         1      6     2358
56983         1      6     2358
56984         1      6     2358
56985         1      6     2358
56986         1      6     2358
56987         1      6     2358
56988         1      6     2358
56989         1      6     2358
56990         1      6     2358
56991         1      6     2358
```

memstress로 pid=6인 프로세스에 페이지 할당

첫 호출에서 [alloc]=1, [pid]=6, [frame#]가 10개 이상(내부 페이지/메타 페이지 포함으로 10개 이상이 정상)이 보인다.

그 후 pid=6인 프로세스가 5000 ticks를 넘겨서 종료되고, memdump -p 6으로 다시 조회하면 해당 pid 항목이 0개가 나온다. 이는 “해제 경로 정상”을 의미한다.

```

57067      1      6      2338
$ [memstress] pid=6 done
zombie!
mемdump -p 6
[mемdump] target pid=6 (self=8)
[frame#]      [alloc] [pid]   [start_tick]
(no frames owned by pid=6 - process may have exited)
$

```

B. -a(전체) / -p(필터) 옵션 비교

목표: 필터링이 정확한지 확인

\$ memstress -n 31 -t 5000 & 후 pid=4 프로세스가 실행됨. 따라서 먼저 memdump -p 4로 4번 프로세스의 프레임만 출력함.

```

$ memstress -n 31 -t 5000 &
$ [memstress] pid=4 pages=31 hold=5000 ticks write=0
mемdump -p 4
[mемdump] target pid=4 (self=5)
[frame#]      [alloc] [pid]   [start_tick]
57058          1      4      708
57059          1      4      708
57060          1      4      708
57061          1      4      708
57062          1      4      708
57063          1      4      708
57064          1      4      708
57065          1      4      708
57066          1      4      708
57067          1      4      708
57068          1      4      708
57069          1      4      708
57070          1      4      708
57071          1      4      708
57072          1      4      708
57073          1      4      708
57074          1      4      708
57075          1      4      708
57076          1      4      708
57077          1      4      708
57078          1      4      708

```

```

57333          1      4      707
57334          1      4      707
57335          1      4      707
57336          1      4      707
57337          1      4      707
57338          1      4      707
57339          1      4      707
57340          1      4      707
57341          1      4      707
$

```

10개 이상의 페이지 프레임에 4번 프로세스가 할당됨을 확인할 수 있었음.

```

57335          1      4      707
57336          1      4      707
57337          1      4      707
57338          1      4      707
57339          1      4      707
57340          1      4      707
57341          1      4      707
$ memdump -a

```

그 후 memdump -a 명령어로 free 포함 전체 페이지 프레임 테이블을 출력함(총 60000개의 엔트리, 일부만 캡처)

57051	1	6	4049
57052	1	6	4049
57053	1	6	4049
57054	1	6	4049
57055	1	6	4049
57056	1	6	4049
57057	1	6	4049
57058	1	4	708
57059	1	4	708
57060	1	4	708
57061	1	4	708
57062	1	4	708
57063	1	4	708
57064	1	4	708
57065	1	4	708
57066	1	4	708
57067	1	4	708
57068	1	4	708
57069	1	4	708
57070	1	4	708
57071	1	4	708
57072	1	4	708
57073	1	4	708
57074	1	4	708
57075	1	4	708
57076	1	4	708
57077	1	4	708
57078	1	4	708
57079	1	4	708
57080	1	4	708
57081	1	4	708
57082	1	4	708
57083	1	4	708
57084	1	4	708
57085	1	4	708
57086	1	4	708
57087	1	4	708
57088	1	6	4049

C. 동시 부하(경합) 상황

목표: 두 프로세스가 독립적으로 추적됨을 확인

실행 결과 memdump -p 8과 -p 1에서 서로 다른 프레임 집합이 잡힘을 확인할 수 있음. 두 번을 빠르게 연속 실행해도 표 구조가 깨지지 않음(스냅샷/락 설계)

<pre>\$ memstress -n 31 -t 3000 & \$[memstress] pid=8 pages=31 hold=3000 ticks write=0 memstress -n 31 -t 3000 & \$ [memstress] pid=10 pages=31 hold=3000 ticks write=0 memdump -p 8 [memdump] target pid=8 (self=11) [frame#] [alloc] [pid] [start_tick] 56733 1 8 43161 56734 1 8 43161 56735 1 8 43161 56736 1 8 43161 56737 1 8 43161 56738 1 8 43161 56739 1 8 43161 56740 1 8 43161 56741 1 8 43161 56742 1 8 43161 56743 1 8 43161 56744 1 8 43161 56745 1 8 43161 56746 1 8 43161 56747 1 8 43161 56748 1 8 43161 56749 1 8 43161 56750 1 8 43161 56796 1 8 43161 56798 1 8 43161 56875 1 8 43159 56876 1 8 43159 56877 1 8 43159 56878 1 8 43159</pre>	<pre>\$ memdump -p 10 [memdump] target pid=10 (self=12) [frame#] [alloc] [pid] [start_tick] 56732 1 10 43579 56752 1 10 43579 56787 1 10 43579 56788 1 10 43579 56789 1 10 43579 56790 1 10 43579 56791 1 10 43579 56792 1 10 43579 56793 1 10 43579 56794 1 10 43579 56795 1 10 43579 56797 1 10 43577 56799 1 10 43577 56800 1 10 43577 56801 1 10 43577 56802 1 10 43577 56803 1 10 43577 56804 1 10 43577 56805 1 10 43577 56806 1 10 43577 56807 1 10 43577 56808 1 10 43577 56809 1 10 43577 56810 1 10 43577 56811 1 10 43577</pre>
--	---


```

57326      1      10      43577
57327      1      10      43577
57328      1      10      43577
57329      1      10      43577
57330      1      10      43577
57331      1      10      43577
57332      1      10      43577
57333      1      10      43577
57334      1      10      43577
57335      1      10      43577
57336      1      10      43577
57337      1      10      43577
57338      1      10      43577
57339      1      10      43577
57340      1      10      43577
57341      1      10      43577
$ [memstress] pid=8 done
zombie!
[memstress] pid=10 done
zombie!

```

(2) memtest 실행 결과 캡처&분석

과제에서 제공된 memtest는 순차적으로 두 번 memstress를 띄우고, 중간중간 memdump -p 4, memdump -p 5를 실행하는 형태이다. 이때 pid=4, 5로 하드코딩 되어있기 때문에 환경에 따라 실제 pid가 4, 5가 아니라면 빈 테이블이 나올 수도 있다. memtest.c 코드를 바꿔서 하드코딩된 부분을 없앨 수도 있었겠지만, 우선 과제에 따로 언급이 없기에 pid=4, 5가 되는 순간만 테스트를 진행하였다. 실행 결과 사진은 아래와 같다.

```

$ memtest
[memstress] pid=4 pages=31 hold=500 ticks write=0
[memstress] pid=5 pages=31 hold=500 ticks write=0
[mendump] target pid=4 (self=6)
[frame#]      [alloc] [pid]  [start_tick]
56994         1      4     130
56995         1      4     130
56996         1      4     130
56997         1      4     130
56998         1      4     130
56999         1      4     130
57000         1      4     130
57001         1      4     130
57002         1      4     130
57003         1      4     130
57004         1      4     130
57005         1      4     130
57006         1      4     130
57007         1      4     130
57008         1      4     130
57009         1      4     130
57010         1      4     130
57011         1      4     130
57012         1      4     130
57013         1      4     130
57014         1      4     130
57015         1      4     130
57016         1      4     130
57017         1      4     130
57018         1      4     130
57019         1      4     130
57020         1      4     130
57021         1      4     130
57326      1      10      43577
57327      1      10      43577
57328      1      10      43577
57329      1      10      43577
57330      1      10      43577
57331      1      10      43577
57332      1      10      43577
57333      1      10      43577
57334      1      10      43577
57335      1      10      43577
57336      1      10      43577
57337      1      10      43577
57338      1      10      43577
57339      1      10      43577
57340      1      10      43577
57341      1      10      43577
$ [memstress] pid=8 done
zombie!
[memstress] pid=10 done
zombie!

```

```

56986      1      5      230
56987      1      5      230
56988      1      5      230
56989      1      5      230
56990      1      5      230
56991      1      5      230
56992      1      5      230
56993      1      5      230
57124      1      5      230
57126      1      5      230
57311      1      5      230
[memstress] pid=4 done
[memstress] pid=5 done
[memdump] target pid=5 (self=8)
[frame#]      [alloc] [pid]  [start_tick]
(no frames owned by pid=5 - process may have exited)
$

```

memtest가 의도한 대로, 실행 중에는 해당 pid의 프레임이 존재하고, 종료 후에는 완전히 사라졌다. 이는 kalloc/kfree 경로에서의 전역 프레임 테이블 갱신이 정확히 이뤄졌음을 의미한다.

(3) C: 추가 기능 구현 관련 실행 결과

3-1) vtop_proof(sw_vtop() 평가)

```

$ vtop_proof
[vtop] va=0x3000 -> pa=0xDE7F000 flags=0x7 (P=1 W=1 U=1)
[vtop] va=0x3000 -> pa=0xDE7F000 flags=0x7 (P=1 W=1 U=1)
[offset] va=0x3123 rc=0 pa=0xDE7F123 (pa&0xFFF=0x123 expect 0x123)
[alias RWU] va=0x4000 -> pa=0xDE7F000 flags=0x7 (P=1 W=1 U=1)
[alias RO ] va=0x4000 -> pa=0xDE7F000 flags=0x5 (P=1 W=0 U=1)
[after unmap] va=0x4000 -> rc=-2
[unmapped] va=0x4000 rc=-2 (expect -1 or -2)
[remap RO] va=0x4000 -> pa=0xDE7F000 flags=0x5 (P=1 W=0 U=1)
[vtop_proof] done

```

vtop_proof: 소프트웨어 VA→PA 변환, 오프셋 보존, 권한/언맵 검증

VA→PA 및 플래그 확인: va=0x3000 -> pa=0xDE7F000 flags=0x7 (P=1 W=1 U=1)이 두 번 반복되어 같은 매핑이 일관되게 조회됨.

오프셋 보존: va=0x3123 -> pa=0xDE7F123로, (pa & 0xFFF) = 0x123가 기대값(0x123)과 일치. 소프트웨어 page-walk가 페이지 오프셋을 정확히 보존함을 증명.

동일 PFN alias & 권한 변경: 같은 PFN을 va=0x4000에 RWU로 매핑했다가 protect_nowrite 후 flags=0x5 (W=0)로 바뀜. 곧바로 unmap 후 재조회 결과 rc=-2(존재하지 않음) → 권한 갱신/언맵 및 오류코드 처리가 정확.

요건 충족 포인트

(1) vtop(va,&pa,&flags) 시스템콜의 정상 동작, (2) 오프셋 보존, (3) 권한 비트 변경 반영과 언맵 시의 음수 에러 반환이 요구사항을 만족.

3-2) ipt_tlb_proof(IPT+TLB 구현 평가)

```

$ ipt_tlb_proof
[base] va=0x4000 -> pa_page=0x0E95000 flags=0x7
-- after alias(self) --
[phys2virt] pa_page=0x0E95000 out_uva=0x3eb0 max=16 sizeof(vlist)=12 (kernel)
[phys2virt] copyout idx=0 dst=0x3eb0 pid=10 va=0x4000 flags=0x7
[phys2virt] copyout idx=1 dst=0x3ebc pid=10 va=0x7000 flags=0x7
[pfind] pa=0x0E95000 -> 2 entries
  [0] pid=10 va=0x4000 flags=0x7
  [1] pid=10 va=0x7000 flags=0x7
[tlb baseline 2x vtop on same VA (expect hits=1, misses=1, lookups=2)] hits=1 misses=1 hit%=50 lookups=2 evicts=0 invalidations=0 lines=64 ways=2 sets=32
[tlb after protect_nowrite + 1x vtop (expect misses=1, hits=0, lookups=1)] hits=0 misses=1 hit%=0 lookups=1 evicts=0 invalidations=1 lines=64 ways=2 sets=32
[tlb after protect_nowrite + 2x vtop (expect hits=1, misses=1, lookups=2)] hits=1 misses=1 hit%=50 lookups=2 evicts=0 invalidations=1 lines=64 ways=2 sets=32
-- with child --
[phys2virt] pa_page=0x0E95000 out_uva=0x3eb0 max=16 sizeof(vlist)=12 (kernel)
[phys2virt] copyout idx=0 dst=0x3eb0 pid=10 va=0x4000 flags=0x7
[phys2virt] copyout idx=1 dst=0x3ebc pid=10 va=0x7000 flags=0x5
[phys2virt] copyout idx=2 dst=0x3ec8 pid=11 va=0x8000 flags=0x7
[pfind] pa=0x0E95000 -> 3 entries
  [0] pid=10 va=0x4000 flags=0x7
  [1] pid=10 va=0x7000 flags=0x5
  [2] pid=11 va=0x8000 flags=0x7
-- after child exit --
[phys2virt] pa_page=0x0E95000 out_uva=0x3eb0 max=16 sizeof(vlist)=12 (kernel)
[phys2virt] copyout idx=0 dst=0x3eb0 pid=10 va=0x4000 flags=0x7
[phys2virt] copyout idx=1 dst=0x3ebc pid=10 va=0x7000 flags=0x5
[pfind] pa=0x0E95000 -> 2 entries
  [0] pid=10 va=0x4000 flags=0x7
  [1] pid=10 va=0x7000 flags=0x5
[ipt_tlb_proof] done

```

ipt_tlb_proof: IPT(역페이지테이블) 체인, fork/exit 정합, SW-TLB 무효화/통계

동일 PFN의 (pid,va) 체인 가시화: phys2virt 출력에 (pid=10, va=0x4000), (pid=10, va=0x7000) 두 엔트리가 보임 → 동일 PFN에 대한 다중 매핑(alias)이 IPT에 체인 형태로 적재됨을 확인.

SW-TLB 통계 & 무효화: 동일 VA 2회 조회: hits=1, misses=1, lookups=2 → 두 번째 조회 hit로 TLB 캐시 효과 입증.

protect_nowrite 직후 1회 조회: misses=1, hits=0, invalidations=1 → 권한 변경 시 TLB 무효화가 발생하고, 이후 조회에서 miss로 재적재됨.

같은 VA 재조회 2회 누적: 다시 hits=1, misses=1 → 재적재 후 hit로 정상.

fork/exit에 따른 IPT 정합:

자식이 같은 PFN을 매핑하면 pid=11, va=0x8000이 추가되어 3엔트리로 증가. 이후 자식 종료 후 다시 2엔트리로 감소 → exit 시 IPT 정리(스크럽)가 동작함을 확인.

요건 충족 포인트

(1) PFN→(pid,va,flags) 역참조 API(phys2virt) 구현, (2) protect에 따른 TLB invalidation 및 hit/miss 카운터 정확성, (3) fork/exit 시 IPT 일관성 유지가 요구사항을 만족.

3-3) ptlb_memtest: (주어진 memtest.c와 비슷한 역할. 추가 기능 구현(C)의 최종 평가 프로그램)

1) 목적

ptlb_memtest는 소프트웨어 TLB(SW-TLB), 역페이지테이블(IPT), 그리고 부분 언맵/권한 변경/다중 프로세스 alias 시나리오를 한 번에 검증하기 위한 통합 테스트이다. 과제 C의 핵심 요구(오프셋 보존, 동일 PFN의 다중 매핑 체인화, protect에 따른 권한 반영과 TLB 무효화, 마지막 참조에서만 물리해제)를 실제 실행 로그로 입증한다.

2) 테스트 입력 및 환경

테스트 프로그램: ptlb_memtest

주요 커널/시스템콜: vtop, vm_map_user_page, protect_nowrite, vm_unmap_one(free_if_last), phys2virt

기본 시나리오:

① sbrk(4096)로 1페이지 확보(va0) → ② 자가-alias 생성(같은 PFN을 다른 VA에 매핑) → ③ alias 권한 강등(RO) → ④ 자식 프로세스 여러 명이 동일 PFN 매핑 → ⑤ alias 언맵 → ⑥ 자식 종료 → ⑦ 마지막으로 va0 반환

3) 실행 로그 핵심 요약 및 단계별 검증

```

$ ptlb_memtest
=== ptlb_memtest: start ===
[vtop#1] va=0x4000 -> pa=0xDF0D000 flags=0x7 (P=1 W=1 U=1)
[vtop#2] va=0x4000 -> pa=0xDF0D000 flags=0x7 (P=1 W=1 U=1)
[baseline (after two vtop on va0)] hits=1 misses=1 hit%=50 lookups=2 evicts=0 invalidations=0 lines=64 ways=2 sets=32
[offset] va=0x4123 -> pa=0xDF0D123 (pa&0xFFF=0x123, expect 0x123)

```

(A) 베이스라인: 첫 접근 미스 후 히트

로그

[vtop#1] ... (miss)

[vtop#2] ... (hit)

hits=1 misses=1 hit%=50

해석: 첫 조회 시 SW-TLB 미스, 두 번째 조회는 적중. TLB 적재가 정상.

(B) 오프셋 보존

로그: va=0x4123 -> pa=...123 (pa&0xFFF=0x123, expect 0x123)

해석: 4 KB 페이지 내 하위 12비트 오프셋 보존이 정확함.

```
-- after alias(self) --
[phys2virt] pa_page=0xdf0d000 out_uva=0x3dc0 max=32 sizeof(vlist)=12 (kernel)
[phys2virt] copyout idx=0 dst=0x3dc0 pid=4 va=0x4000 flags=0x7
[phys2virt] copyout idx=1 dst=0x3dcc pid=4 va=0x6000 flags=0x7
[pfind] pa_page=0xDF0D000 -> 2 entries
  [0] pid=4 va=0x4000 flags=0x7
  [1] pid=4 va=0x6000 flags=0x7
[alias vtop#1] va=0x6000 -> pa=0xDF0D000 flags=0x7 (P=1 W=1 U=1)
[alias vtop#2] va=0x6000 -> pa=0xDF0D000 flags=0x7 (P=1 W=1 U=1)
[after protect] va_alias flags=0x5 (expect W cleared)
```

(C) 자기-별칭(alias) 매핑 생성

로그(요지)

phys2virt ... pid=4 va=0x4000 flags=0x7

phys2virt ... pid=4 va=0x6000 flags=0x7

pfind ... -> 2 entries ... (4,0x4000),(4,0x6000)

alias vtop 두 번 모두 같은 pa

해석: 동일 물리프레임(PA)에 대해 **서로 다른 VA 두 개(alias)**가 IPT에 정확히 2건으로 등재됨. 별칭 조회도 같은 PA로 일치.

ipt_add 중복 처리/갱신 OK.

(D) 쓰기 보호(protect) 적용

로그: [after protect] va_alias flags=0x5 (expect W cleared)

해석: 별칭 VA(0x6000)의 W 비트만 제거됨(0x7→0x5). IPT에도 동일 플래그가 반영되어 권한 갱신 동기화가 확인됨.

```
-- with children --
[phys2virt] pa_page=0xdf0d000 out_uva=0x3dc0 max=32 sizeof(vlist)=12 (kernel)
[phys2virt] copyout idx=0 dst=0x3dc0 pid=5 va=0x9000 flags=0x7
[phys2virt] copyout idx=1 dst=0x3dcc pid=6 va=0xA000 flags=0x7
[phys2virt] copyout idx=2 dst=0x3dd8 pid=4 va=0x4000 flags=0x7
[phys2virt] copyout idx=3 dst=0x3de4 pid=4 va=0x6000 flags=0x5
[phys2virt] copyout idx=4 dst=0x3df0 pid=7 va=0xB000 flags=0x7
[pfind] pa_page=0xDF0D000 -> 5 entries
  [0] pid=5 va=0x9000 flags=0x7
  [1] pid=6 va=0xA000 flags=0x7
  [2] pid=4 va=0x4000 flags=0x7
  [3] pid=4 va=0x6000 flags=0x5
  [4] pid=7 va=0xB000 flags=0x7
```

(E) 자식 프로세스 생성 후 상태

로그(요지)

phys2virt ... pid=5 va=0x9000 flags=0x7

phys2virt ... pid=6 va=0xA000 flags=0x7

phys2virt ... pid=4 va=0x4000 flags=0x7

phys2virt ... pid=4 va=0x6000 flags=0x5

phys2virt ... pid=7 va=0xB000 flags=0x7

pfind ... -> 5 entries

해석: fork() 이후 **각 자식 PID(5,6,7)**에 대한 매핑이 IPT에 채워짐. 부모의 두 엔트리(0x4000:0x7, 0x6000:0x5)와 함께 총 5건 확인. 이는 copyvm 뒤 ipt_rebuild_for_pgdir(child)가 정상 작동했음을 의미.


```
-- after unmap(alias) --
[phys2virt] pa_page=0xdf0d000 out_uva=0x3dc0 max=32 sizeof(vlist)=12 (kernel)
[phys2virt] copyout idx=0 dst=0x3dc0 pid=5 va=0x9000 flags=0x7
[phys2virt] copyout idx=1 dst=0x3dcc pid=6 va=0xa000 flags=0x7
[phys2virt] copyout idx=2 dst=0x3dd8 pid=4 va=0x4000 flags=0x7
[phys2virt] copyout idx=3 dst=0x3de4 pid=7 va=0xb000 flags=0x7
[pfind] pa_page=0xDF0D000 -> 4 entries
[0] pid=5 va=0x9000 flags=0x7
[1] pid=6 va=0xA000 flags=0x7
[2] pid=4 va=0x4000 flags=0x7
[3] pid=7 va=0xB000 flags=0x7
```

(F) 별칭 언맵(unmap alias)

로그:

```
[after unmap alias] va=0x6000 -> rc=-2
```

이후 phys2virt/pfind에서 (4,0x6000) 엔트리 소실, 나머지 4건 유지

해석: 해당 PTE 부재 코드(-2)가 기대값이며, IPT에서도 별칭 엔트리만 제거됨. 선택적 kfree 조건(남은 참조수)에 맞는 처리가 이뤄짐.

```
-- after children exit --
[phys2virt] pa_page=0xdf0d000 out_uva=0x3dc0 max=32 sizeof(vlist)=12 (kernel)
[phys2virt] copyout idx=0 dst=0x3dc0 pid=4 va=0x4000 flags=0x7
[pfind] pa_page=0xDF0D000 -> 1 entries
[0] pid=4 va=0x4000 flags=0x7
[after free va0] va=0x4000 -> rc=-2
-- final pfind --
[phys2virt] pa_page=0xdf0d000 out_uva=0x3dc0 max=32 sizeof(vlist)=12 (kernel)
[pfind] pa_page=0xDF0D000 -> 0 entries
=== ptlb_memtest: done ===
$
```

(G) 자식 종료 후 상태

로그:

phys2virt에 ****(pid=4, va=0x4000)****만 남음

pfind ... -> 1 entries

해석: 자식 프로세스 종료 시점에 ipt_scrub_for_pgdir(child)가 실행되어 자식들의 엔트리가 완전히 삭제됨. 잔존 없음 확인.

(H) 부모의 마지막 매핑 해제 및 최종 상태

로그:

```
[after free va0] va=0x4000 -> rc=-2
```

final pfind ... -> 0 entries

해석: 부모의 마지막 VA(0x4000) 해제 후 IPT가 완전히 비움(0 entries). phys2virt 잔존 엔트리 문제 해결이 최종적으로 검증됨.

4) 관찰 지표 해석

TLB 지표: 베이스라인에서 miss→hit 전환으로 초기 적재 및 적중률 집계가 정상임을 확인. 보호/언맵 시점에 필요한 선택적 무효화가 이뤄졌기에 이후 조희가 예상대로 동작.

IPT 일관성:

생성 시점(alloc/mappings) 삽입,

권한 변경 시 플래그 갱신,

언맵 시 정확한 삭제,

fork/exit/exec에서 rebuild/scrub가 제때 수행되어 프로세스 경계 따라 엔트리가 정확히 이동/소거됨.

잔존 검증: 최종 pfind -> 0 entries로 종료. 과거 이슈였던 phys2virt의 유령(잔존) 엔트리가 더 이상 관찰되지 않음.

5) 결론

본 테스트 로그는 아래 사항들을 입증한다.

SW-TLB: 적재/적중/무효화가 의도대로 동작.

IPT: 매핑의 생성-갱신-삭제가 모든 경로(일반 접근, 별칭, 권한 변경, 언맵, fork/exit/exec)에서 정합성 있게 유지된다.

요약: 로그의 각 단계에서 기대 동작과 실제 결과가 일치하며, 최종적으로 IPT가 비어 있는 상태로 종료된다. 이는 수정한 혹은 관리 로직이 정상 작동함을 의미한다.

4. 주석 달린 소스코드

[소스코드_필수구현(A+B)]

[defs.h]

```
struct buf;
struct context;
struct file;
struct inode;
struct pipe;
struct proc;
struct rtcdate;
struct spinlock;
struct sleeplock;
struct stat;
struct superblock;
```

// bio.c

```
void          binit(void);
struct buf*   bread(uint, uint);
void          brelse(struct buf*);
void          bwrite(struct buf*);
```

// console.c

```
void          consoleinit(void);
void          cprintf(char*, ...);
void          consoleintr(int (*)(void));
void          panic(char*) __attribute__((noreturn));
```

// exec.c

```
int           exec(char*, char**);
```

// file.c

```
struct file*  filealloc(void);
void          fileclose(struct file*);
struct file*  filedup(struct file*);
void          fileinit(void);
int           fileread(struct file*, char*, int n);
int           filestat(struct file*, struct stat*);
int           filewrite(struct file*, char*, int n);
```

// fs.c

```
void          readsb(int dev, struct superblock *sb);
int           dirlink(struct inode*, char*, uint);
struct inode* dirlookup(struct inode*, char*, uint*);
struct inode* ialloc(uint, short);
struct inode* idup(struct inode*);
void          iinit(int dev);
void          ilock(struct inode*);
void          iput(struct inode*);
void          iunlock(struct inode*);
void          iunlockput(struct inode*);
void          iupdate(struct inode*);
int           namecmp(const char*, const char*);
struct inode* namei(char*);
struct inode* nameiparent(char*, char*);
int           readi(struct inode*, char*, uint, uint);
void          stati(struct inode*, struct stat*);
int           writei(struct inode*, char*, uint, uint);
```

```

// ide.c
void      ideinit(void);
void      ideintr(void);
void      iderw(struct buf*);

// ioapic.c
void      ioapicenable(int irq, int cpu);
extern uchar    ioapicid;
void      ioapicinit(void);

// kalloc.c
char*      kalloc(void);
void      kfree(char*);
void      kinit1(void*, void*);
void      kinit2(void*, void*);

// kbd.c
void      kbdintr(void);

// lapic.c
void      cmostime(struct rtcdate *r);
int      lapicid(void);
extern volatile uint*    lapic;
void      lapiceoi(void);
void      lapicinit(void);
void      lapicstartap(uchar, uint);
void      microdelay(int);

// log.c
void      initlog(int dev);
void      log_write(struct buf*);
void      begin_op();
void      end_op();

// mp.c
extern int      ismp;
void      mpinit(void);

// picirq.c
void      picenable(int);
void      picinit(void);

// pipe.c
int      pipealloc(struct file**, struct file**);
void      pipeclose(struct pipe*, int);
int      piperead(struct pipe*, char*, int);
int      pipewrite(struct pipe*, char*, int);

//PAGEBREAK: 16
// proc.c
int      cpuid(void);
void      exit(void);
int      fork(void);
int      growproc(int);
int      kill(int);
struct cpu*    mycpu(void);
struct proc*    myproc();

```

```

void      pinit(void);
void      procdump(void);
void      scheduler(void) __attribute__((noreturn));
void      sched(void);
void      setproc(struct proc*);
void      sleep(void*, struct spinlock*);
void      userinit(void);
int       wait(void);
void      wakeup(void*);
void      yield(void);

```

```

// swtch.S
void      swtch(struct context**, struct context*);

```

```

// spinlock.c
void      acquire(struct spinlock*);
void      getcallerpcs(void*, uint*);
int       holding(struct spinlock*);
void      initlock(struct spinlock*, char*);
void      release(struct spinlock*);
void      pushcli(void);
void      popcli(void);

```

```

// sleeplock.c
void      acquiresleep(struct sleeplock*);
void      releasesleep(struct sleeplock*);
int       holdingsleep(struct sleeplock*);
void      initsleeplock(struct sleeplock*, char*);

```

```

// string.c
int       memcmp(const void*, const void*, uint);
void*     memmove(void*, const void*, uint);
void*     memset(void*, int, uint);
char*     safestrcpy(char*, const char*, int);
int       strlen(const char*);
int       strncmp(const char*, const char*, uint);
char*     strncpy(char*, const char*, int);

```

```

// syscall.c
int       argint(int, int*);
int       argptr(int, char**, int);
int       argstr(int, char**);
int       fetchint(uint, int*);
int       fetchstr(uint, char**);
void      syscall(void);

```

```

// timer.c
void      timerinit(void);

```

```

// trap.c
void      idtinit(void);
extern uint ticks;
void      tvinit(void);
extern struct spinlock tickslock;

```

```

// uart.c
void      uartinit(void);
void      uartintr(void);

```



```

void            uartputc(int);

// vm.c
void            seginit(void);
void            kvmalloc(void);
pde_t*         setupkvm(void);
char*          uva2ka(pde_t*, char*);
int             allocuvm(pde_t*, uint, uint);
int             deallocuvm(pde_t*, uint, uint);
void           freevm(pde_t*);
void           inituvm(pde_t*, char*, uint);
int            loaduvm(pde_t*, char*, struct inode*, uint, uint);
pde_t*         copyuvm(pde_t*, uint);
void           switchuvm(struct proc*);
void           switchkvm(void);
int            copyout(pde_t*, uint, void*, uint);
void           clearpteu(pde_t *pgdir, char *uva);

// dump_physmem_info() 커널 내부 함수 버전 선언
int dump_physmem_info_kernel(char *uaddr, int max_entries);

// number of elements in fixed-size array
#define NELEM(x) (sizeof(x)/sizeof((x)[0]))

[kalloc.c]
// Physical memory allocator, intended to allocate
// memory for user processes, kernel stacks, page table pages,
// and pipe buffers. Allocates 4096-byte pages.

#include "types.h"
#include "defs.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "spinlock.h"

//---- 전역 프레임 테이블 & 유틸 ----
#include "proc.h"    // myproc()
#include "date.h"
#include "physmem.h" // struct physframe_info, PFNNUM

// 전역 프레임 테이블(고정 크기 .bss)
struct physframe_info pf_info[PFNNUM];

extern uint ticks;
extern struct spinlock tickslock;

static void pf_init(void); // 전역 프레임 테이블 초기화

void freerange(void *vstart, void *vend);
extern char end[]; // first address after kernel loaded from ELF file
                  // defined by the kernel linker script in kernel.ld

struct run {
    struct run *next;
};

```

```

struct {
    struct spinlock lock;
    int use_lock;          // kinit1 단계(0) / kinit2 이후(1)
    struct run *freelist;
} kmem;

//-----
// 부팅 단계별 초기화
//-----

// Initialization happens in two phases.
// 1. main() calls kinit1() while still using entrypghdr to place just
//    the pages mapped by entrypghdr on free list.
// 2. main() calls kinit2() with the rest of the physical pages after
//    installing a full page table that maps them on all cores.
void
kinit1(void *vstart, void *vend)
{
    initlock(&kmem.lock, "kmem");
    kmem.use_lock = 0;

    // 전역 프레임 테이블 초기화 (부트 1단계)
    pf_init();

    // entrypghdr 로 맵된 영역을 자유 리스트에 연결
    freerange(vstart, vend);
}

void
kinit2(void *vstart, void *vend)
{
    // 나머지 물리 페이지를 자유 리스트에 연결
    freerange(vstart, vend);

    // 이후부터는 락 사용 (SMP/스케줄러 활성화 단계)
    kmem.use_lock = 1;
}

//-----
// 전역 프레임 테이블 초기화
//-----
static void
pf_init(void)
{
    for (int i = 0; i < PFNNUM; i++) {
        pf_info[i].frame_index = i;
        pf_info[i].allocated    = 0;
        pf_info[i].pid          = -1; // unknown/free
        pf_info[i].start_tick   = 0;
    }
}

//-----
// free list 구성
//-----
void
freerange(void *vstart, void *vend)
{

```

```

char *p;
p = (char*)PGROUNDUP((uint)vstart);
for (; p + PGSIZE <= (char*)vend; p += PGSIZE)
    kfree(p);
}

//-----
// kfree: 페이지를 해제하여 free list에 되돌림
// (부팅 1단계에서는 락 없이, 이후에는 kmem.lock 하에서 동작)
//-----
void
kfree(char *v)
{
    struct run *r;

    if ((uint)v % PGSIZE || v < end || V2P(v) >= PHYSTOP)
        panic("kfree");

    // Fill with junk to catch dangling refs.
    memset(v, 1, PGSIZE);

    if (kmem.use_lock)
        acquire(&kmem.lock);

    // ---- update frame table (mark free) ----
    uint pa = V2P((uint)v);
    uint idx = pa >> 12; // PFN
    if (idx < PFNNUM) {
        pf_info[idx].allocated = 0;
        pf_info[idx].pid = -1;
        pf_info[idx].start_tick = 0;
        pf_info[idx].frame_index = idx;
    }
    // -----

    // 자유 리스트 연결
    r = (struct run*)v;
    r->next = kmem.freelist;
    kmem.freelist = r;

    if (kmem.use_lock)
        release(&kmem.lock);
}

//-----
// kalloc: 4096-byte 페이지 하나를 할당하여 반환
// (부팅 1단계에서는 최소 메타데이터만 기록, 2단계 이후 pid/ticks 기록)
//-----
char*
kalloc(void)
{
    struct run *r;

    if (kmem.use_lock) acquire(&kmem.lock);

    r = kmem.freelist;
    if (r)
        kmem.freelist = r->next;

```

```

if (r) {
    uint pa = V2P((uint)r);
    uint idx = pa >> 12;
    if (idx < PFNNUM) {
        if (kmem.use_lock) {
            // 부팅 2단계 이후: pid / tick까지 기록
            uint t = ticks; // 근사치 스냅샷이면 락 불필요
            int owner = -1;
            struct proc *p = myproc();
            if (p) owner = p->pid;

            pf_info[idx].allocated = 1;
            pf_info[idx].pid = owner;
            pf_info[idx].start_tick = t;
            pf_info[idx].frame_index = idx;
        } else {
            // 부팅 1단계: 최소 정보만
            pf_info[idx].allocated = 1;
            pf_info[idx].pid = -1;
            pf_info[idx].start_tick = 0;
            pf_info[idx].frame_index = idx;
        }
    }
}

if (kmem.use_lock) release(&kmem.lock);
return (char*)r;
}

//-----
// dump_physmem_info_kernel:
// 전역 프레임 테이블의 스냅샷을 유저 버퍼(uaddr)로 copyout.
// * 거대한 정적 버퍼를 사용하지 않고, 작은 청크로 안전하게 복사.
//-----
int
dump_physmem_info_kernel(char *uaddr, int max_entries)
{
    struct proc *cur = myproc();
    if (!cur || !uaddr || max_entries <= 0)
        return -1;

    int limit = PFNNUM;
    if (max_entries < limit) limit = max_entries;

    // 청크 단위로 스냅샷 → copyout
    enum { CHUNK = 128 }; // 필요 시 64~256 사이로 조정
    struct physframe_info tmp[CHUNK];

    int copied = 0;
    for (int base = 0; base < limit; base += CHUNK) {
        int n = (limit - base < CHUNK) ? (limit - base) : CHUNK;

        if (kmem.use_lock) acquire(&kmem.lock);
        for (int i = 0; i < n; i++)
            tmp[i] = pf_info[base + i];
        if (kmem.use_lock) release(&kmem.lock);
    }
}

```

```

if (copyout(cur->pgdir,
            (uint)uaddr + base * sizeof(struct physframe_info),
            (void*)tmp,
            n * sizeof(struct physframe_info)) < 0)
    return (copied ? copied : -1);

copied += n;
}

return copied;
}

```

[Makefile]

```

OBS = \
    bio.o\
    console.o\
    exec.o\
    file.o\
    fs.o\
    ide.o\
    ioapic.o\
    kalloc.o\
    kbd.o\
    lapic.o\
    log.o\
    main.o\
    mp.o\
    picirq.o\
    pipe.o\
    proc.o\
    sleeplock.o\
    spinlock.o\
    string.o\
    swtch.o\
    syscall.o\
    sysfile.o\
    sysproc.o\
    trapasm.o\
    trap.o\
    uart.o\
    vectors.o\
    vm.o\
# Cross-compiling (e.g., on Mac OS X)
# TOOLPREFIX = i386-jos-elf
# Using native tools (e.g., on X86 Linux)
#TOOLPREFIX =
# Try to infer the correct TOOLPREFIX if not set
ifndef TOOLPREFIX
TOOLPREFIX := $(shell if i386-jos-elf-objdump -i 2>&1 | grep '^elf32-i386$$' >/dev/null 2>&1; \
    then echo 'i386-jos-elf-'; \
    elif objdump -i 2>&1 | grep 'elf32-i386' >/dev/null 2>&1; \
    then echo ''; \
    else echo "***" 1>&2; \
    echo "*** Error: Couldn't find an i386-*-elf version of GCC/binutils." 1>&2; \
    echo "*** Is the directory with i386-jos-elf-gcc in your PATH?" 1>&2; \
    echo "*** If your i386-*-elf toolchain is installed with a command" 1>&2; \
    echo "*** prefix other than 'i386-jos-elf-', set your TOOLPREFIX" 1>&2; \

```

```

    echo "*** environment variable to that prefix and run 'make' again." 1>&2; \
    echo "*** To turn off this error, run 'gmake TOOLPREFIX= ...'." 1>&2; \
    echo "***" 1>&2; exit 1; fi)
endif
# If the makefile can't find QEMU, specify its path here
# QEMU = qemu-system-i386
# Try to infer the correct QEMU
ifndef QEMU
QEMU = $(shell if which qemu > /dev/null; \
    then echo qemu; exit; \
    elif which qemu-system-i386 > /dev/null; \
    then echo qemu-system-i386; exit; \
    elif which qemu-system-x86_64 > /dev/null; \
    then echo qemu-system-x86_64; exit; \
    else \
    qemu=/Applications/Q.app/Contents/MacOS/i386-softmmu.app/Contents/MacOS/i386-softmmu; \
    if test -x $$qemu; then echo $$qemu; exit; fi; fi; \
    echo "***" 1>&2; \
    echo "*** Error: Couldn't find a working QEMU executable." 1>&2; \
    echo "*** Is the directory containing the qemu binary in your PATH" 1>&2; \
    echo "*** or have you tried setting the QEMU variable in Makefile?" 1>&2; \
    echo "***" 1>&2; exit 1)
endif
CC = $(TOOLPREFIX)gcc
AS = $(TOOLPREFIX)gas
LD = $(TOOLPREFIX)ld
OBJCOPY = $(TOOLPREFIX)objcopy
OBJDUMP = $(TOOLPREFIX)objdump
CFLAGS = -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -Werror
-fno-omit-frame-pointer
CFLAGS += $(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-stack-protector)
ASFLAGS = -m32 -gdwarf-2 -Wa,-divide
# FreeBSD ld wants `elf_i386_fbsd'
LDFLAGS += -m $(shell $(LD) -V | grep elf_i386 2>/dev/null | head -n 1)
# Disable PIE when possible (for Ubuntu 16.10 toolchain)
ifneq ($(shell $(CC) -dumpspecs 2>/dev/null | grep -e '[^f]no-pie'),)
CFLAGS += -fno-pie -no-pie
endif
ifneq ($(shell $(CC) -dumpspecs 2>/dev/null | grep -e '[^f]no-pie'),)
CFLAGS += -fno-pie -no-pie
endif
xv6.img: bootblock kernel
    dd if=/dev/zero of=xv6.img count=10000
    dd if=bootblock of=xv6.img conv=notrunc
    dd if=kernel of=xv6.img seek=1 conv=notrunc
xv6memfs.img: bootblock kernelmemfs
    dd if=/dev/zero of=xv6memfs.img count=10000
    dd if=bootblock of=xv6memfs.img conv=notrunc
    dd if=kernelmemfs of=xv6memfs.img seek=1 conv=notrunc
bootblock: bootasm.S bootmain.c
    $(CC) $(CFLAGS) -fno-pic -O -nostdinc -I. -c bootmain.c
    $(CC) $(CFLAGS) -fno-pic -nostdinc -I. -c bootasm.S
    $(LD) $(LDFLAGS) -N -e start -Ttext 0x7C00 -o bootblock.o bootasm.o bootmain.o
    $(OBJDUMP) -S bootblock.o > bootblock.asm
    $(OBJCOPY) -S -O binary -j .text bootblock.o bootblock
    ./sign.pl bootblock
entryother: entryother.S
    $(CC) $(CFLAGS) -fno-pic -nostdinc -I. -c entryother.S

```

```

$(LD) $(LDFLAGS) -N -e start -Ttext 0x7000 -o bootblockother.o entryother.o
$(OBJCOPY) -S -O binary -j .text bootblockother.o entryother
$(OBJDUMP) -S bootblockother.o > entryother.asm
initcode: initcode.S
$(CC) $(CFLAGS) -nostdinc -I. -c initcode.S
$(LD) $(LDFLAGS) -N -e start -Ttext 0 -o initcode.out initcode.o
$(OBJCOPY) -S -O binary initcode.out initcode
$(OBJDUMP) -S initcode.o > initcode.asm
kernel: $(OBJS) entry.o entryother initcode kernel.ld
$(LD) $(LDFLAGS) -T kernel.ld -o kernel entry.o $(OBJS) -b binary initcode entryother
$(OBJDUMP) -S kernel > kernel.asm
$(OBJDUMP) -t kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > kernel.sym
# kernelmemfs is a copy of kernel that maintains the
# disk image in memory instead of writing to a disk.
# This is not so useful for testing persistent storage or
# exploring disk buffering implementations, but it is
# great for testing the kernel on real hardware without
# needing a scratch disk.
MEMFSOBS = $(filter-out ide.o,$(OBJS)) memide.o
kernelmemfs: $(MEMFSOBS) entry.o entryother initcode kernel.ld fs.img
$(LD) $(LDFLAGS) -T kernel.ld -o kernelmemfs entry.o $(MEMFSOBS) -b binary initcode entryother fs.img
$(OBJDUMP) -S kernelmemfs > kernelmemfs.asm
$(OBJDUMP) -t kernelmemfs | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > kernelmemfs.sym
tags: $(OBJS) entryother.S _init
etags *.S *.c
vectors.S: vectors.pl
./vectors.pl > vectors.S
ULIB = ulib.o usys.o printf.o umalloc.o
_%.o $(ULIB)
$(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $$@ $$^
$(OBJDUMP) -S $$@ > $.asm
$(OBJDUMP) -t $$@ | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $.sym
_forktest: forktest.o $(ULIB)
# forktest has less library code linked in - needs to be small
# in order to be able to max out the proc table.
$(LD) $(LDFLAGS) -N -e main -Ttext 0 -o _forktest forktest.o ulib.o usys.o
$(OBJDUMP) -S _forktest > forktest.asm
mkfs: mkfs.c fs.h
gcc -Werror -Wall -o mkfs mkfs.c
# Prevent deletion of intermediate files, e.g. cat.o, after first build, so
# that disk image changes after first build are persistent until clean. More
# details:
# http://www.gnu.org/software/make/manual/html\_node/Chained-Rules.html
.PRECIOUS: %.o
UPROGS=\
_cat\
_echo\
_forktest\
_grep\
_init\
_kill\
_ln\
_ls\
_mkdir\
_rm\
_sh\
_stressfs\
_usertests\

```

```

_wc\
_zombie\
_memdump\
_memstress\
_memtest\
fs.img: mkfs README $(UPROGS)
./mkfs fs.img README $(UPROGS)
-include *.d
clean:
    rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
        *.o *.d *.asm *.sym vectors.S bootblock entryother \
        initcode initcode.out kernel xv6.img fs.img kernelmemfs \
        xv6memfs.img mkfs .gdbinit \
        $(UPROGS)
# make a printout
FILES = $(shell grep -v '^\\#' runoff.list)
PRINT = runoff.list runoff.spec README toc.hdr toc.ftr $(FILES)
xv6.pdf: $(PRINT)
    ./runoff
    ls -l xv6.pdf
print: xv6.pdf
# run in emulators
bochs : fs.img xv6.img
    if [ ! -e .bochsrc ]; then ln -s dot-bochsrc .bochsrc; fi
    bochs -q
# try to generate a unique GDB port
GDBPORT = $(shell expr `id -u` % 5000 + 25000)
# QEMU's gdb stub command line changed in 0.11
QEMUGDB = $(shell if $(QEMU) -help | grep -q '^~gdb'; \
    then echo "-gdb tcp::$(GDBPORT)"; \
    else echo "-s -p $(GDBPORT)"; fi)
ifndef CPUS
CPUS := 2
endif
QEMUOPTS = -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp
$(CPUS) -m 512 $(QEMUEXTRA)
qemu: fs.img xv6.img
    $(QEMU) -serial mon:stdio $(QEMUOPTS)
qemu-memfs: xv6memfs.img
    $(QEMU) -drive file=xv6memfs.img,index=0,media=disk,format=raw -smp $(CPUS) -m 256
qemu-nox: fs.img xv6.img
    $(QEMU) -nographic $(QEMUOPTS)
.gdbinit: .gdbinit.tmpl
    sed "s/localhost:1234/localhost:$(GDBPORT)/" < $^ > $@
qemu-gdb: fs.img xv6.img .gdbinit
    @echo "*** Now run 'gdb'." 1>&2
    $(QEMU) -serial mon:stdio $(QEMUOPTS) -S $(QEMUGDB)
qemu-nox-gdb: fs.img xv6.img .gdbinit
    @echo "*** Now run 'gdb'." 1>&2
    $(QEMU) -nographic $(QEMUOPTS) -S $(QEMUGDB)
# CUT HERE
# prepare dist for students
# after running make dist, probably want to
# rename it to rev0 or rev1 or so on and then
# check in that version.
EXTRA=\
    mkfs.c ulib.c user.h cat.c echo.c forktest.c grep.c kill.c\
    ln.c ls.c mkdir.c rm.c stressfs.c usertests.c wc.c zombie.c\

```



```

printf.c umalloc.c\
README dot-bochsrc *.pl toc.* runoff runoff1 runoff.list\
.gdbinit.tmpl gdbutil\
dist:
    rm -rf dist
    mkdir dist
    for i in $(FILES); \
    do \
        grep -v PAGEBREAK $$i >dist/$$i; \
    done
    sed '/CUT HERE/,,$$d' Makefile >dist/Makefile
    echo >dist/runoff.spec
    cp $(EXTRA) dist
dist-test:
    rm -rf dist
    make dist
    rm -rf dist-test
    mkdir dist-test
    cp dist/* dist-test
    cd dist-test; $(MAKE) print
    cd dist-test; $(MAKE) bochs || true
    cd dist-test; $(MAKE) qemu
# update this rule (change rev#) when it is time to
# make a new revision.
tar:
    rm -rf /tmp/xv6
    mkdir -p /tmp/xv6
    cp dist/* dist/.gdbinit.tmpl /tmp/xv6
    (cd /tmp; tar cf - xv6) | gzip >xv6-rev10.tar.gz # the next one will be 10 (9/17)
.PHONY: dist-test dist

```

[memdump.c]

```

/*=====
* memdump.c | 전역 물리 프레임 테이블 덤프/필터링 유틸리티
*
* 목적 : 커널 시스템콜 dump_physmem_info()를 통해 물리 프레임 테이블을 사용자 공간으로
* 수집하고, 옵션에 따라 전체/할당만/특정 PID 소유 프레임만 표 형식으로 출력.
* 사용법: memdump [-a] [-p PID]
* -a : free 프레임까지 포함하여 전체 테이블 출력
* -p PID : 지정 PID가 점유 중인 프레임만 출력
* 전체 : 커널에 struct physframe_info { frame_index, allocated, pid, start_tick, ... } 와
* dump_physmem_info(buf, max) 시스템콜이 구현되어 있어야 함
* 규모 : MAX_FRINFO(60000) 엔트리까지 수신(환경에 맞게 조정 가능)
*=====*/

```

```

#include "types.h"
#include "stat.h"
#include "user.h"
#include "fcntl.h"

```

```

#define MAX_FRINFO 60000 // 수신 버퍼 상한(가용 PFN 수보다 크거나 같게 설정 권장)

```

```

/* 잘못된 인자 사용 시 간단한 도움말을 출력하고 종료. */

```

```

static void
usage(void)
{
    printf(1, "usage: memdump [-a] [-p PID]\n");
    printf(1, "    -p PID : 해당 PID가 점유중인 물리 프레임만 출력\n");
}

```

```

printf(1, "    -a      : free 프레임까지 포함해 전체 테이블 출력\n");
exit();
}

int
main(int argc, char *argv[])
{
    int show_all = 0;    // -a 지정 여부
    int filter_pid = 0;  // -p 지정 여부
    int pid_value = -1;  // -p 인자로 받은 대상 PID

    // 옵션 파싱: 허용된 조합은 (-a) 또는 (-p PID) 중 하나이며, 미지정이면 usage 출력
    for (int i = 1; i < argc; i++) {
        if (argv[i][0] != '-') usage();
        if (argv[i][1] == 'a' && argv[i][2] == '\0') {
            show_all = 1;
        } else if (argv[i][1] == 'p' && argv[i][2] == '\0') {
            if (i + 1 >= argc) usage();
            pid_value = atoi(argv[++i]);
            filter_pid = 1;
            if (pid_value <= 0) usage(); // 유효하지 않은 PID 방지
        } else {
            usage();
        }
    }
}

if (argc == 1) usage(); // 옵션 미제공 시 사용법 안내

// 커널로부터 물리 프레임 정보 테이블을 한 번에 받아온다
static struct physframe_info buf[MAX_FRINFO];
int n = dump_physmem_info((void*)buf, MAX_FRINFO);
if (n < 0) {
    printf(1, "memdump: dump_physmem_info failed\n");
    exit();
}

// 실행 모드 헤더와 컬럼명을 출력.
if (filter_pid) {
    printf(1, "[memdump] target pid=%d (self=%d)\n", pid_value, getpid());
} else if (show_all) {
    printf(1, "[memdump] show_all (self=%d)\n", getpid());
} else {
    printf(1, "[memdump] allocated-only (self=%d)\n", getpid());
}
printf(1, "[frame#]\t[alloc]\t[pid]\t[start_tick]\n");

// 본문 출력: 모드에 따라 필터링하여 행을 출력하고, 출력 건수를 집계.
int printed = 0;
for (int i = 0; i < n; i++) {
    int alloc = buf[i].allocated; // 1=할당됨, 0=free
    int owner = buf[i].pid;       // 프레임 소유 PID (free의 경우 구현에 따라 0/-1 등)

    if (filter_pid) {
        if (alloc == 1 && owner == pid_value) {
            printf(1, "%d\t%d\t%d\t%d\n",
                buf[i].frame_index, alloc, owner, buf[i].start_tick);
            printed++;
        }
    } else if (show_all) {

```

```

    printf(1, "%d\t\t%d\t%d\t%d\n",
           buf[i].frame_index, alloc, owner, buf[i].start_tick);
    printed++;
} else { // allocated-only: 할당된 프레임만 출력
    if (alloc == 1) {
        printf(1, "%d\t\t%d\t%d\t%d\n",
               buf[i].frame_index, alloc, owner, buf[i].start_tick);
        printed++;
    }
}
}

// 출력된 행이 없을 때 사용자에게 상황을 안내.
if (printed == 0) {
    if (filter_pid)
        printf(1, "(no frames owned by pid=%d - process may have exited)\n", pid_value);
    else if (!show_all)
        printf(1, "(no allocated frames)\n");
}
exit();
}

[memstress.c]
/*=====
 * memstress.c | 메모리 스트레스/관찰용 사용자 프로그램
 *
 * 목적 : 지정한 개수의 페이지를 sbrk()로 확보하고, 일정 tick 동안 보유한 뒤 종료한다.
 * 기능 : (선택) 각 페이지의 첫 바이트를 실제로 쓰기(-w)하여 페이지 접근과 TLB 채움을 유도한다.
 * 사용법: memstress [-n pages] [-t ticks] [-w]
 *      -n <pages> : 확보할 페이지 수 (기본 31)
 *      -t <ticks> : 보유 시간 ticks (기본 500)
 *      -w          : 각 페이지의 첫 바이트에 실제로 write 수행
 * 비고 : 페이지 크기 4096을 상수로 사용합니다. 환경에 따라 PGSIZE 상수로 대체하는 것이 바람직하다.
 *=====*/

#include "types.h"
#include "stat.h"
#include "user.h"
#include "fcntl.h"

// 잘못된 인자 사용 시 사용법을 안내하고 즉시 종료한다.
static void
usage(void) {
    printf(1, "usage: memstress [-n pages] [-t ticks] [-w]\n");
    printf(1, "  -n <pages> : 확보할 페이지 수 (기본 31)\n");
    printf(1, "  -t <ticks> : 보유 시간 ticks (기본 500)\n");
    printf(1, "  -w          : 각 페이지 첫 바이트에 실제로 write\n");
    exit();
}

int
main(int argc, char *argv[])
{
    int pages = 31;      // 기본 확보 페이지 수
    int hold_ticks = 500; // 기본 보유 시간(틱)
    int do_write = 0;    // -w 옵션 처리(1이면 각 페이지 첫 바이트에 쓰기 수행)

    // 옵션 파싱: 형식 오류나 인자 누락 시 usage() 호출 후 종료한다.

```

```

for (int i = 1; i < argc; i++) {
    if (argv[i][0] != '-') usage();

    if (argv[i][1] == 'n' && argv[i][2] == '\0') {
        // -n <pages> : 확보할 페이지 수 설정
        if (i + 1 >= argc) usage();
        pages = atoi(argv[++i]);
        if (pages <= 0) usage(); // 0 이하 방지
    } else if (argv[i][1] == 't' && argv[i][2] == '\0') {
        // -t <ticks> : 보유 시간(틱) 설정
        if (i + 1 >= argc) usage();
        hold_ticks = atoi(argv[++i]);
        if (hold_ticks < 0) usage(); // 음수 방지
    } else if (argv[i][1] == 'w' && argv[i][2] == '\0') {
        // -w : 각 페이지 첫 바이트에 실제 쓰기 수행
        do_write = 1;
    } else {
        usage();
    }
}

// 실행 구성 로깅: 이후 memdump/tlbstat 등과 함께 관찰 시 유용하다.
int pid = getpid();
printf(1, "[memstress] pid=%d pages=%d hold=%d ticks write=%d\n",
        pid, pages, hold_ticks, do_write);

// 메모리 확보: sbrk로 pages * 4096 바이트만큼 힙을 증가시킵니다. (4096 = 4KiB 페이지 가정)
int inc = pages * 4096;
char *base = sbrk(inc);
if (base == (char*)-1) {
    // 메모리 부족 등으로 sbrk 실패 시 즉시 종료.
    printf(1, "[memstress] sbrk failed\n");
    exit();
}

// -w 옵션: 각 페이지의 첫 바이트를 한 번씩 기록하여 실제 접근을 유도.
// 이는 페이지 워크/프레임 할당 및 TLB 채움을 확실히 발생.
if (do_write) {
    for (int p = 0; p < pages; p++) {
        base[p * 4096] = (char)(p & 0xff); // 페이지 p의 첫 바이트에 식별 가능한 패턴 기록
    }
}

// 보유 단계: 설정한 tick 동안 점유 상태를 유지.
// 이 구간에 memdump -p <pid> 등으로 프레임 점유 여부를 관찰 가능.
sleep(hold_ticks);

// 종료 로그 후 정상 종료합니다.
printf(1, "[memstress] pid=%d done\n", pid);
exit();
}

```

[memtest.c]

```

/*=====
* memtest.c | 메모리 스트레스/관찰 통합 시나리오 오케스트레이터
*
* 목적 : memstress(페이지 확보/보유)와 memdump(프레임 테이블 조회)를
* 시간차로 실행하여, 프레임 할당·보유·해제 흐름을 관찰.

```

- * 흐름 : memstress 두 개를 순차 실행 → 중간에 memdump -p <PID>로 상태 확인 →
 - * 자식 종료 대기 → 마지막으로 재확인(memdump -p 5).
 - * 주의 : 아래 코드는 PID 4, 5를 하드코딩하여 memdump를 실행.
- =====*/

```
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(int argc, char *argv[])
{
    int pid;

    /* 첫 번째 memstress 실행: 31페이지를 500틱 동안 보유하도록 자식 프로세스 생성 */
    pid = fork();
    if(pid < 0){
        printf(1, "fork failed\n");
        exit();
    }

    if(pid == 0){
        char *args[] = { "memstress", "-n", "31", "-t", "500", 0 };
        exec("memstress", args);          // 자식: memstress로 프로그램 교체
        printf(1, "exec memstress failed\n");
        exit();
    }

    sleep(100); /* 완충 대기: 두 번째 memstress가 첫 번째와 겹치는 보유 구간을 만들기 위함 */

    /* 두 번째 memstress 실행: 동일한 조건으로 또 하나의 자식 생성 */
    int pid2 = fork();
    if(pid2 == 0){
        char *args2[] = { "memstress", "-n", "31", "-t", "500", 0 };
        exec("memstress", args2);
        printf(1, "exec memstress failed\n");
        exit();
    }

    sleep(100); /* 완충 대기: 이후 memdump가 두 memstress의 보유 구간 안에서 실행되도록 조정 */

    /* 첫 번째 관찰: PID=4의 프레임만 출력 (하드코딩) */
    int pid3 = fork();
    if(pid3 < 0){
        printf(1, "fork failed\n");
        exit();
    }

    if(pid3 == 0){
        char *args3[] = { "memdump", "-p", "4", 0 };
        exec("memdump", args3);          // 자식: memdump -p 4 실행
        printf(1, "exec memdump failed\n");
        exit();
    }

    sleep(100); /* 완충 대기: 다음 관찰과의 시간 간격 확보 */

    /* 두 번째 관찰: PID=5의 프레임만 출력 (하드코딩) */
```

```

int pid4 = fork();
if(pid4 < 0){
    printf(1, "fork failed\n");
    exit();
}

if(pid4 == 0){
    char *args4[] = { "memdump", "-p", "5", 0 };
    exec("memdump", args4);
    printf(1, "exec memdump failed\n");
    exit();
}

/* 앞서 생성한 네 개의 자식(memstress×2, memdump×2) 종료 대기 */
wait();
wait();
wait();
wait();

sleep(100); /* 정리 후 재검증 전 짧은 대기: 해제/무효화가 반영될 시간을 줌 */

/* 최종 확인: PID=5의 프레임이 해제되었는지 재확인 (보통 0행 기대) */
int pid5 = fork();
if(pid5 < 0){
    printf(1, "fork failed\n");
    exit();
}

if(pid5 == 0){
    char *args5[] = { "memdump", "-p", "5", 0 };
    exec("memdump", args5);
    printf(1, "exec memdump failed\n");
    exit();
}

wait(); /* 마지막 memdump 자식 종료 대기 */

exit();
}

```

```

[physmem.c]
// physmem.h
#ifdef PHYSMEM_H
#define PHYSMEM_H

#include "types.h" // xv6의 uint 정의

#define PFNNUM 60000

struct physframe_info {
    uint frame_index; // 물리 프레임 번호 (pa >> 12)
    int allocated;    // 1=할당됨, 0=free
    int pid;          // 소유 PID, 없으면 -1
    uint start_tick;  // 할당 시작 tick
};

#endif
[syscall.c]

```

```

#include "types.h"
#include "defs.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "proc.h"
#include "x86.h"
#include "syscall.h"

// User code makes a system call with INT T_SYSCALL.
// System call number in %eax.
// Arguments on the stack, from the user call to the C
// library system call function. The saved user %esp points
// to a saved program counter, and then the first argument.

// Fetch the int at addr from the current process.
int
fetchint(uint addr, int *ip)
{
    struct proc *curproc = myproc();

    if(addr >= curproc->sz || addr+4 > curproc->sz)
        return -1;
    *ip = *(int*)(addr);
    return 0;
}

// Fetch the nul-terminated string at addr from the current process.
// Doesn't actually copy the string - just sets *pp to point at it.
// Returns length of string, not including nul.
int
fetchstr(uint addr, char **pp)
{
    char *s, *ep;
    struct proc *curproc = myproc();

    if(addr >= curproc->sz)
        return -1;
    *pp = (char*)addr;
    ep = (char*)curproc->sz;
    for(s = *pp; s < ep; s++){
        if(*s == 0)
            return s - *pp;
    }
    return -1;
}

// Fetch the nth 32-bit system call argument.
int
argint(int n, int *ip)
{
    return fetchint((myproc()->tf->esp) + 4 + 4*n, ip);
}

// Fetch the nth word-sized system call argument as a pointer
// to a block of memory of size bytes. Check that the pointer
// lies within the process address space.
int

```

```

argptr(int n, char **pp, int size)
{
    int i;
    struct proc *curproc = myproc();

    if(argint(n, &i) < 0)
        return -1;
    if(size < 0 || (uint)i >= curproc->sz || (uint)i+size > curproc->sz)
        return -1;
    *pp = (char*)i;
    return 0;
}

// Fetch the nth word-sized system call argument as a string pointer.
// Check that the pointer is valid and the string is nul-terminated.
// (There is no shared writable memory, so the string can't change
// between this check and being used by the kernel.)
int
argstr(int n, char **pp)
{
    int addr;
    if(argint(n, &addr) < 0)
        return -1;
    return fetchstr(addr, pp);
}

extern int sys_chdir(void);
extern int sys_close(void);
extern int sys_dup(void);
extern int sys_exec(void);
extern int sys_exit(void);
extern int sys_fork(void);
extern int sys_fstat(void);
extern int sys_getpid(void);
extern int sys_kill(void);
extern int sys_link(void);
extern int sys_mkdir(void);
extern int sys_mknod(void);
extern int sys_open(void);
extern int sys_pipe(void);
extern int sys_read(void);
extern int sys_sbrk(void);
extern int sys_sleep(void);
extern int sys_unlink(void);
extern int sys_wait(void);
extern int sys_write(void);
extern int sys_uptime(void);
extern int sys_dump_physmem_info(void);

static int (*syscalls[])(void) = {
[SYS_fork]    sys_fork,
[SYS_exit]    sys_exit,
[SYS_wait]    sys_wait,
[SYS_pipe]    sys_pipe,
[SYS_read]    sys_read,
[SYS_kill]    sys_kill,
[SYS_exec]    sys_exec,
[SYS_fstat]    sys_fstat,

```



```

[SYS_chdir]    sys_chdir,
[SYS_dup]     sys_dup,
[SYS_getpid]  sys_getpid,
[SYS_sbrk]    sys_sbrk,
[SYS_sleep]   sys_sleep,
[SYS_uptime]  sys_uptime,
[SYS_open]    sys_open,
[SYS_write]   sys_write,
[SYS_mknod]   sys_mknod,
[SYS_unlink]  sys_unlink,
[SYS_link]    sys_link,
[SYS_mkdir]   sys_mkdir,
[SYS_close]   sys_close,
[SYS_dump_physmem_info] sys_dump_physmem_info,
};

```

```

void
syscall(void)
{
    int num;
    struct proc *curproc = myproc();

    num = curproc->tf->eax;
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        curproc->tf->eax = syscalls[num]();
    } else {
        cprintf("%d %s: unknown sys call %d\n",
            curproc->pid, curproc->name, num);
        curproc->tf->eax = -1;
    }
}

```

```

[syscall.h]
// System call numbers
#define SYS_fork    1
#define SYS_exit    2
#define SYS_wait    3
#define SYS_pipe    4
#define SYS_read    5
#define SYS_kill    6
#define SYS_exec    7
#define SYS_fstat    8
#define SYS_chdir    9
#define SYS_dup     10
#define SYS_getpid  11
#define SYS_sbrk    12
#define SYS_sleep   13
#define SYS_uptime  14
#define SYS_open    15
#define SYS_write   16
#define SYS_mknod   17
#define SYS_unlink  18
#define SYS_link    19
#define SYS_mkdir   20
#define SYS_close   21
#define SYS_dump_physmem_info 22

```

```

[sysproc.c]

```

```

#include "types.h"
#include "x86.h"
#include "defs.h"
#include "date.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "proc.h"
#include "physmem.h"    // struct physframe_info 정의가 보이게 추가

```

```

int
sys_fork(void)
{
    return fork();
}

```

```

int
sys_exit(void)
{
    exit();
    return 0;    // not reached
}

```

```

int
sys_wait(void)
{
    return wait();
}

```

```

int
sys_kill(void)
{
    int pid;

    if(argint(0, &pid) < 0)
        return -1;
    return kill(pid);
}

```

```

int
sys_getpid(void)
{
    return myproc()->pid;
}

```

```

int
sys_sbrk(void)
{
    int addr;
    int n;

    if(argint(0, &n) < 0)
        return -1;
    addr = myproc()->sz;
    if(growproc(n) < 0)
        return -1;
    return addr;
}

```

```

int
sys_sleep(void)
{
    int n;
    uint ticks0;

    if(argint(0, &n) < 0)
        return -1;
    acquire(&tickslock);
    ticks0 = ticks;
    while(ticks - ticks0 < n){
        if(myproc()->killed){
            release(&tickslock);
            return -1;
        }
        sleep(&ticks, &tickslock);
    }
    release(&tickslock);
    return 0;
}

// return how many clock tick interrupts have occurred
// since start.
int
sys_uptime(void)
{
    uint xticks;

    acquire(&tickslock);
    xticks = ticks;
    release(&tickslock);
    return xticks;
}

// 전역 물리 페이지 관리 테이블 관련 시스템콜
int
sys_dump_physmem_info(void)
{
    char *uaddr;
    int maxe;

    // 1) 두 번째 인자부터 먼저 읽어서 크기 결정
    if (argint(1, &maxe) < 0) return -1;
    if (maxe <= 0) return -1;

    // 2) 첫 번째 인자(버퍼 포인터) 유효성 검사: 필요한 총 바이트 수로 체크
    if (argptr(0, &uaddr, maxe * sizeof(struct physframe_info)) < 0) return -1;

    // 3) 실제 작업 호출
    return dump_physmem_info_kernel(uaddr, maxe);
}

[user.h]
#include "physmem.h"

struct stat;
struct rtcdate;

```

```

// system calls
int fork(void);
int exit(void) __attribute__((noreturn));
int wait(void);
int pipe(int*);
int write(int, const void*, int);
int read(int, void*, int);
int close(int);
int kill(int);
int exec(char*, char**);
int open(const char*, int);
int mknod(const char*, short, short);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int getpid(void);
char* sbrk(int);
int sleep(int);
int uptime(void);

// ulib.c
int stat(const char*, struct stat*);
char* strcpy(char*, const char*);
void *memmove(void*, const void*, int);
char* strchr(const char*, char c);
int strcmp(const char*, const char*);
void printf(int, const char*, ...);
char* gets(char*, int max);
uint strlen(const char*);
void* memset(void*, int, uint);
void* malloc(uint);
void free(void*);
int atoi(const char*);

// #include "phymem.h" // 유저 프로그램, 커널이 모두 같은 구조체 사용하도록 공용 헤더 추가
int dump_phymem_info(void *addr, int max_entries); // 시스템콜 프로토타입

[usys.S]
#include "syscall.h"
#include "traps.h"

#define SYSCALL(name) \
.globl name; \
name: \
    movl $SYS_ ## name, %eax; \
    int $T_SYSCALL; \
    ret

SYSCALL(fork)
SYSCALL(exit)
SYSCALL(wait)
SYSCALL(pipe)

```

```

SYSCALL(read)
SYSCALL(write)
SYSCALL(close)
SYSCALL(kill)
SYSCALL(exec)
SYSCALL(open)
SYSCALL(mknod)
SYSCALL(unlink)
SYSCALL(fstat)
SYSCALL(link)
SYSCALL(mkdir)
SYSCALL(chdir)
SYSCALL(dup)
SYSCALL(getpid)
SYSCALL(sbrk)
SYSCALL(sleep)
SYSCALL(uptime)
SYSCALL(dump_physmem_info)

```

[소스코드_추가구현(C)]

[alias2map.c]

```

#include "types.h"
#include "user.h"
#include "vlist.h"
#include "stat.h"

```

```

#define PGSIZE 4096
#define PTE_U  0x004
#define PTE_W  0x002

```

```

// 이미 있다면 생략: sw_vtop 래퍼
//extern int vtop_sys(void *va, uint *pa_out, uint *flags_out);

```

```

int
main(void)
{
    // 1) 원본 페이지 하나 확보
    char *p = sbrk(PGSIZE);
    if ((int)p == -1) { printf(1, "sbrk fail\n"); exit(); }
    p[0] = 0x5A; // A/D 비트 세우고 내용도 기록

    // 2) sw_vtop로 물리 페이지 얻기
    uint pa, fl;
    if (vtop(p, &pa, &fl) < 0) { printf(1, "vtop_sys fail\n"); exit(); }
    uint pa_page = pa & ~0xFFF;
    printf(1, "base VA=0x%x -> pa_page=0x%x flags=0x%x\n", (uint)p, pa_page, fl);

    // 3) 비어 있을 법한 다른 VA 선택(원본으로부터 4페이지 뒤)
    uint va2 = ((uint)p) + 4*PGSIZE;
    va2 &= ~0xFFF;

    // 4) 같은 물리페이지를 다른 VA에 추가 매핑 (U|W)
    if (vm_map_user_page(va2, pa_page, PTE_U | PTE_W) < 0) {
        printf(1, "vm_map_user_page fail\n"); exit();
    }
}

```

```

printf(1, "aliased map: va2=0x%x -> same pa_page=0x%x\n", va2, pa_page);

// 5) alias 확인(동일 데이터가 보이는지)
char *q = (char*)va2;
printf(1, "before: p[0]=0x%x q[0]=0x%x\n", p[0] & 0xFF, q[0] & 0xFF);
q[0] ^= 0xFF; // 뒤집기
printf(1, "after : p[0]=0x%x q[0]=0x%x (should be same)\n", p[0] & 0xFF, q[0] & 0xFF);

// 6) phys2virt로 IPT 체인 확인
struct vlist buf[8];
int n = phys2virt(pa_page, buf, 8);
if (n < 0) { printf(1, "phys2virt fail\n"); exit(); }

printf(1, "phys2virt(pa_page=0x%x) -> %d entries\n", pa_page, n);
for (int i=0; i<n; i++)
    printf(1, " [%d] pid=%d va=0x%x flags=0x%x\n", i, buf[i].pid, buf[i].va_page, buf[i].flags);

// 기대: 같은 pid로 서로 다른 va 두 줄 이상( $\geq 2$ )
exit();
}
[defs.h]
#include "types.h"

struct buf;
struct context;
struct file;
struct inode;
struct pipe;
struct proc;
struct rtcdate;
struct spinlock;
struct sleeplock;
struct stat;
struct superblock;

// bio.c
void          binit(void);
struct buf*   bread(uint, uint);
void          brelse(struct buf*);
void          bwrite(struct buf*);

// console.c
void          consoleinit(void);
void          cprintf(char*, ...);
void          consoleintr(int(*)(void));
void          panic(char*) __attribute__((noreturn));

// exec.c
int           exec(char*, char**);

// file.c
struct file*  filealloc(void);
void          fileclose(struct file*);
struct file*  filedup(struct file*);
void          fileinit(void);
int           fileread(struct file*, char*, int n);
int           filestat(struct file*, struct stat*);
int           filewrite(struct file*, char*, int n);

```

```

// fs.c
void      readsb(int dev, struct superblock *sb);
int       dirlink(struct inode*, char*, uint);
struct inode*  dirlookup(struct inode*, char*, uint*);
struct inode*  ialloc(uint, short);
struct inode*  idup(struct inode*);
void        iinit(int dev);
void        ilock(struct inode*);
void        iput(struct inode*);
void        iunlock(struct inode*);
void        iunlockput(struct inode*);
void        iupdate(struct inode*);
int         namecmp(const char*, const char*);
struct inode*  namei(char*);
struct inode*  nameiparent(char*, char*);
int         readi(struct inode*, char*, uint, uint);
void        stati(struct inode*, struct stat*);
int         writei(struct inode*, char*, uint, uint);

```

```

// ide.c
void        ideinit(void);
void        ideintr(void);
void        iderw(struct buf*);

```

```

// ioapic.c
void        ioapicenable(int irq, int cpu);
extern uchar ioapicid;
void        ioapicinit(void);

```

```

// kalloc.c
char*       kalloc(void);
void        kfree(char*);
void        kinit1(void*, void*);
void        kinit2(void*, void*);

```

```

// kbd.c
void        kbdintr(void);

```

```

// lapic.c
void        cmostime(struct rtcdate *r);
int         lapicid(void);
extern volatile uint* lapic;
void        lapiceoi(void);
void        lapicinit(void);
void        lapicstartap(uchar, uint);
void        microdelay(int);

```

```

// log.c
void        initlog(int dev);
void        log_write(struct buf*);
void        begin_op();
void        end_op();

```

```

// mp.c
extern int  ismp;
void        mpinit(void);

```

```

// picirq.c
void      picenable(int);
void      picinit(void);

// pipe.c
int        pipealloc(struct file**, struct file**);
void       pipeclose(struct pipe*, int);
int        piperead(struct pipe*, char*, int);
int        pipewrite(struct pipe*, char*, int);

//PAGEBREAK: 16
// proc.c
int        cpuid(void);
void       exit(void);
int        fork(void);
int        growproc(int);
int        kill(int);
struct cpu* mycpu(void);
struct proc* myproc();
void       pinit(void);
void       procdump(void);
void       scheduler(void) __attribute__((noreturn));
void       sched(void);
void       setproc(struct proc*);
void       sleep(void*, struct spinlock*);
void       userinit(void);
int        wait(void);
void       wakeup(void*);
void       yield(void);

// swtch.S
void       swtch(struct context**, struct context*);

// spinlock.c
void       acquire(struct spinlock*);
void       getcallerpcs(void*, uint*);
int        holding(struct spinlock*);
void       initlock(struct spinlock*, char*);
void       release(struct spinlock*);
void       pushcli(void);
void       popcli(void);

// sleeplock.c
void       acquiresleep(struct sleeplock*);
void       releasesleep(struct sleeplock*);
int        holdingsleep(struct sleeplock*);
void       initsleeplock(struct sleeplock*, char*);

// string.c
int        memcmp(const void*, const void*, uint);
void*      memmove(void*, const void*, uint);
void*      memset(void*, int, uint);
char*      safestrcpy(char*, const char*, int);
int        strlen(const char*);
int        strncmp(const char*, const char*, uint);
char*      strncpy(char*, const char*, int);

// syscall.c

```



```

int      argint(int, int*);
int      argptr(int, char**, int);
int      argstr(int, char**);
int      fetchint(uint, int*);
int      fetchstr(uint, char**);
void     syscall(void);

// timer.c
void     timerinit(void);

// trap.c
void     idtinit(void);
extern uint ticks;
void     tvinit(void);
extern struct spinlock tickslock;

// uart.c
void     uartinit(void);
void     uartintr(void);
void     uartputc(int);

// vm.c
void     seginit(void);
void     kvmalloc(void);
pde_t*   setupkvm(void);
char*    uva2ka(pde_t*, char*);
int      allocuvm(pde_t*, uint, uint);
int      deallocuvm(pde_t*, uint, uint);
void     freevm(pde_t*);
void     inituvm(pde_t*, char*, uint);
int      loaduvm(pde_t*, char*, struct inode*, uint, uint);
pde_t*   copyuvm(pde_t*, uint);
void     switchuvm(struct proc*);
void     switchkvm(void);
int      copyout(pde_t*, uint, void*, uint);
void     clearpteu(pde_t *pgdir, char *uva);

// dump_physmem_info() 커널 내부 함수 버전 선언
int dump_physmem_info_kernel(char *uaddr, int max_entries); // 전역 프레임 테이블을 사용자 버퍼(uaddr)로 복사

// defs.h
int sw_vtop(pde_t *pgdir, const void *va, uint32_t *pa_out, uint32_t *pte_flags_out); // 소프트웨어 vtop: va→pa 변환 및
PTE 플래그 반환

// 디버그용 walkpgdir 래퍼
int hw_walk_probe(pde_t *pgdir, const void *va, uint *pa_out, uint *flags_out); // 하드웨어 경로 모사/검증용 페이지
워크 래퍼

// 역페이지 테이블
void ipt_init(void); // IPT 초기화
void ipt_add(pde_t *pgdir, uint32_t pid, uint32_t va_page, uint32_t pa, uint32_t pte_flags); // (pid,va_page)→pa 매핑
추가
int ipt_remove(pde_t *pgdir, uint32_t pid, uint32_t va_page, uint32_t pa); // 매핑 제거 및 체인/참조
정리
struct ipt_entry *ipt_lookup(uint32_t pid, uint32_t va_page); // (pid,va_page) 조회
uint32_t ipt_count_pfn(uint32_t pfn); // 특정 PFN을 참조하는
엔트리 수

```

```

int vm_map_user_page(pde_t *pgdir, uint va_page, uint pa_page, int perm);           // 사용자 VA 페이지에 물리 페이지
매핑

int protect_nowrite(pde_t *pgdir, void *uva);                                   // 해당 VA의 쓰기 권한 제거(RO 보호)
및 필요 시 TLB 무효화
int vm_unmap_one(pde_t *pgdir, uint va_any, int free_if_last, int *remaining_out); // 단일 VA 언맵; 마지막 참조면 해제,
남은 참조수 반환

// number of elements in fixed-size array
#define NELEM(x) (sizeof(x)/sizeof((x)[0]))
[exec.c]
#include "types.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "proc.h"
#include "defs.h"
#include "x86.h"
#include "elf.h"
#include "tlb.h"
#include "ipt.h"

int
exec(char *path, char **argv)
{
    char *s, *last;
    int i, off;
    uint argc, sz, sp, ustack[3+MAXARG+1];
    struct elfhdr elf;
    struct inode *ip;
    struct proghdr ph;
    pde_t *pgdir, *oldpgdir;
    struct proc *curproc = myproc();

    begin_op();

    if((ip = namei(path)) == 0){
        end_op();
        cprintf("exec: fail\n");
        return -1;
    }
    ilock(ip);
    pgdir = 0;

    // ELF 헤더 확인
    if(readi(ip, (char*)&elf, 0, sizeof(elf)) != sizeof(elf)) goto bad;
    if(elf.magic != ELF_MAGIC) goto bad;

    if((pgdir = setupkvm()) == 0) goto bad;

    // 프로그램 로드
    sz = 0;
    for(i=0, off=elf.phoff; i<elf.phnum; i++, off+=sizeof(ph)){
        if(readi(ip, (char*)&ph, off, sizeof(ph)) != sizeof(ph)) goto bad;
        if(ph.type != ELF_PROG_LOAD) continue;
        if(ph.memsz < ph.filesz) goto bad;
        if(ph.vaddr + ph.memsz < ph.vaddr) goto bad;
        if((sz = allocvm(pgdir, sz, ph.vaddr + ph.memsz)) == 0) goto bad;

```

```

    if(ph.vaddr % PGSIZE != 0) goto bad;
    if(loaduvm(pgdir, (char*)ph.vaddr, ip, ph.off, ph.filesz) < 0) goto bad;
}
iunlockput(ip); end_op(); ip = 0;

// 스택 2페이지(가드+유저)
sz = PGROUNDUP(sz);
if((sz = allocuvm(pgdir, sz, sz + 2*PGSIZE)) == 0) goto bad;
clearpteu(pgdir, (char*)(sz - 2*PGSIZE));
sp = sz;

// argv 복사
for(argc = 0; argv[argc]; argc++){
    if(argc >= MAXARG) goto bad;
    sp = (sp - (strlen(argv[argc]) + 1)) & ~3;
    if(copyout(pgdir, sp, argv[argc], strlen(argv[argc]) + 1) < 0) goto bad;
    ustack[3+argc] = sp;
}
ustack[3+argc] = 0;
ustack[0] = 0xffffffff; // fake return PC
ustack[1] = argc;
ustack[2] = sp - (argc+1)*4; // argv pointer
sp -= (3+argc+1) * 4;
if(copyout(pgdir, sp, ustack, (3+argc+1)*4) < 0) goto bad;

// 프로그램 이름 저장
for(last=s=path; *s; s++) if(*s == '/') last = s+1;
safestrcpy(curproc->name, last, sizeof(curproc->name));

// 커밋
oldpgdir = curproc->pgdir;
curproc->pgdir = pgdir;
curproc->sz = sz;
curproc->tf->eip = elf.entry;
curproc->tf->esp = sp;
// cprintf("[exec] %s: sz=0x%x sp=0x%x\n", last, sz, sp);

// 새 주소공간으로 하드웨어 전환
switchvm(curproc);

// (A) 소프트 TLB: 같은 PID지만 주소공간이 완전히 바뀌었으므로 전부 무효화
tlb_invalidate_pid(curproc->pid);

// (B) 이전 주소공간의 IPT 엔트리 정리 후 메모리 해제
ipt_scrub_for_pgdir(curproc->pid, oldpgdir);
freevm(oldpgdir);

// (C) 새 주소공간을 IPT에 재구축
ipt_rebuild_for_pgdir(curproc->pid, curproc->pgdir);

return 0;

bad:
if(pgdir) freevm(pgdir);
if(ip){ iunlockput(ip); end_op(); }
return -1;
}

```

```

[ipt.c]
// ipt.c
#include "types.h"
#include "defs.h"
#include "mmu.h"
#include "spinlock.h"
#include "proc.h"
#include "ipt.h"
#include "param.h"
#include "memlayout.h"

// 전역 심볼 (명세 스타일)
struct ipt_entry *ipt_hash[IPT_BUCKETS];
struct spinlock ipt_lock;

static int count_pfn_locked(uint32_t pfn);

// 간단한 해시 (버킷수가 2^n일 때 마스킹)
static inline uint32_t hash_u32(uint32_t x){
    x ^= x >> 16; x *= 0x7feb352d; x ^= x >> 15; x *= 0x846ca68b; x ^= x >> 16;
    return x & (IPT_BUCKETS - 1);
}

// 키 선택: (pid, va_page)를 키로 사용
static inline uint32_t ipt_bucket_idx(uint32_t pid, uint32_t va_page){
    return hash_u32(pid ^ va_page);
}

void ipt_init(void){
    initlock(&ipt_lock, "ipt");
    for(int i=0;i<IPT_BUCKETS;i++) ipt_hash[i] = 0;
}

// 락 잡힌 상태에서 체인에서 찾기
static struct ipt_entry * find_locked(uint32_t idx, uint32_t pid, uint32_t va_page){
    for(struct ipt_entry *e = ipt_hash[idx]; e; e = e->next){
        if(e->pid == pid && e->va == va_page) return e;
    }
    return 0;
}

void ipt_add(pde_t *pgdir, uint32_t pid, uint32_t va_page, uint32_t pa, uint32_t pte_flags){
    if(pgdir == 0) return;
    // 커널 매핑은 제외(유저 매핑만 추적) - 과제 의도/관리 난이도 측면
    if((pte_flags & PTE_U) == 0) return;
    if(pa == 0) return;

    uint32_t idx = ipt_bucket_idx(pid, va_page);

    acquire(&ipt_lock);
    struct ipt_entry *e = find_locked(idx, pid, va_page);
    if(e){
        // 동일한 (pid,va_page) 매핑이 또 들어오면 refcnt만 증가
        e->refcnt++;
        // 스냅샷/동기화 차원에서 최신 pfn/flags 갱신
        e->pfn = PFN(pa);
        e->flags = (uint16_t)(pte_flags & 0x0FFF);
        release(&ipt_lock);
    }
}

```

```

    return:
}
// 새 엔트리 할당
e = (struct ipt_entry*)kalloc();
if(!e){ release(&ipt_lock); return: }
// 엔트리는 페이지 크기지만 단일 구조체만 사용 - 간단히 0으로
memset(e, 0, PGSIZE);
e->pfn    = PFN(pa);
e->pid    = pid;
e->va     = va_page;
e->flags = (uint16_t)(pte_flags & 0x0FFF);
e->refcnt = 1;
e->next   = ipt_hash[idx];
ipt_hash[idx] = e;
release(&ipt_lock);
}

// 제거 후 "해당 PFN의 남은 총 참조수"를 반환
// - 엔트리를 못 찾은 경우에도 현재 PFN 총 참조수를 반환(=상태 보고용)
// - 호출 전후 모두 락 정합성을 유지
int
ipt_remove(pde_t *pgdir, uint32_t pid, uint32_t va_page, uint32_t pa_page)
{
    // 1) 비정상 인자: 해제 금지 방향(양수)로 반환
    if (pgdir == 0)
        return 1;

    // 2) 방어적 정렬 보정
    va_page &= ~(PGSIZE - 1);

    // 3) 커널 VA 가드(방어): 우리 테이블은 유저 매핑만 관리
    if (va_page >= KERNBASE) {
        acquire(&ipt_lock);
        int remain = count_pfn_locked(PFN(pa_page));
        release(&ipt_lock);
        return remain > 0 ? remain : 1;
    }

    uint32_t pfn = PFN(pa_page);
    uint32_t idx = ipt_bucket_idx(pid, va_page);

    acquire(&ipt_lock);

    struct ipt_entry **pp = &ipt_hash[idx];
    struct ipt_entry *e;

    while ((e = *pp) != 0) {
        if (e->pid == pid && e->va == va_page) {

            if (e->refcnt > 1) {
                e->refcnt--;
                int remain = count_pfn_locked(pfn); // 감소 반영 후
                release(&ipt_lock);
                return remain;
            }

            // refcnt == 1 → 제거
            *pp = e->next;

```

```

    kfree((char*)e);

    int remain = count_pfn_locked(pfn);    // 제거 반영 후
    release(&ipt_lock);
    return remain;
}
pp = &e->next;
}

// 못 찾았으면 음수 반환 (예: -2)
release(&ipt_lock);
return -2;
}

struct ipt_entry *ipt_lookup(uint32_t pid, uint32_t va_page){
    uint32_t idx = ipt_bucket_idx(pid, va_page);
    acquire(&ipt_lock);
    struct ipt_entry *e = find_locked(idx, pid, va_page);
    // 주의: 락 없이 포인터 반환 → 디버깅용으로만 사용 (출력 직전 락 재획득 권장)
    release(&ipt_lock);
    return e;
}

uint32_t ipt_count_pfn(uint32_t pfn){
    uint32_t cnt = 0;
    acquire(&ipt_lock);
    for(int i=0;i<IPT_BUCKETS;i++){
        for(struct ipt_entry *e = ipt_hash[i]; e; e = e->next){
            if(e->pfn == pfn) cnt += e->refcnt;
        }
    }
    release(&ipt_lock);
    return cnt;
}

// 유저 영역 PDE 범위: [0 .. KERNBASE)
static inline uint
user_pdx_end(void)
{
    return (KERNBASE >> PDXSHIFT);
}

// 새 pgdir을 스캔하여 (pid, va_page) → (pa_page, flags) 를 IPT에 채운다
// - pid == 0 이면 아무 것도 하지 않습니다(식별 불가).
// - 유저 매핑(PTE_U)만 넣는다
// - 이미 들어있는 (pid,va_page)는 ipt_add()의 중복 처리(refcnt++)/갱신 로직에 맡긴다.
void
ipt_rebuild_for_pgdir(uint32_t pid, pde_t *pgdir)
{
    if (pid == 0 || pgdir == 0)
        return;

    const uint end_pdx = user_pdx_end();

    for (uint pdx = 0; pdx < end_pdx; pdx++) {
        pde_t pde = pgdir[pdx];

```

```

if ((pde & PTE_P) == 0)
    continue; // 이 PDE에는 PT 자체가 없음

// PDE가 가리키는 페이지 테이블(물리) → 커널가상으로 변환
pte_t *pt = (pte_t*)P2V(PTE_ADDR(pde));

for (uint ptx = 0; ptx < NPTENTRIES; ptx++) {
    pte_t pte = pt[ptx];
    if ((pte & PTE_P) == 0)
        continue; // 엔트리 없음
    if ((pte & PTE_U) == 0)
        continue; // 커널 전용 매핑은 추적 안 함

    uint32_t va_page = PGADDR(pdx, ptx, 0); // 페이지 경계
    uint32_t pa_page = PTE_ADDR(pte);      // 페이지 프레임 물리주소
    uint32_t flags   = (pte & 0x0FFF);      // 하위 12비트(perm/A/D 등)

    // 내부에서 ipt_lock을 잡고, 필요시 kalloc을 호출
    // 중복 키(pid,va_page)면 refcnt++ 또는 최신 flags/pfn으로 갱신
    ipt_add(pgdir, pid, va_page, pa_page, flags);
}
}
}

// 기존 pgdir의 모든 유저 매핑을 IPT에서 제거
// exec에서 oldpgdir 정리나 테스트용으로 씬
// - pid == 0이면 제거 대상 pid를 특정할 수 없으므로 아무 것도 안 함.
void
ipt_scrub_for_pgdir(uint32_t pid, pde_t *pgdir)
{
    if (pid == 0 || pgdir == 0)
        return;

    const uint end_pdx = user_pdx_end();

    for (uint pdx = 0; pdx < end_pdx; pdx++) {
        pde_t pde = pgdir[pdx];
        if ((pde & PTE_P) == 0)
            continue;

        pte_t *pt = (pte_t*)P2V(PTE_ADDR(pde));
        for (uint ptx = 0; ptx < NPTENTRIES; ptx++) {
            pte_t pte = pt[ptx];
            if ((pte & PTE_P) == 0)
                continue;
            if ((pte & PTE_U) == 0)
                continue;

            uint32_t va_page = PGADDR(pdx, ptx, 0);
            uint32_t pa_page = PTE_ADDR(pte);

            // 내부에서 ipt_lock을 잡는다.
            ipt_remove(pgdir, pid, va_page, pa_page);
        }
    }
}

static int

```

```

count_pfn_locked(uint32_t pfn)
{
    int cnt = 0;
    // ipt_lock이 이미 잡혀 있다고 가정
    for (int i = 0; i < IPT_BUCKETS; i++) {
        for (struct ipt_entry *x = ipt_hash[i]; x; x = x->next) {
            if (x->pfn == pfn)
                cnt += x->refcnt;
        }
    }
    return cnt;
}

int
ipt_remove_fallback(uint32_t pid, uint32_t va_page, uint32_t pfn)
{
    acquire(&ipt_lock);

    for (int i = 0; i < IPT_BUCKETS; i++) {
        struct ipt_entry **pp = &ipt_hash[i];
        while (*pp) {
            struct ipt_entry *e = *pp;

            // ★ pid, va, pfn 모두 일치할 때만 제거/감소
            if (e->pid == pid && e->va == va_page && e->pfn == pfn) {
                if (e->refcnt > 1) {
                    e->refcnt--;
                    int remain = count_pfn_locked(pfn);
                    release(&ipt_lock);
                    return remain;
                } else {
                    *pp = e->next;
                    kfree((char*)e);
                    int remain = count_pfn_locked(pfn);
                    release(&ipt_lock);
                    return remain;
                }
            }
            pp = &e->next;
        }
    }

    // 못 찾음: 물리 해제 방지를 위해 음수 반환
    release(&ipt_lock);
    return -2;
}

```

```

[ipt.h]
// ipt.h
#pragma once
#include "types.h"
#include "mmu.h"
#include "defs.h"
// #include "spinlock.h"

#ifdef IPT_BUCKETS
#define IPT_BUCKETS 1024           // 2의 거듭제곱 (마스킹 쉬움)

```



```

#endif

// 물리주소 → PFN
#define PFN(pa) ((uint32_t)(pa) >> 12)
// VA 페이지 경계
#define VAPAGE(va) ((uint32_t)(va) & ~0xFFF)

struct ipt_entry {
    uint32_t pfn;        // 물리 프레임 번호 (pa >> 12)
    uint32_t pid;        // 소유 프로세스 PID
    uint32_t va;         // 매핑된 가상주소(페이지 기준)
    uint16_t flags;      // PTE 권한 스냅샷 (P/W/U 등)
    uint16_t refcnt;     // 역참조 카운트(옵션) - 중복 삽입 시 증가
    struct ipt_entry *next; // 해시 체인
};

// 명세가 요구하는 전역 버킷 배열
extern struct ipt_entry *ipt_hash[IPT_BUCKETS];

// 전역 락(간단히 1개)
extern struct spinlock ipt_lock;

// API
void ipt_init(void);
// 새 pgdir을 전범위 스캔하여 IPT를 재구축
void ipt_rebuild_for_pgdir(uint32_t pid, pde_t *pgdir);
// (선택) old pgdir의 모든 유저 매핑을 IPT에서 제거
void ipt_scrub_for_pgdir(uint32_t pid, pde_t *pgdir);
// pgdir->pid 역매핑 헬퍼로 항상 정확한 pid 확보
int ipt_pid_from_pgdir(pde_t *pgdir);

void ipt_add(pde_t *pgdir, uint32_t pid, uint32_t va_page, uint32_t pa, uint32_t pte_flags);
int ipt_remove(pde_t *pgdir, uint32_t pid, uint32_t va_page, uint32_t pa_page);

struct ipt_entry *ipt_lookup(uint32_t pid, uint32_t va_page); // 편의
uint32_t ipt_count_pfn(uint32_t pfn); // 편의

int ipt_remove_fallback(uint32_t pid, uint32_t va_page, uint32_t pfn);

[ipt_tlb_proof.c]
// ipt_tlb_proof.c (TLB 정량 검증 포함 버전)
#include "types.h"
#include "stat.h"
#include "user.h"
#include "mmu.h"
#include "vlist.h"

// 필요시: user.h에 없더라도 이 파일에서 프로토타입 선언
extern int sw_tlb_reset(int mode); // 0: counters만 0, 1: flush+0
extern int sw_tlb_read(struct tlb_stats *out); // 커널에서 통계 복사

static void print_tlb(const char *tag) {
    struct tlb_stats s;
    if (sw_tlb_read(&s) < 0) {
        printf(1, "[tlb %s] read failed\n", tag);
        return;
    }
}

```

```

uint hitp = s.lookups ? (s.hits * 100) / s.lookups : 0;
printf(1,
    "[tlb %s] hits=%d misses=%d hit%%=%d lookups=%d evicts=%d invalidations=%d lines=%d ways=%d sets=%d\n",
    tag, s.hits, s.misses, hitp, s.lookups, s.evicts, s.invalidations, s.lines, s.ways, s.sets);
}

static void pfind(uint pa_page){
    struct vlist buf[16];
    int n = phys2virt(pa_page, buf, 16);
    printf(1, "[pfind] pa=0x%x -> %d entries\n", pa_page, n);
    for(int i=0;i<n;i++){
        printf(1, " [%d] pid=%d va=0x%x flags=0x%x\n", i, buf[i].pid, buf[i].va_page, buf[i].flags);
    }
}

int
main(void)
{
    // 1) 한 페이지만 확보 (va0)
    char *base = sbrk(4096);
    if((int)base<0){ printf(1,"sbrk fail\n"); exit(); }
    uint va0 = ((uint)base) & ~0xFFF;

    // 2) va0의 pa_page
    uint pa0, fl0;
    if(vtop((void*)va0, &pa0, &fl0)!=0){ printf(1,"vtop fail\n"); exit(); }
    uint pa_page = pa0 & ~0xFFF;
    printf(1, "[base] va0=0x%x -> pa_page=0x%x flags=0x%x\n", va0, pa_page, fl0);

    // 3) 같은 PFN을 다른 VA로 alias 매핑
    uint va1 = va0 + 0x3000; // 확실히 비어 있을 거리
    if(vm_map_user_page(va1, pa_page, PTE_U|PTE_W) < 0){
        printf(1,"vm_map_user_page(alias) fail\n"); exit();
    }
    printf(1, "-- after alias(self) --\n");
    pfind(pa_page);

    // ----- A. 기본 TLB 동작(히트/미스) 수치 검증 -----
    // flush + counters 0 → 첫 vtop은 miss, 두번째는 hit 기대
    sw_tlb_reset(1);
    uint pa, fl;
    vtop((void*)va1, &pa, &fl); // miss
    vtop((void*)va1, &pa, &fl); // hit
    print_tlb("baseline 2x vtop on same VA (expect hits=1, misses=1, lookups=2)");

    // ----- B. 권한 변경 시 invalidate 발생 검증 -----
    // 엔트리는 유지(카운터만 0)한 뒤, protect_nowrite가 해당 엔트리를 무효화해야 함.
    sw_tlb_reset(0); // 카운터만 0 (엔트리는 그대로)
    if (protect_nowrite((void*)va1) < 0){
        printf(1,"protect_nowrite fail\n"); exit();
    }
    // invalidate로 인해 다음 vtop은 miss여야 함
    vtop((void*)va1, &pa, &fl); // miss (재채움)
    print_tlb("after protect_nowrite + 1x vtop (expect misses=1, hits=0, lookups=1)");
    // 한 번 더 호출해서 재히트 확인
    vtop((void*)va1, &pa, &fl); // hit
    print_tlb("after protect_nowrite + 2x vtop (expect hits=1, misses=1, lookups=2)");

    // ----- C. 프로세스 경계(자식)에서 동일 PFN 매핑 → IPT 체인 증가 -----

```

```

int pid = fork();
if(pid < 0){ printf(1,"fork fail\n"); exit(); }
if(pid == 0){
    uint sz = (uint)sbrk(0);
    uint va_child = ((sz + 0x3000) & ~0xFFF);
    if(vm_map_user_page(va_child, pa_page, PTE_U|PTE_W) < 0){
        printf(1,"child map fail\n"); exit();
    }
    sleep(50);
    exit();
}
sleep(10);
printf(1, "-- with child --\n");
pfind(pa_page);

// ----- D. 자식 종료 후 IPT 정리 -----
wait();
printf(1, "-- after child exit --\n");
pfind(pa_page);

printf(1,"[ipt_tlb_proof] done\n");
exit();
}

[kalloc.c]
// Physical memory allocator, intended to allocate
// memory for user processes, kernel stacks, page table pages,
// and pipe buffers. Allocates 4096-byte pages.

#include "types.h"
#include "defs.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "spinlock.h"

// ---- 전역 프레임 테이블(과제용) ----
#include "proc.h"    // myproc()
#include "date.h"
#include "physmem.h" // PFNNUM, struct physframe_info

// #define KMEM_SANITY_DEBUG 1 // 필요 시 활성화: 프리리스트 무결성 검사 panic
// #define KMEM_LOG 1 // 필요 시 활성화: 디버그 로그

struct physframe_info pf_info[PFNNUM];

extern uint ticks;
extern struct spinlock tickslock;

static void pf_init(void); // proto

void freerange(void *vstart, void *vend);
extern char end[]; // from kernel.ld

struct run {
    struct run *next;
};

```

```

struct {
    struct spinlock lock;
    int use_lock;
    struct run *freelist;
} kmem;

// ----- 내부 헬퍼: 프리리스트 무결성 검사 -----
#ifdef KMEM_SANITY_DEBUG
static int freelist_sanity(void) {
    struct run *r = kmem.freelist;
    int steps = 0, max_steps = 200000;
    while (r) {
        if (((uint)r) % PGSIZE) return -1;           // 정렬 깨짐
        if ((char*)r < end || (char*)r >= (char*)P2V(PHYSTOP)) return -2;
        r = r->next;
        if (++steps > max_steps) return -3;         // 순환 의심
    }
    return 0;
}
#endif

// ----- 초기화 -----
void
kinit1(void *vstart, void *vend)
{
    initlock(&kmem.lock, "kmem");
    kmem.use_lock = 0;

    pf_init(); // 전역 프레임 테이블 초기화

    freerange(vstart, vend);
}

void
kinit2(void *vstart, void *vend)
{
    freerange(vstart, vend);
    kmem.use_lock = 1;
}

static void
pf_init(void)
{
    for (int i = 0; i < PFNNUM; i++) {
        pf_info[i].frame_index = i;
        pf_info[i].allocated   = 0;
        pf_info[i].pid         = -1;
        pf_info[i].start_tick  = 0;
    }
}

// ----- 초기 프리리스트 구성 -----
void
freerange(void *vstart, void *vend)
{
    char *p = (char*)PGROUNDUP((uint)vstart);
    for (; p + PGSIZE <= (char*)vend; p += PGSIZE)

```

```

    kfree(p);
}

// ----- 해제(kfree) -----
// v: 커널 가상주소(KVA)이며 페이지 정렬되어야 함.
void
kfree(char *v)
{

// 수정:
if ((uint)v % PGSIZE || v < end || v >= (char*)P2V(PHYSTOP)) {
    cprintf("kfree: bad addr v=0x%x pa=0x%x\n", (uint)v, V2P((uint)v));
    panic("kfree: bad addr");
}

uint pa = V2P((uint)v);
uint idx = pa >> 12;

if (kmem.use_lock) acquire(&kmem.lock);

// kmem.use_lock==1 (kinit2 이후)부터는 double free 가드 가능
if (kmem.use_lock && idx < PFNNUM) {
    if (pf_info[idx].allocated == 0) {
        // 이미 free 상태인데 또 free 시도
        if (kmem.use_lock) release(&kmem.lock);
        cprintf("kfree: double free pa=0x%x\n", pa);
        panic("kfree: double free");
    }
}

// 1) 먼저 페이지를 채워 dangling ref 잡기 (헤더를 나중에 쓴다!)
memset(v, 1, PGSIZE);

// 2) 프레임 테이블 업데이트(Free 표시)
if (idx < PFNNUM) {
    pf_info[idx].allocated = 0;
    pf_info[idx].pid = -1;
    pf_info[idx].start_tick = 0;
    pf_info[idx].frame_index = idx;
}

// 3) 프리리스트에 연결
struct run *r = (struct run*)v;
r->next = kmem.freelist;
kmem.freelist = r;

#ifdef KMEM_SANITY_DEBUG
int s = freelist_sanity();
if (s != 0) {
    if (kmem.use_lock) release(&kmem.lock);
    panic("kfree: freelist corrupt (%d)", s);
}
#endif

if (kmem.use_lock) release(&kmem.lock);
}

// ----- 할당(kalloc) -----

```

```

char*
kalloc(void)
{
    struct run *r;

    if (kmem.use_lock) acquire(&kmem.lock);

#ifdef KMEM_SANITY_DEBUG
    {
        int s = freelist_sanity();
        if (s != 0) {
            if (kmem.use_lock) release(&kmem.lock);
            panic("kalloc: freelist corrupt (%d)", s);
        }
    }
#endif

    r = kmem.freelist;
    if (r)
        kmem.freelist = r->next;

    // 과제용 메타데이터 업데이트는 락 내부에서 일관되게
    if (r) {
        uint pa = V2P((uint)r);
        uint idx = pa >> 12;

        if (idx < PFNNUM) {
            if (kmem.use_lock) {
                uint t = ticks; // 스냅샷 정도면 락 불필요
                int owner = -1;
                struct proc *p = myproc(); // 락 안에서 호출해도 xv6에서는 안전
                if (p) owner = p->pid;

                pf_info[idx].allocated = 1;
                pf_info[idx].pid = owner;
                pf_info[idx].start_tick = t;
                pf_info[idx].frame_index = idx;
            } else {
                // 부팅 초기(kinit1 구간)에는 최소 표기만
                pf_info[idx].allocated = 1;
                pf_info[idx].pid = -1;
                pf_info[idx].start_tick = 0;
                pf_info[idx].frame_index = idx;
            }
        }
    }

    if (kmem.use_lock) release(&kmem.lock);

#ifdef KMEM_LOG
    if (!r) cprintf("[kalloc] out of memory\n");
#endif
    return (char*)r;
}

// ----- 과제용 dump -----
int
dump_physmem_info_kernel(char *uaddr, int max_entries)

```

```

{
    struct proc *cur = myproc();
    if (!cur || !uaddr || max_entries <= 0) return -1;

    int limit = PFNNUM;
    if (max_entries < limit) limit = max_entries;

    int copied = 0;
    for (int i = 0; i < limit; i++) {
        if (copyout(cur->pgdir,
                    (uint)uaddr + i * sizeof(struct physframe_info),
                    (void*)&pf_info[i],
                    sizeof(struct physframe_info)) < 0)
            break;
        copied++;
    }
    return copied;
}

```

[main.c]

```

#include "types.h"
#include "defs.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "proc.h"
#include "x86.h"
#include "tlb.h"

static void startothers(void);
static void mpmain(void) __attribute__((noreturn));
extern pde_t *kpgdir;
extern char end[]; // first address after kernel loaded from ELF file

```

```

// Bootstrap processor starts running C code here.
// Allocate a real stack and switch to it, first
// doing some setup required for memory allocator to work.
int

```

```

main(void)
{
    kinit1(end, P2V(4*1024*1024)); // phys page allocator
    kvmalloc(); // kernel page table
    mpinit(); // detect other processors
    lapicinit(); // interrupt controller
    seginit(); // segment descriptors
    picinit(); // disable pic
    ioapicinit(); // another interrupt controller
    consoleinit(); // console hardware
    uartinit(); // serial port
    pinit(); // process table
    tvinit(); // trap vectors
    binit(); // buffer cache
    fileinit(); // file table
    ideinit(); // disk
    startothers(); // start other processors
    kinit2(P2V(4*1024*1024), P2V(PHYSTOP)); // must come after startothers()
}

```

```

// SW TLB 초기화
tlb_init();

// 역페이지 테이블을 main.c에서 메모리 초기화 이후 한 번 호출
ipt_init();

userinit();      // first user process
mpmain();        // finish this processor's setup
}

// Other CPUs jump here from entryother.S.
static void
mpenter(void)
{
    switchkvm();
    seginit();
    lapicinit();
    mpmain();
}

// Common CPU setup code.
static void
mpmain(void)
{
    cprintf("cpu%d: starting %d\n", cpuid(), cpuid());
    idtinit();      // load idt register
    xchg(&(mycpu()->started), 1); // tell startothers() we're up
    scheduler();    // start running processes
}

pde_t entrypgdir[]; // For entry.S

// Start the non-boot (AP) processors.
static void
startothers(void)
{
    extern uchar _binary_entryother_start[], _binary_entryother_size[];
    uchar *code;
    struct cpu *c;
    char *stack;

    // Write entry code to unused memory at 0x7000.
    // The linker has placed the image of entryother.S in
    // _binary_entryother_start.
    code = P2V(0x7000);
    memmove(code, _binary_entryother_start, (uint)_binary_entryother_size);

    for(c = cpus; c < cpus+ncpu; c++){
        if(c == mycpu()) // We've started already.
            continue;

        // Tell entryother.S what stack to use, where to enter, and what
        // pgdir to use. We cannot use kpgdir yet, because the AP processor
        // is running in low memory, so we use entrypgdir for the APs too.
        stack = kalloc();
        *(void**)(code-4) = stack + KSTACKSIZE;
        *(void**)(void)(code-8) = mpenter;
        *(int**)(code-12) = (void *) V2P(entrypgdir);
    }
}

```



```

    lapicstartap(c->apicid, V2P(code));

    // wait for cpu to finish mpmain()
    while(c->started == 0)
        ;
}
}

// The boot page table used in entry.S and entryother.S.
// Page directories (and page tables) must start on page boundaries,
// hence the __aligned__ attribute.
// PTE_PS in a page directory entry enables 4Mbyte pages.

__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
    // Map VA's [0, 4MB) to PA's [0, 4MB)
    [0] = (0 | PTE_P | PTE_W | PTE_PS,
    // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
    [KERNBASE>>PDXSHIFT] = (0 | PTE_P | PTE_W | PTE_PS,
};

//PAGEBREAK!
// Blank page.
//PAGEBREAK!
// Blank page.
//PAGEBREAK!
// Blank page.

```

[Makefile]

```

OBS = \
    bio.o\
    console.o\
    exec.o\
    file.o\
    fs.o\
    ide.o\
    ioapic.o\
    kalloc.o\
    kbd.o\
    lapic.o\
    log.o\
    main.o\
    mp.o\
    picirq.o\
    pipe.o\
    proc.o\
    sleeplock.o\
    spinlock.o\
    string.o\
    swtch.o\
    syscall.o\
    sysfile.o\
    sysproc.o\
    trapasm.o\
    trap.o\

```

```

uart.o\
vectors.o\
vm.o\
ipt.o\
tlb.o\

# Cross-compiling (e.g., on Mac OS X)
# TOOLPREFIX = i386-jos-elf

# Using native tools (e.g., on X86 Linux)
#TOOLPREFIX =

# Try to infer the correct TOOLPREFIX if not set
ifndef TOOLPREFIX
TOOLPREFIX := $(shell if i386-jos-elf-objdump -i 2>&1 | grep '^elf32-i386$$' >/dev/null 2>&1; \
    then echo 'i386-jos-elf-'; \
    elif objdump -i 2>&1 | grep 'elf32-i386' >/dev/null 2>&1; \
    then echo ''; \
    else echo "***" 1>&2; \
    echo "*** Error: Couldn't find an i386-*-elf version of GCC/binutils." 1>&2; \
    echo "*** Is the directory with i386-jos-elf-gcc in your PATH?" 1>&2; \
    echo "*** If your i386-*-elf toolchain is installed with a command" 1>&2; \
    echo "*** prefix other than 'i386-jos-elf-', set your TOOLPREFIX" 1>&2; \
    echo "*** environment variable to that prefix and run 'make' again." 1>&2; \
    echo "*** To turn off this error, run 'gmake TOOLPREFIX= ...'." 1>&2; \
    echo "***" 1>&2; exit 1; fi)
endif

# If the makefile can't find QEMU, specify its path here
# QEMU = qemu-system-i386

# Try to infer the correct QEMU
ifndef QEMU
QEMU = $(shell if which qemu > /dev/null; \
    then echo qemu; exit; \
    elif which qemu-system-i386 > /dev/null; \
    then echo qemu-system-i386; exit; \
    elif which qemu-system-x86_64 > /dev/null; \
    then echo qemu-system-x86_64; exit; \
    else \
    qemu=/Applications/Q.app/Contents/MacOS/i386-softmmu.app/Contents/MacOS/i386-softmmu; \
    if test -x $$qemu; then echo $$qemu; exit; fi; fi; \
    echo "***" 1>&2; \
    echo "*** Error: Couldn't find a working QEMU executable." 1>&2; \
    echo "*** Is the directory containing the qemu binary in your PATH" 1>&2; \
    echo "*** or have you tried setting the QEMU variable in Makefile?" 1>&2; \
    echo "***" 1>&2; exit 1)
endif

CC = $(TOOLPREFIX)gcc
AS = $(TOOLPREFIX)gas
LD = $(TOOLPREFIX)ld
OBJCOPY = $(TOOLPREFIX)objcopy
OBJDUMP = $(TOOLPREFIX)objdump
CFLAGS = -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -Werror
        -fno-omit-frame-pointer
CFLAGS += $(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-stack-protector)
ASFLAGS = -m32 -gdwarf-2 -Wa,-divide

```

```

# FreeBSD ld wants ``elf_i386_fbsd''
LDFLAGS += -m $(shell $(LD) -V | grep elf_i386 2>/dev/null | head -n 1)

# Disable PIE when possible (for Ubuntu 16.10 toolchain)
ifneq ($(shell $(CC) -dumpspeaks 2>/dev/null | grep -e '^[^f]no-pie'),)
CFLAGS += -fno-pie -no-pie
endif
ifneq ($(shell $(CC) -dumpspeaks 2>/dev/null | grep -e '^[^f]nopie'),)
CFLAGS += -fno-pie -nopie
endif

xv6.img: bootblock kernel
    dd if=/dev/zero of=xv6.img count=10000
    dd if=bootblock of=xv6.img conv=notrunc
    dd if=kernel of=xv6.img seek=1 conv=notrunc

xv6memfs.img: bootblock kernelmemfs
    dd if=/dev/zero of=xv6memfs.img count=10000
    dd if=bootblock of=xv6memfs.img conv=notrunc
    dd if=kernelmemfs of=xv6memfs.img seek=1 conv=notrunc

bootblock: bootasm.S bootmain.c
    $(CC) $(CFLAGS) -fno-pic -O -nostdinc -I. -c bootmain.c
    $(CC) $(CFLAGS) -fno-pic -nostdinc -I. -c bootasm.S
    $(LD) $(LDFLAGS) -N -e start -Ttext 0x7C00 -o bootblock.o bootasm.o bootmain.o
    $(OBJDUMP) -S bootblock.o > bootblock.asm
    $(OBJCOPY) -S -O binary -j .text bootblock.o bootblock
    ./sign.pl bootblock

entryother: entryother.S
    $(CC) $(CFLAGS) -fno-pic -nostdinc -I. -c entryother.S
    $(LD) $(LDFLAGS) -N -e start -Ttext 0x7000 -o bootblockother.o entryother.o
    $(OBJCOPY) -S -O binary -j .text bootblockother.o entryother
    $(OBJDUMP) -S bootblockother.o > entryother.asm

initcode: initcode.S
    $(CC) $(CFLAGS) -nostdinc -I. -c initcode.S
    $(LD) $(LDFLAGS) -N -e start -Ttext 0 -o initcode.out initcode.o
    $(OBJCOPY) -S -O binary initcode.out initcode
    $(OBJDUMP) -S initcode.o > initcode.asm

kernel: $(OBJS) entry.o entryother initcode kernel.ld
    $(LD) $(LDFLAGS) -T kernel.ld -o kernel entry.o $(OBJS) -b binary initcode entryother
    $(OBJDUMP) -S kernel > kernel.asm
    $(OBJDUMP) -t kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > kernel.sym

# kernelmemfs is a copy of kernel that maintains the
# disk image in memory instead of writing to a disk.
# This is not so useful for testing persistent storage or
# exploring disk buffering implementations, but it is
# great for testing the kernel on real hardware without
# needing a scratch disk.
MEMFSOBS = $(filter-out ide.o,$(OBJS)) memide.o
kernelmemfs: $(MEMFSOBS) entry.o entryother initcode kernel.ld fs.img
    $(LD) $(LDFLAGS) -T kernel.ld -o kernelmemfs entry.o $(MEMFSOBS) -b binary initcode entryother fs.img
    $(OBJDUMP) -S kernelmemfs > kernelmemfs.asm
    $(OBJDUMP) -t kernelmemfs | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > kernelmemfs.sym

```

```

tags: $(OBJ) entryother.S _init
      etags *.S *.c

vectors.S: vectors.pl
      ./vectors.pl > vectors.S

ULIB = ulib.o usys.o printf.o umalloc.o

_%: %.o $(ULIB)
      $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $$ $^
      $(OBJDUMP) -S $$ > $.asm
      $(OBJDUMP) -t $$ | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $.sym

_forktest: forktest.o $(ULIB)
      # forktest has less library code linked in - needs to be small
      # in order to be able to max out the proc table.
      $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o _forktest forktest.o ulib.o usys.o
      $(OBJDUMP) -S _forktest > forktest.asm

mkfs: mkfs.c fs.h
      gcc -Werror -Wall -o mkfs mkfs.c

# Prevent deletion of intermediate files, e.g. cat.o, after first build, so
# that disk image changes after first build are persistent until clean.  More
# details:
# http://www.gnu.org/software/make/manual/html_node/Chained-Rules.html
.PRECIOUS: %.o

UPROGS=\
_cat\
_echo\
_forktest\
_grep\
_init\
_kill\
_ln\
_ls\
_mkdir\
_rm\
_sh\
_stressfs\
_usertests\
_wc\
_zombie\
_vtop\
_tlbstat\
_pfind\
_alias2map\
_vtop_proof\
_ip_tlb_proof\
_ptlb_memtest\

fs.img: mkfs README $(UPROGS)
      ./mkfs fs.img README $(UPROGS)

-include *.d

clean:

```

```

rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*.o *.d *.asm *.sym vectors.S bootblock entryother \
initcode initcode.out kernel xv6.img fs.img kernelmemfs \
xv6memfs.img mkfs .gdbinit \
$(UPROGS)

# make a printout
FILES = $(shell grep -v '^\\#' runoff.list)
PRINT = runoff.list runoff.spec README toc.hdr toc.ftr $(FILES)

xv6.pdf: $(PRINT)
    ./runoff
    ls -l xv6.pdf

print: xv6.pdf

# run in emulators

bochs : fs.img xv6.img
    if [ ! -e .bochsrc ]; then ln -s dot-bochsrc .bochsrc; fi
    bochs -q

# try to generate a unique GDB port
GDBPORT = $(shell expr `id -u` % 5000 + 25000)
# QEMU's gdb stub command line changed in 0.11
QEMUGDB = $(shell if $(QEMU) -help | grep -q '^-gdb'; \
    then echo "-gdb tcp::$(GDBPORT)"; \
    else echo "-s -p $(GDBPORT)"; fi)
ifndef CPUS
CPUS := 2
endif
QEMUOPTS = -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp
$(CPUS) -m 512 $(QEMUEXTRA)

qemu: fs.img xv6.img
    $(QEMU) -serial mon:stdio $(QEMUOPTS)

qemu-memfs: xv6memfs.img
    $(QEMU) -drive file=xv6memfs.img,index=0,media=disk,format=raw -smp $(CPUS) -m 256

qemu-nox: fs.img xv6.img
    $(QEMU) -nographic $(QEMUOPTS)

.gdbinit: .gdbinit.tmpl
    sed "s/localhost:1234/localhost:$(GDBPORT)/" < $^ > $@

qemu-gdb: fs.img xv6.img .gdbinit
    @echo "*** Now run 'gdb'." 1>&2
    $(QEMU) -serial mon:stdio $(QEMUOPTS) -S $(QEMUGDB)

qemu-nox-gdb: fs.img xv6.img .gdbinit
    @echo "*** Now run 'gdb'." 1>&2
    $(QEMU) -nographic $(QEMUOPTS) -S $(QEMUGDB)

# CUT HERE
# prepare dist for students
# after running make dist, probably want to
# rename it to rev0 or rev1 or so on and then

```

```
# check in that version.
```

```
EXTRA=\
mkfs.c ulib.c user.h cat.c echo.c forktest.c grep.c kill.c\
ln.c ls.c mkdir.c rm.c stressfs.c usertests.c wc.c zombie.c\
printf.c umalloc.c\
README dot-bochsrc *.pl toc.* runoff runoff1 runoff.list\
.gdbinit.tmpl gdbutil\
```

```
dist:
rm -rf dist
mkdir dist
for i in $(FILES); \
do \
    grep -v PAGEBREAK $$i >dist/$$i; \
done
sed '/CUT HERE/,$$d' Makefile >dist/Makefile
echo >dist/runoff.spec
cp $(EXTRA) dist
```

```
dist-test:
rm -rf dist
make dist
rm -rf dist-test
mkdir dist-test
cp dist/* dist-test
cd dist-test; $(MAKE) print
cd dist-test; $(MAKE) bochs || true
cd dist-test; $(MAKE) qemu
```

```
# update this rule (change rev#) when it is time to
# make a new revision.
```

```
tar:
rm -rf /tmp/xv6
mkdir -p /tmp/xv6
cp dist/* dist/.gdbinit.tmpl /tmp/xv6
(cd /tmp; tar cf - xv6) | gzip >xv6-rev10.tar.gz # the next one will be 10 (9/17)
```

```
.PHONY: dist-test dist
```

```
[mmu.h]
// This file contains definitions for the
// x86 memory management unit (MMU).
```

```
#ifndef XV6_MMU_H
#define XV6_MMU_H
```

```
// Eflags register
#define FL_IF          0x00000200      // Interrupt Enable

// Control Register flags
#define CR0_PE         0x00000001      // Protection Enable
#define CR0_WP         0x00010000      // Write Protect
#define CR0_PG         0x80000000      // Paging

#define CR4_PSE        0x00000010      // Page size extension

// various segment selectors.
```

```

#define SEG_KCODE 1 // kernel code
#define SEG_KDATA 2 // kernel data+stack
#define SEG_UCODE 3 // user code
#define SEG_UDATA 4 // user data+stack
#define SEG_TSS 5 // this process's task state

// cpu->gdt[NSEGS] holds the above segments.
#define NSEGS 6

#ifdef __ASSEMBLER__
// Segment Descriptor
struct segdesc {
    uint lim_15_0 : 16; // Low bits of segment limit
    uint base_15_0 : 16; // Low bits of segment base address
    uint base_23_16 : 8; // Middle bits of segment base address
    uint type : 4; // Segment type (see STS_ constants)
    uint s : 1; // 0 = system, 1 = application
    uint dpl : 2; // Descriptor Privilege Level
    uint p : 1; // Present
    uint lim_19_16 : 4; // High bits of segment limit
    uint avl : 1; // Unused (available for software use)
    uint rsv1 : 1; // Reserved
    uint db : 1; // 0 = 16-bit segment, 1 = 32-bit segment
    uint g : 1; // Granularity: limit scaled by 4K when set
    uint base_31_24 : 8; // High bits of segment base address
};

// Normal segment
#define SEG(type, base, lim, dpl) (struct segdesc) \
{ ((lim) >> 12) & 0xffff, (uint)(base) & 0xffff, \
  ((uint)(base) >> 16) & 0xff, type, 1, dpl, 1, \
  (uint)(lim) >> 28, 0, 0, 1, 1, (uint)(base) >> 24 }
#define SEG16(type, base, lim, dpl) (struct segdesc) \
{ (lim) & 0xffff, (uint)(base) & 0xffff, \
  ((uint)(base) >> 16) & 0xff, type, 1, dpl, 1, \
  (uint)(lim) >> 16, 0, 0, 1, 0, (uint)(base) >> 24 }
#endif

#define DPL_USER 0x3 // User DPL

// Application segment type bits
#define STA_X 0x8 // Executable segment
#define STA_W 0x2 // Writeable (non-executable segments)
#define STA_R 0x2 // Readable (executable segments)

// System segment type bits
#define STS_T32A 0x9 // Available 32-bit TSS
#define STS_IG32 0xE // 32-bit Interrupt Gate
#define STS_TG32 0xF // 32-bit Trap Gate

// A virtual address 'va' has a three-part structure as follows:
//
// +-----10-----+-----10-----+-----12-----+
// | Page Directory | Page Table | Offset within Page |
// | Index | Index | |
// +-----+-----+-----+
// \--- PDX(va) ---/ \--- PTX(va) ---/

```

```

// page directory index
#define PDX(va)          (((uint)(va) >> PDXSHIFT) & 0x3FF)

// page table index
#define PTX(va)          (((uint)(va) >> PTXSHIFT) & 0x3FF)

// construct virtual address from indexes and offset
#define PGADDR(d, t, o) ((uint)((d) << PDXSHIFT | (t) << PTXSHIFT | (o)))

// Page directory and page table constants.
#define NPENTRIES        1024    // # directory entries per page directory
#define NPTENTRIES        1024    // # PTEs per page table
#define PGSIZE            4096    // bytes mapped by a page

#define PTXSHIFT          12      // offset of PTX in a linear address
#define PDXSHIFT          22      // offset of PDX in a linear address

#define PGROUNDUP(sz)     (((sz)+PGSIZE-1) & ~(PGSIZE-1))
#define PGROUNDDOWN(a)    (((a)) & ~(PGSIZE-1))

// Page table/directory entry flags.
#define PTE_P              0x001   // Present
#define PTE_W              0x002   // Writeable
#define PTE_U              0x004   // User
#define PTE_PS             0x080   // Page Size

// Address in page table or page directory entry
#define PTE_ADDR(pte)     ((uint)(pte) & ~0xFFF)
#define PTE_FLAGS(pte)    ((uint)(pte) & 0xFFF)

#ifdef __ASSEMBLER__
typedef uint pte_t; // page table entry(32-bit)
// 추가
typedef uint pde_t; // page directory entry(32-bit)

// Task state segment format
struct taskstate {
    uint link;           // Old ts selector
    uint esp0;           // Stack pointers and segment selectors
    ushort ss0;          // after an increase in privilege level
    ushort padding1;
    uint *esp1;
    ushort ss1;
    ushort padding2;
    uint *esp2;
    ushort ss2;
    ushort padding3;
    void *cr3;           // Page directory base
    uint *eip;           // Saved state from last task switch
    uint eflags;
    uint eax;            // More saved state (registers)
    uint ecx;
    uint edx;
    uint ebx;
    uint *esp;
    uint *ebp;
    uint esi;
    uint edi;

```



```

ushort es:          // Even more saved state (segment selectors)
ushort padding4:
ushort cs:
ushort padding5:
ushort ss:
ushort padding6:
ushort ds:
ushort padding7:
ushort fs:
ushort padding8:
ushort gs:
ushort padding9:
ushort ldt:
ushort padding10:
ushort t:           // Trap on task switch
ushort iomb:        // I/O map base address
};

```

```

// Gate descriptors for interrupts and traps
struct gatedesc {
    uint off_15_0 : 16; // low 16 bits of offset in segment
    uint cs : 16;        // code segment selector
    uint args : 5;       // # args, 0 for interrupt/trap gates
    uint rsv1 : 3;       // reserved(should be zero I guess)
    uint type : 4;       // type(STS_{IG32,TG32})
    uint s : 1;         // must be 0 (system)
    uint dpl : 2;       // descriptor(meaning new) privilege level
    uint p : 1;         // Present
    uint off_31_16 : 16; // high bits of offset in segment
};

```

```

// Set up a normal interrupt/trap gate descriptor.
// - istrap: 1 for a trap (= exception) gate, 0 for an interrupt gate.
// - sel: Code segment selector for interrupt/trap handler
// - off: Offset in code segment for interrupt/trap handler
// - dpl: Descriptor Privilege Level -
//       the privilege level required for software to invoke
//       this interrupt/trap gate explicitly using an int instruction.
#define SETGATE(gate, istrap, sel, off, d) \
{ \
    (gate).off_15_0 = (uint)(off) & 0xffff; \
    (gate).cs = (sel); \
    (gate).args = 0; \
    (gate).rsv1 = 0; \
    (gate).type = (istrap) ? STS_TG32 : STS_IG32; \
    (gate).s = 0; \
    (gate).dpl = (d); \
    (gate).p = 1; \
    (gate).off_31_16 = (uint)(off) >> 16; \
}
#endif /* !__ASSEMBLER__ */

#endif /* XV6_MMU_H */

```

// pfind.c - 지정한 물리 페이지(pa_page)에 매핑된 (pid, va, flags) 목록을 사용자 공간에서 조회/출력

```
#include "types.h"
#include "user.h"
#include "fcntl.h"
#include "vlist.h" // struct vlist { int pid; uint va_page; uint flags; }
#include "stat.h"
```

// 16진수 문자열을 u32로 변환(선택적 "0x"/"0X" 프리픽스 지원)

```
static uint parse_hex_u32(const char *s) {
    uint v = 0;
    if (!s) return 0;
    if (s[0] == '0' && (s[1] == 'x' || s[1] == 'X')) s += 2;
    for (; *s; s++) {
        int d;
        if (*s >= '0' && *s <= '9') d = *s - '0';
        else if (*s >= 'a' && *s <= 'f') d = *s - 'a' + 10;
        else if (*s >= 'A' && *s <= 'F') d = *s - 'A' + 10;
        else break;
        v = (v << 4) | (uint)d;
    }
    return v;
}
```

/*

아래 주석 블록은 이전(힙 malloc) 버전 예시.

- 현 코드는 스택 고정 버퍼를 사용해 malloc 의존성을 제거함.

```
int main(int argc, char **argv) { ... }
```

*/

```
int main(int argc, char **argv) {
    if (argc < 2) { printf(1, "usage: pfind PA_PAGE_HEX\n"); exit(); }
    uint pa_page = parse_hex_u32(argv[1]); // 물리 페이지 주소(페이지 정렬된 값) 파싱

    int max = 1; // 커널에 요청할 최대 엔트리 수(작게 시작)
    struct vlist buf[4]; // 스택 고정 버퍼(여유분 4개 확보); overflow 방지는 커널이 max를
    준수한다는 전제 // 필요 시 max를 늘려 재호출하는 전략으로 확장 가능
}
```

```
int n = phys2virt(pa_page, buf, max); // 커널 시스템콜/래퍼: pa_page에 매핑된 (pid, va, flags) 목록 채움
if (n < 0) { printf(1, "phys2virt failed\n"); exit(); }
```

```
for (int i = 0; i < n; i++)
    printf(1, "pid=%d va=0x%x flags=0x%x\n", buf[i].pid, buf[i].va_page, buf[i].flags);

exit();
}
```

[physmem.h]

// physmem.h

#ifndef PHYSMEM_H

#define PHYSMEM_H

#include "types.h" // xv6의 uint 정의

#define PFNNUM 60000

```
struct physframe_info {
```

```

uint frame_index; // 물리 프레임 번호 (pa >> 12)
int allocated;    // 1=할당됨, 0=free
int pid;         // 소유 PID, 없으면 -1
uint start_tick; // 할당 시작 tick
};

#endif

```

```

[proc.c]
#include "types.h"
#include "defs.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "x86.h"
#include "proc.h"
#include "spinlock.h"
#include "ipt.h"
#include "tlb.h"

```

```

struct {
    struct spinlock lock;
    struct proc proc[NPROC];
} ptable;

```

```

static struct proc *initproc;

```

```

int nextpid = 1;
extern void forkret(void);
extern void trapret(void);

```

```

static void wakeup1(void *chan);

```

```

void
pinit(void)
{
    initlock(&ptable.lock, "ptable");
}

```

```

// Must be called with interrupts disabled
int
cpuid() {
    return mycpu() - cpus;
}

```

```

// Must be called with interrupts disabled to avoid the caller being
// rescheduled between reading lapicid and running through the loop.

```

```

struct cpu*
mycpu(void)
{
    int apicid, i;

```

```

    if(readeflags() & FL_IF)
        panic("mycpu called with interrupts enabled\n");

```

```

    apicid = lapicid();
    // APIC IDs are not guaranteed to be contiguous. Maybe we should have

```

```

// a reverse map, or reserve a register to store &cpus[i].
for (i = 0; i < ncpu; ++i) {
    if (cpus[i].apicid == apicid)
        return &cpus[i];
}
panic("unknown apicid\n");
}

```

```

// Disable interrupts so that we are not rescheduled
// while reading proc from the cpu structure
struct proc*
myproc(void) {
    struct cpu *c;
    struct proc *p;
    pushcli();
    c = mycpu();
    p = c->proc;
    popcli();
    return p;
}

```

```

//PAGEBREAK: 32
// Look in the process table for an UNUSED proc.
// If found, change state to EMBRYO and initialize
// state required to run in the kernel.
// Otherwise return 0.
static struct proc*
allocproc(void)
{
    struct proc *p;
    char *sp;

    acquire(&ptable.lock);

    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
        if(p->state == UNUSED)
            goto found;

    release(&ptable.lock);
    return 0;

```

```

found:
    p->state = EMBRYO;
    p->pid = nextpid++;

    release(&ptable.lock);

    // Allocate kernel stack.
    if((p->kstack = kalloc()) == 0){
        p->state = UNUSED;
        return 0;
    }
    sp = p->kstack + KSTACKSIZE;

```

```

// Leave room for trap frame.
sp -= sizeof *p->tf;
p->tf = (struct trapframe*)sp;

```

```

// Set up new context to start executing at forkret,
// which returns to trapret.
sp -= 4;
*(uint*)sp = (uint)trapret;

sp -= sizeof *p->context;
p->context = (struct context*)sp;
memset(p->context, 0, sizeof *p->context);
p->context->eip = (uint)forkret;

return p;
}

//PAGEBREAK: 32
// Set up first user process.
void
userinit(void)
{
    struct proc *p;
    extern char _binary_initcode_start[], _binary_initcode_size[];

    p = allocproc();
    initproc = p;

    if((p->pgdir = setupkvm()) == 0)
        panic("userinit: out of memory?");
    initvm(p->pgdir, _binary_initcode_start, (int)_binary_initcode_size);

    // initcode가 올라간 "새 주소공간"을 IPT에 반영
    ipt_rebuild_for_pgdir(p->pid, p->pgdir);

    p->sz = PGSIZE;
    memset(p->tf, 0, sizeof(*p->tf));
    p->tf->cs = (SEG_UCODE << 3) | DPL_USER;
    p->tf->ds = (SEG_UDATA << 3) | DPL_USER;
    p->tf->es = p->tf->ds;
    p->tf->ss = p->tf->ds;
    p->tf->eflags = FL_IF;
    p->tf->esp = PGSIZE;
    p->tf->eip = 0; // beginning of initcode.S

    safestrcpy(p->name, "initcode", sizeof(p->name));
    p->cwd = namei("/");

    acquire(&ptable.lock);
    p->state = RUNNABLE;
    release(&ptable.lock);
}

// Grow current process's memory by n bytes.
// Return 0 on success, -1 on failure.
int
growproc(int n)
{
    uint sz;
    struct proc *curproc = myproc();

    sz = curproc->sz;

```

```

if(n > 0){
    if((sz = allocuvm(curproc->pgdir, sz, sz + n)) == 0)
        return -1;
} else if(n < 0){
    if((sz = deallocuvm(curproc->pgdir, sz, sz + n)) == 0)
        return -1;
}
curproc->sz = sz;
switchuvm(curproc);
return 0;
}

// Create a new process copying p as the parent.
// Sets up stack to return as if from system call.
// Caller must set state of returned proc to RUNNABLE.
int
fork(void)
{
    int i, pid;
    struct proc *np;
    struct proc *curproc = myproc();

    if((np = allocproc()) == 0)
        return -1;

    // 자식 주소공간 생성
    if((np->pgdir = copyuvm(curproc->pgdir, curproc->sz)) == 0){
        kfree(np->kstack);
        np->kstack = 0;
        np->state = UNUSED;
        return -1;
    }

    // 자식 주소공간을 IPT에 채움
    ipt_rebuild_for_pgdir(np->pid, np->pgdir);

    np->sz = curproc->sz;
    np->parent = curproc;
    *np->tf = *curproc->tf;
    np->tf->eax = 0;

    for(i = 0; i < NOFILE; i++)
        if(curproc->ofile[i])
            np->ofile[i] = filedup(curproc->ofile[i]);
    np->cwd = idup(curproc->cwd);

    safestrcpy(np->name, curproc->name, sizeof(np->name));
    pid = np->pid;

    acquire(&ptable.lock);
    np->state = RUNNABLE;
    release(&ptable.lock);

    return pid;
}

// Exit the current process. Does not return.
// An exited process remains in the zombie state

```

```

// until its parent calls wait() to find out it exited.
void
exit(void)
{
    struct proc *curproc = myproc();
    struct proc *p;
    int fd;

    if(curproc == initproc)
        panic("init exiting");

    // 파일/디렉터리 정리
    for(fd = 0; fd < NOFILE; fd++){
        if(curproc->ofile[fd]){
            fileclose(curproc->ofile[fd]);
            curproc->ofile[fd] = 0;
        }
    }
    begin_op();
    iput(curproc->cwd);
    end_op();
    curproc->cwd = 0;

    acquire(&ptable.lock);

    // 부모 깨우기
    wakeup1(curproc->parent);

    // 고아 프로세스 처리
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(p->parent == curproc){
            p->parent = initproc;
            if(p->state == ZOMBIE)
                wakeup1(initproc);
        }
    }

    // 과제 요구: exit 직후 IPT 정리(좀비 잔재 방지)
    ipt_scrub_for_pgdir(curproc->pid, curproc->pgdir);

    // 해당 PID의 SW-TLB 전부 무효화
    tlb_invalidate_pid(curproc->pid);

    // 좀비 전환
    curproc->state = ZOMBIE;
    sched();
    panic("zombie exit");
}

// Wait for a child process to exit and return its pid.
// Return -1 if this process has no children.
int
wait(void)
{
    struct proc *p;
    int havekids, pid;
    struct proc *curproc = myproc();

```

```

acquire(&ptable.lock);
for(;;){
    havekids = 0;
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(p->parent != curproc)
            continue;
        havekids = 1;
        if(p->state == ZOMBIE){
            // ▲ TLB 안전 차원에서 한 번 더 무효화(중복이어도 무해)
            tlb_invalidate_pid(p->pid);

            // 메모리 실제 해제(ipt_scrub은 exit에서 이미 수행함)
            kfree(p->kstack); p->kstack = 0;
            freevm(p->pgdir);

            pid = p->pid;
            p->pid = 0;
            p->parent = 0;
            p->name[0] = 0;
            p->killed = 0;
            p->state = UNUSED;
            release(&ptable.lock);
            return pid;
        }
    }
    if(!havekids || curproc->killed){
        release(&ptable.lock);
        return -1;
    }
    sleep(curproc, &ptable.lock);
}
}

```

```

//PAGEBREAK: 42
// Per-CPU process scheduler.
// Each CPU calls scheduler() after setting itself up.
// Scheduler never returns. It loops, doing:
//  - choose a process to run
//  - swtch to start running that process
//  - eventually that process transfers control
//    via swtch back to the scheduler.
void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;

    for(;;){
        // Enable interrupts on this processor.
        sti();

        // Loop over process table looking for process to run.
        acquire(&ptable.lock);
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->state != RUNNABLE)

```



```

        continue;

    // Switch to chosen process. It is the process's job
    // to release ptable.lock and then reacquire it
    // before jumping back to us.
    c->proc = p;
    switchvm(p);
    p->state = RUNNING;

    swtch(&(c->scheduler), p->context);
    switchkvm();

    // Process is done running for now.
    // It should have changed its p->state before coming back.
    c->proc = 0;
}
release(&ptable.lock);

}
}

// Enter scheduler. Must hold only ptable.lock
// and have changed proc->state. Saves and restores
// intena because intena is a property of this
// kernel thread, not this CPU. It should
// be proc->intena and proc->ncli, but that would
// break in the few places where a lock is held but
// there's no process.
void
sched(void)
{
    int intena;
    struct proc *p = myproc();

    if(!holding(&ptable.lock))
        panic("sched ptable.lock");
    if(mycpu()->ncli != 1)
        panic("sched locks");
    if(p->state == RUNNING)
        panic("sched running");
    if(readeflags() & FL_IF)
        panic("sched interruptible");
    intena = mycpu()->intena;
    swtch(&p->context, mycpu()->scheduler);
    mycpu()->intena = intena;
}

// Give up the CPU for one scheduling round.
void
yield(void)
{
    acquire(&ptable.lock); //DOC: yieldlock
    myproc()->state = RUNNABLE;
    sched();
    release(&ptable.lock);
}

// A fork child's very first scheduling by scheduler()

```

```

// will switch here. "Return" to user space.
void
forkret(void)
{
    static int first = 1;
    // Still holding ptable.lock from scheduler.
    release(&ptable.lock);

    if (first) {
        // Some initialization functions must be run in the context
        // of a regular process (e.g., they call sleep), and thus cannot
        // be run from main().
        first = 0;
        iinit(ROOTDEV);
        initlog(ROOTDEV);
    }

    // Return to "caller", actually trapret (see allocproc).
}

// Atomically release lock and sleep on chan.
// Reacquires lock when awakened.
void
sleep(void *chan, struct spinlock *lk)
{
    struct proc *p = myproc();

    if(p == 0)
        panic("sleep");

    if(lk == 0)
        panic("sleep without lk");

    // Must acquire ptable.lock in order to
    // change p->state and then call sched.
    // Once we hold ptable.lock, we can be
    // guaranteed that we won't miss any wakeup
    // (wakeup runs with ptable.lock locked),
    // so it's okay to release lk.
    if(lk != &ptable.lock){ //DOC: sleeplock0
        acquire(&ptable.lock); //DOC: sleeplock1
        release(lk);
    }
    // Go to sleep.
    p->chan = chan;
    p->state = SLEEPING;

    sched();

    // Tidy up.
    p->chan = 0;

    // Reacquire original lock.
    if(lk != &ptable.lock){ //DOC: sleeplock2
        release(&ptable.lock);
        acquire(lk);
    }
}

```

```

//PAGEBREAK!
// Wake up all processes sleeping on chan.
// The ptable lock must be held.
static void
wakeup1(void *chan)
{
    struct proc *p;

    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
        if(p->state == SLEEPING && p->chan == chan)
            p->state = RUNNABLE;
}

// Wake up all processes sleeping on chan.
void
wakeup(void *chan)
{
    acquire(&ptable.lock);
    wakeup1(chan);
    release(&ptable.lock);
}

// Kill the process with the given pid.
// Process won't exit until it returns
// to user space (see trap in trap.c).
int
kill(int pid)
{
    struct proc *p;

    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(p->pid == pid){
            p->killed = 1;
            // Wake process from sleep if necessary.
            if(p->state == SLEEPING)
                p->state = RUNNABLE;
            release(&ptable.lock);
            return 0;
        }
    }
    release(&ptable.lock);
    return -1;
}

//PAGEBREAK: 36
// Print a process listing to console. For debugging.
// Runs when user types ^P on console.
// No lock to avoid wedging a stuck machine further.
void
procdump(void)
{
    static char *states[] = {
        [UNUSED]    "unused",
        [EMBRYO]    "embryo",
        [SLEEPING]  "sleep ",
        [RUNNABLE]  "runble",
    }

```

```

[RUNNING]    "run    ",
[ZOMBIE]     "zombie"
};
int i;
struct proc *p;
char *state;
uint pc[10];

for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->state == UNUSED)
        continue;
    if(p->state >= 0 && p->state < NELEM(states) && states[p->state])
        state = states[p->state];
    else
        state = "???";
    cprintf("%d %s %s", p->pid, state, p->name);
    if(p->state == SLEEPING){
        getcallerpcs((uint*)p->context->ebp+2, pc);
        for(i=0; i<10 && pc[i] != 0; i++)
            cprintf(" %p", pc[i]);
    }
    cprintf("\n");
}
}

// pgdir -> pid 역매핑 유틸 (락 상태에 안전)
// - ptable.lock을 이미 들고 있으면 추가 acquire 없이 바로 순회
// - 아니면 acquire/release로 보호
int
ipt_pid_from_pgdir(pde_t *pgdir)
{
    if (pgdir == 0)
        return -1;                // 못 찾음은 -1

    int pid = -1;                  // 기본값 -1
    int need_unlock = 0;

    if (!holding(&ptable.lock)) {
        acquire(&ptable.lock);
        need_unlock = 1;
    }

    for (struct proc *p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
        if (p->state != UNUSED && p->pgdir == pgdir) {
            pid = p->pid;
            break;
        }
    }

    if (need_unlock)
        release(&ptable.lock);

    return pid;                    // -1 또는 실제 pid
}

```

```

#include "param.h"

// Per-CPU state
struct cpu {
    uchar apicid;           // Local APIC ID
    struct context *scheduler; // swtch() here to enter scheduler
    struct taskstate ts;     // Used by x86 to find stack for interrupt
    struct segdesc gdt[NSEGS]; // x86 global descriptor table
    volatile uint started;   // Has the CPU started?
    int ncli;                // Depth of pushcli nesting.
    int intena;              // Were interrupts enabled before pushcli?
    struct proc *proc;       // The process running on this cpu or null
};

extern struct cpu cpus[NCPU];
extern int ncpu;

//PAGEBREAK: 17
// Saved registers for kernel context switches.
// Don't need to save all the segment registers (%cs, etc),
// because they are constant across kernel contexts.
// Don't need to save %eax, %ecx, %edx, because the
// x86 convention is that the caller has saved them.
// Contexts are stored at the bottom of the stack they
// describe; the stack pointer is the address of the context.
// The layout of the context matches the layout of the stack in swtch.S
// at the "Switch stacks" comment. Switch doesn't save eip explicitly,
// but it is on the stack and allocproc() manipulates it.
struct context {
    uint edi;
    uint esi;
    uint ebx;
    uint ebp;
    uint eip;
};

enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
    uint sz;                // Size of process memory (bytes)
    pde_t* pgdir;           // Page table
    char *kstack;           // Bottom of kernel stack for this process
    enum procstate state;    // Process state
    int pid;                // Process ID
    struct proc *parent;     // Parent process
    struct trapframe *tf;    // Trap frame for current syscall
    struct context *context; // swtch() here to run process
    void *chan;             // If non-zero, sleeping on chan
    int killed;              // If non-zero, have been killed
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;       // Current directory
    char name[16];          // Process name (debugging)
};

// Process memory is laid out contiguously, low addresses first:
//  text
//  original data and bss

```

```

// fixed-size stack
// expandable heap

[ptlb_memtest.c]
// ptlb_memtest.c
#include "types.h"
#include "stat.h"
#include "user.h"
#include "mmu.h"
#include "vlist.h"

// ---- 선택: TLB 통계 찍고 싶을 때 사용 ----
static void tlb_print(const char *tag) {
    struct tlb_stats s;
    if (sw_tlb_read(&s) == 0) {
        int hitpct = s.lookups ? (int)(s.hits * 100 / s.lookups) : 0;
        printf(1, "[%s] hits=%d misses=%d hit%%=%d lookups=%d evicts=%d invalidations=%d lines=%d ways=%d sets=%d\n",
            tag, s.hits, s.misses, hitpct, s.lookups, s.evicts, s.invalidations, s.lines, s.ways, s.sets);
    }
}

// ---- 유틸 ----
static void print_vtop(const char *tag, uint va) {
    uint pa, fl;
    int rc = vtop((void*)va, &pa, &fl);
    if (rc == 0) {
        printf(1, "%s va=0x%x -> pa=0x%x flags=0x%x (P=%d W=%d U=%d)\n",
            tag, va, pa, fl, !(fl&PTE_P), !(fl&PTE_W), !(fl&PTE_U));
    } else {
        printf(1, "%s va=0x%x -> rc=%d\n", tag, va, rc);
    }
}

static void pfind(uint pa_page) {
    struct vlist buf[32];
    int n = phys2virt(pa_page, buf, 32);
    printf(1, "[pfind] pa_page=0x%x -> %d entries\n", pa_page, n);
    for (int i = 0; i < n; i++)
        printf(1, " [%d] pid=%d va=0x%x flags=0x%x\n",
            i, buf[i].pid, buf[i].va_page, buf[i].flags);
}

// 이미 매핑돼 있으면 "언맵만" 해서 깔끔한 상태로 만든다.
// free_if_last=0 으로 호출하여 실제 물리프레임은 절대 해제하지 않음.
static void ensure_unmapped(uint va){
    uint pa, fl;
    int rc = vtop((void*)va, &pa, &fl);
    if (rc == 0) {
        if (vm_unmap_one((void*)va, 0) < 0) { // ★ 변경: unmap1() → vm_unmap_one(...,0)
            printf(1, "ensure_unmapped: vm_unmap_one(0x%x) fail\n", va);
            exit();
        }
    }
}

// ---- 본 테스트 ----
int

```

```

main(void)
{
    printf(1, "=== ptlb_memtest: start ===\n");

    // sw_tlb_reset(1); // 1: 엔트리 flush+카운터0, 0: 카운터만 0

    // 1) 힙에서 정확히 1 페이지 확보 (top 페이지)
    char *p = sbrk(4096);
    if ((int)p < 0) { printf(1, "sbrk fail\n"); exit(); }
    uint va0 = ((uint)p) & ~0xFFF;

    // sw_vtop 기본 동작 & TLB hit(동일 VA 2회) + 오프셋 보존
    print_vtop("[vtop#1]", va0);
    print_vtop("[vtop#2]", va0); // 두 번째 호출은 SW-TLB hit 기대
    tlb_print("baseline (after two vtop on va0)"); // 원하면 주석 해제

    uint pa0, fl0;
    if (vtop((void*)va0, &pa0, &fl0) != 0) { printf(1, "vtop base fail\n"); exit(); }
    uint pa_page = pa0 & ~0xFFF;

    // 오프셋 보존: va0+0x123 → (pa & 0xFFF)=0x123 기대
    uint pa_chk, fl_chk;
    if (vtop((void*)(va0+0x123), &pa_chk, &fl_chk) == 0) {
        printf(1, "[offset] va=0x%x -> pa=0x%x (pa&0xFFF=0x%x, expect 0x123)\n",
            va0+0x123, pa_chk, pa_chk & 0xFFF);
    }

    // 2) 같은 프로세스 내 alias 생성 → IPT 중복 체인 + TLB 검증
    uint va_alias = va0 + 0x2000; // 비어있도록 떨어진 주소 사용
    ensure_unmapped(va_alias);
    if (vm_map_user_page(va_alias, pa_page, PTE_U|PTE_W) < 0) {
        printf(1, "alias map fail\n"); exit();
    }
    printf(1, "-- after alias(self) --\n");
    pfind(pa_page); // (pid,va0)와 (pid,va_alias) 두 엔트리 기대
    print_vtop("[alias vtop#1]", va_alias);
    print_vtop("[alias vtop#2]", va_alias); // 두 번째 호출은 TLB hit 기대
    // tlb_print("after alias two vtop on alias");

    // 권한 변경: W 제거(RO). IPT 갱신 + SW-TLB invalidation 결과 확인
    if (protect_nowrite((void*)va_alias) < 0) {
        printf(1, "protect_nowrite fail\n"); exit();
    }
    if (vtop((void*)va_alias, &pa_chk, &fl_chk) == 0) {
        printf(1, "[after protect] va_alias flags=0x%x (expect W cleared)\n", fl_chk);
    }
    // tlb_print("after protect_nowrite + one vtop(alias)");

    // 3) 자식들 여러 명이 같은 PFN을 각자 매핑 → 체인 개수 증가 확인
    int kids = 3;
    for (int i=0; i<kids; i++) {
        int pid = fork();
        if (pid < 0) { printf(1, "fork fail\n"); exit(); }
        if (pid == 0) {
            // child: 자신의 힙 위쪽 빈 페이지에 같은 PFN 매핑
            uint brk = (uint)sbrk(0);
            uint va_c = (brk + 0x4000 + (i<<12)) & ~0xFFF);
            ensure_unmapped(va_c);

```

```

    if (vm_map_user_page(va_c, pa_page, PTE_U|PTE_W) < 0) {
        printf(1, "child map fail\n"); exit();
    }
    // 부모가 관측할 시간 제공
    sleep(50);
    // 해제 경로를 다양화: 일부는 언맵 후 종료(물리해제는 하지 않음)
    if ((i & 1) == 0) vm_unmap_one((void*)va_c, 0);
    exit();
}
}

// 부모: 아이들이 매핑을 붙인 뒤 관측
sleep(20);
printf(1, "-- with children --\n");
pfind(pa_page);

// 4) alias 해제 → 마지막 참조 아니면 프레임 유지(=double free 방지)
{
    int rc = vm_unmap_one((void*)va_alias, 1);
    if (rc < 0) { printf(1, "vm_unmap_one(alias) fail rc=%d\n", rc); exit(); }
}
print_vtop("[after unmap alias]", va_alias); // 음수(rc) 기대
printf(1, "-- after unmap(alias) --\n");
pfind(pa_page); // parent는 보통 va0만 남고, child 매핑들이 남아 있을 수 있음
// tlb_print("after unmap alias + one vtop(alias)");

// 5) 자식 종료 대기 → exit() 후 IPT 정리 확인
for (int i=0; i<kids; i++) wait();
printf(1, "-- after children exit --\n");
pfind(pa_page);

// 6) 부모의 마지막 참조 해제: top 페이지 반환 → 남은 참조가 0이면 실제 프레임 해제
if (sbrk(-4096) == (char*)-1) { printf(1, "sbrk(-4096) fail\n"); exit(); }
print_vtop("[after free va0]", va0); // 음수(rc) 기대
printf(1, "-- final pfind --\n");
pfind(pa_page); // 0 entries 기대(모든 참조 해제)

printf(1, "=== ptlb_memtest: done ===\n");
exit();
}

```

```

[spinlock.h]
#ifndef XV6_SPINLOCK_H
#define XV6_SPINLOCK_H

// Mutual exclusion lock.
struct spinlock {
    uint locked; // Is the lock held?

    // For debugging:
    char *name; // Name of lock.
    struct cpu *cpu; // The cpu holding the lock.
    uint pcs[10]; // The call stack (an array of program counters)
                  // that locked the lock.
};

#endif /* XV6_SPINLOCK */

```



```

[syscall.c]
#include "types.h"
#include "defs.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "proc.h"
#include "x86.h"
#include "syscall.h"

// User code makes a system call with INT T_SYSCALL.
// System call number in %eax.
// Arguments on the stack, from the user call to the C
// library system call function. The saved user %esp points
// to a saved program counter, and then the first argument.

// Fetch the int at addr from the current process.
/*int
fetchint(uint addr, int *ip)
{
    struct proc *curproc = myproc();

    if(addr >= curproc->sz || addr+4 > curproc->sz)
        return -1;
    *ip = *(int*)(addr);
    return 0;
} */

int fetchint(uint addr, int *ip)
{
    struct proc *cur = myproc();
    if(addr >= cur->sz || addr+4 > cur->sz)
        return -1;

    // 페이지 경계 안전 접근
    char *kva = uva2ka(cur->pgdir, (char*)PGROUNDDOWN(addr));
    if (!kva) return -1;
    *ip = *(int*)(kva + (addr & (PGSIZE-1)));
    return 0;
}

// Fetch the nul-terminated string at addr from the current process.
// Doesn't actually copy the string - just sets *pp to point at it.
// Returns length of string, not including nul.
/*int
fetchstr(uint addr, char **pp)
{
    char *s, *ep;
    struct proc *curproc = myproc();

    if(addr >= curproc->sz)
        return -1;
    *pp = (char*)addr;
    ep = (char*)curproc->sz;
    for(s = *pp; s < ep; s++){
        if(*s == 0)

```

```

        return s - *pp;
    }
    return -1;
}*/

// syscall.c
int
fetchstr(uint addr, char **pp)
{
    struct proc *cur = myproc();
    if (addr >= cur->sz) return -1;

    *pp = (char*)addr;          // exec()에서 그대로 user VA를 넘겨야 하므로 유지
    uint a = addr;
    uint end = cur->sz;

    while (a < end) {
        char *kva = uva2ka(cur->pgdir, (char*)PGROUNDDOWN(a));
        if (!kva) return -1;    // 해당 페이지가 없으면 실패

        uint off = a & (PGSIZE - 1);
        for (; off < PGSIZE && a < end; off++, a++) {
            if (kva[off] == 0)
                return a - addr; // NUL 발견 → 길이 반환(원래 xv6 관례)
        }
    }
    return -1;                  // NUL을 끝까지 못 찾음
}

// Fetch the nth 32-bit system call argument.
int
argint(int n, int *ip)
{
    return fetchint((myproc()->tf->esp) + 4 + 4*n, ip);
}

// Fetch the nth word-sized system call argument as a pointer
// to a block of memory of size bytes. Check that the pointer
// lies within the process address space.
int
argptr(int n, char **pp, int size)
{
    int i;
    struct proc *curproc = myproc();

    if(argint(n, &i) < 0)
        return -1;
    if(size < 0 || (uint)i >= curproc->sz || (uint)i+size > curproc->sz)
        return -1;
    *pp = (char*)i;
    return 0;
}

// Fetch the nth word-sized system call argument as a string pointer.
// Check that the pointer is valid and the string is nul-terminated.
// (There is no shared writable memory, so the string can't change
// between this check and being used by the kernel.)
int

```

```

argstr(int n, char **pp)
{
    int addr;
    if(argint(n, &addr) < 0)
        return -1;
    return fetchstr(addr, pp);
}

extern int sys_chdir(void);
extern int sys_close(void);
extern int sys_dup(void);
extern int sys_exec(void);
extern int sys_exit(void);
extern int sys_fork(void);
extern int sys_fstat(void);
extern int sys_getpid(void);
extern int sys_kill(void);
extern int sys_link(void);
extern int sys_mkdir(void);
extern int sys_mknod(void);
extern int sys_open(void);
extern int sys_pipe(void);
extern int sys_read(void);
extern int sys_sbrk(void);
extern int sys_sleep(void);
extern int sys_unlink(void);
extern int sys_wait(void);
extern int sys_write(void);
extern int sys_uptime(void);
extern int sys_dump_physmem_info(void);
extern int sys_vtop(void);
extern int sys_tlb_stats_sys(void);
extern int sys_phys2virt(void);
extern int sys_vm_alias_by_pa(void);
extern int sys_vm_unmap_one(void);
extern int sys_sw_tlb_reset(void);
extern int sys_sw_tlb_read(void);
extern int sys_vm_map_user_page(void);
extern int sys_protect_nowrite(void);

static int (*syscalls[])(void) = {
[SYS_fork]    sys_fork,
[SYS_exit]    sys_exit,
[SYS_wait]    sys_wait,
[SYS_pipe]    sys_pipe,
[SYS_read]    sys_read,
[SYS_kill]    sys_kill,
[SYS_exec]    sys_exec,
[SYS_fstat]    sys_fstat,
[SYS_chdir]    sys_chdir,
[SYS_dup]    sys_dup,
[SYS_getpid]    sys_getpid,
[SYS_sbrk]    sys_sbrk,
[SYS_sleep]    sys_sleep,
[SYS_uptime]    sys_uptime,
[SYS_open]    sys_open,
[SYS_write]    sys_write,
[SYS_mknod]    sys_mknod,

```

```

[SYS_unlink] sys_unlink,
[SYS_link] sys_link,
[SYS_mkdir] sys_mkdir,
[SYS_close] sys_close,
[SYS_dump_physmem_info] sys_dump_physmem_info,
[SYS_vtop] sys_vtop,
[SYS_tlb_stats_sys] sys_tlb_stats_sys,
[SYS_phys2virt] sys_phys2virt,
[SYS_vm_alias_by_pa] sys_vm_alias_by_pa,
[SYS_vm_unmap_one] sys_vm_unmap_one,
[SYS_sw_tlb_reset] sys_sw_tlb_reset,
[SYS_sw_tlb_read] sys_sw_tlb_read,
[SYS_vm_map_user_page] sys_vm_map_user_page,
[SYS_protect_nowrite] sys_protect_nowrite,
};

```

```

void
syscall(void)
{

```

```

    int num;
    struct proc *curproc = myproc();

```

```

    num = curproc->tf->eax;

```

```

    // --- pfind가 phys2virt 부를 때만 사전 로그 ---
    if (curproc &&
        strcmp(curproc->name, "pfind", 5) == 0 &&
        (num == SYS_phys2virt || num == SYS_vtop)) {
        cprintf("[syscall pre] pid=%d name=%s num=%d esp=0x%x eip=0x%x sz=0x%x\n",
            curproc->pid, curproc->name, num,
            curproc->tf->esp, curproc->tf->eip, curproc->sz);
    }

```

```

    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        curproc->tf->eax = syscalls[num]();
    } else {
        cprintf("%d %s: unknown sys call %d\n",
            curproc->pid, curproc->name, num);
        curproc->tf->eax = -1;
    }
}

```

```

[syscall.h]
// System call numbers
#define SYS_fork    1
#define SYS_exit    2
#define SYS_wait    3
#define SYS_pipe    4
#define SYS_read    5
#define SYS_kill    6
#define SYS_exec    7
#define SYS_fstat    8
#define SYS_chdir    9
#define SYS_dup     10
#define SYS_getpid  11
#define SYS_sbrk     12
#define SYS_sleep   13

```

```

#define SYS_uptime 14
#define SYS_open 15
#define SYS_write 16
#define SYS_mknod 17
#define SYS_unlink 18
#define SYS_link 19
#define SYS_mkdir 20
#define SYS_close 21
#define SYS_dump_physmem_info 22
#define SYS_vtop 23
#define SYS_tlb_stats_sys 26
#define SYS_phys2virt 28
#define SYS_vm_alias_by_pa 29
#define SYS_vm_unmap_one 30
#define SYS_sw_tlb_reset 32
#define SYS_sw_tlb_read 33
#define SYS_vm_map_user_page 34
#define SYS_protect_nowrite 36

```

```

[sysproc.c]
#include "types.h"
#include "x86.h"
#include "defs.h"
#include "date.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "proc.h"
#include "physmem.h" // struct physframe_info 정의가 보이게 추가
#include "ipt.h"
#include "tlb.h"
#include "vlist.h"

```

```

int
sys_fork(void)
{
    return fork();
}

```

```

int
sys_exit(void)
{
    exit();
    return 0; // not reached
}

```

```

int
sys_wait(void)
{
    return wait();
}

```

```

int
sys_kill(void)
{
    int pid;

    if(argint(0, &pid) < 0)

```

```

        return -1;
    return kill(pid);
}

int
sys_getpid(void)
{
    return myproc()->pid;
}

/*int
sys_sbrk(void)
{
    int addr;
    int n;

    if(argint(0, &n) < 0)
        return -1;
    addr = myproc()->sz;
    if(growproc(n) < 0)
        return -1;
    return addr;
} */

int sys_sbrk(void){
    int n; struct proc *p = myproc();
    if(argint(0,&n) < 0) return -1;
    int addr = p->sz;
    if(n > 0){
        if(growproc(n) < 0) return -1; // 내부에서 allocuvn → mappages → ipt_add
    }else if(n < 0){
        // deallocuvn → vm_unmap_one → ipt_remove
        if(deallocuvn(p->pgdir, addr, addr+n) == 0) return -1;
        p->sz += n;
    }
    return addr;
}

int
sys_sleep(void)
{
    int n;
    uint ticks0;

    if(argint(0, &n) < 0)
        return -1;
    acquire(&tickslock);
    ticks0 = ticks;
    while(ticks - ticks0 < n){
        if(myproc()->killed){
            release(&tickslock);
            return -1;
        }
        sleep(&ticks, &tickslock);
    }
    release(&tickslock);
    return 0;
}

```

```

}

// return how many clock tick interrupts have occurred
// since start.
int
sys_uptime(void)
{
    uint xticks;

    acquire(&tickslock);
    xticks = ticks;
    release(&tickslock);
    return xticks;
}

// 전역 물리 페이지 관리 테이블 관련 시스템콜
int
sys_dump_physmem_info(void)
{
    char *uaddr;
    int maxe;

    // 1) 두 번째 인자부터 먼저 읽어서 크기 결정
    if (argint(1, &maxe) < 0) return -1;
    if (maxe <= 0) return -1;

    // 2) 첫 번째 인자(버퍼 포인터) 유효성 검사: 필요한 총 바이트 수로 체크
    if (argptr(0, &uaddr, maxe * sizeof(struct physframe_info)) < 0) return -1;

    // 3) 실제 작업 호출
    return dump_physmem_info_kernel(uaddr, maxe);
}

int sys_vtop(void)
{
    uint32_t va_arg;          // raw user VA (정수로 받기)
    uint32_t u_pa_out_arg;    // user buffer address to write PA
    uint32_t u_flags_out_arg; // user buffer address to write flags

    // 0: VA 값
    if (argint(0, (int*)&va_arg) < 0) return -1;
    // 1: pa_out 주소
    if (argint(1, (int*)&u_pa_out_arg) < 0) return -1;
    // 2: flags_out 주소
    if (argint(2, (int*)&u_flags_out_arg) < 0) return -1;

    uint32_t pa = 0, flags = 0;
    struct proc *p = myproc();
    if (!p) return -1;

    int rc = sw_vtop(p->pgdir, (void*)va_arg, &pa, &flags);

    // 성공일 때만 결과를 user buffer로 복사
    if (rc == 0) {
        if (copyout(p->pgdir, (uint)u_pa_out_arg, (char*)&pa, sizeof(pa)) < 0) return -1;
        if (copyout(p->pgdir, (uint)u_flags_out_arg, (char*)&flags, sizeof(flags)) < 0) return -1;
    }
}

```

```

// --- 선택: 하드웨어 참조와 비교 ---
// vm.c에 hw_walk_probe 구현 & defs.h에 프로토타입 선언해둠
// 안 쓸 거면 이 블록을 주석(#if 0) 처리
#ifdef VTOP_DEBUG
extern int hw_walk_probe(pde_t *pgdir, const void *va, uint *pa_out, uint *fl_out);
uint hw_pa=0, hw_flags=0;
int hw_rc = hw_walk_probe(p->pgdir, (void*)va_arg, &hw_pa, &hw_flags);
if (hw_rc==0 && (hw_pa != pa || hw_flags != flags)) {
    cprintf("[mismatch] va=0x%x sw:pa=0x%x fl=0x%x hw:pa=0x%x fl=0x%x\n",
        va_arg, pa, flags, hw_pa, hw_flags);
}
#endif

return rc;
}

```

```

// (추가) SW_TLB를 위한 시스템콜 핸들러 구현
int sys_tlb_stats_sys(void){
    struct tlb_stats st;
    if (tlb_get_stats(&st) < 0) return -1;
    char *u;
    if (argptr(0, &u, sizeof(st)) < 0) return -1;
    return copyout(myproc()->pgdir, (uint)u, (char*)&st, sizeof(st)) < 0 ? -1 : 0;
}

```

```

int
sys_sw_tlb_reset(void)
{
    int mode = 0;          // 0: 카운터만 0, 1: 엔트리 flush + 카운터 0
    argint(0, &mode);
    if (mode == 1) tlb_reset_all();
    else          tlb_reset_counters();
    return 0;
}

```

```

int
sys_sw_tlb_read(void)
{
    char *uaddr;
    if (argptr(0, &uaddr, sizeof(struct tlb_stats)) < 0)
        return -1;

    struct tlb_stats s;    // ★ tlb.h의 것 (모든 멤버 uint32_t)
    tlb_get_stats(&s);     // 내부에서 채움

    if (copyout(myproc()->pgdir, (uint)uaddr, (char*)&s, sizeof(s)) < 0)
        return -1;

    return 0;
}

```

```

// phys2virt(pa_page, out[], max)
int sys_phys2virt(void) {
    uint pa_page; int max; uint out_uva;
    if (argint(0, (int*)&pa_page) < 0) return -1;
    if (argint(1, (int*)&out_uva) < 0) return -1; // ← 순서를 0,1,2로 통일(가독성)
}

```



```

if (argint(2, &max) < 0) return -1;
if (max < 0) return -1;
if (max > 1024) max = 1024;

cprintf("[phys2virt] pa_page=0x%x out_uva=0x%x max=%d sizeof(vlist)=%d (kernel)\n",
        pa_page, out_uva, max, (int)sizeof(struct vlist));

// 필요시 상한 설정: if (max > 1024) max = 1024;

uint target_pfn = PFN(pa_page);
int n = 0;
struct vlist tmp;

// 출력 버퍼 범위 검사 (오버플로우 방지 포함)
uint need = (uint)max * (uint)sizeof(struct vlist);
if (max > 0) {
    if (out_uva >= myproc()->sz) return -1;
    if (need > myproc()->sz) return -1;
    if (out_uva + need < out_uva) return -1;          // wrap-around
    if (out_uva + need > myproc()->sz) return -1;    // 범위 초과
}

acquire(&ipt_lock);
for (int i = 0; i < IPT_BUCKETS && n < max; i++) {
    for (struct ipt_entry *e = ipt_hash[i]; e && n < max; e = e->next) {
        if (e->pfn == target_pfn) {
            tmp.pid = e->pid;
            tmp.va_page = e->va;
            tmp.flags = e->flags;

            uint dst = out_uva + n * sizeof(struct vlist);
            cprintf("[phys2virt] copyout idx=%d dst=0x%x pid=%d va=0x%x flags=0x%x\n",
                    n, dst, tmp.pid, tmp.va_page, tmp.flags);

            // out_uva + n*sizeof(struct vlist) 에 1개씩 안전하게 복사
            if (copyout(myproc()->pgdir,
                        out_uva + n * sizeof(struct vlist),
                        (char*)&tmp, sizeof(tmp)) < 0) {
                release(&ipt_lock);
                return -1; // 가드 페이지 등으로 쓰기 실패 → 깔끔히 실패 반환
            }
            n++;
        }
    }
}
release(&ipt_lock);
return n;
}

// vm_alias_by_pa(pa_page, dst_va, flags)
int sys_vm_alias_by_pa(void) {
    uint pa_page, dst_va, flags;
    if (argint(0, (int*)&pa_page) < 0) return -1;
    if (argint(1, (int*)&dst_va) < 0) return -1; // << 변경: argptr → argint
    if (argint(2, (int*)&flags) < 0) return -1;

```

```

uint va_page = PGROUNDDOWN(dst_va);
uint pa_aln  = PTE_ADDR(pa_page);

pde_t *pgdir = myproc()->pgdir;
pde_t pde    = pgdir[PDX(va_page)];
if (pde & PTE_P) {
    pte_t *pt = (pte_t*)P2V(PTE_ADDR(pde));
    pte_t *pte = &pt[PTX(va_page)];
    if (*pte & PTE_P) return -1;    // 이미 매핑되어 있으면 금지
}
return vm_map_user_page(pgdir, va_page, pa_aln, flags);
}

int sys_vm_unmap_one(void) {
    uint uva;
    int free_if_last;
    if (argint(0, (int*)&uva) < 0) return -1;        // 포인터 값을 '정수'로 받기
    if (argint(1, &free_if_last) < 0) return -1;

    return vm_unmap_one(myproc()->pgdir, PGROUNDDOWN(uva), free_if_last, 0);
    // 마지막 인자 0: '해당 VA가 없어도 -1로 에러 반환' 하지 않도록(필요 시 정책대로)
}

// sys_vm_map_user_page(va_page, pa_page, perm)
int sys_vm_map_user_page(void) {
    uint va_page, pa_page;
    int perm;
    if (argint(0, (int*)&va_page) < 0) return -1;
    if (argint(1, (int*)&pa_page) < 0) return -1;
    if (argint(2, &perm) < 0) return -1;

    // 페이지 정렬 요구
    if ((va_page & (PGSIZE-1)) || (pa_page & (PGSIZE-1))) return -1;

    struct proc *p = myproc();
    return vm_map_user_page(p->pgdir, va_page, pa_page, perm);
}

int sys_protect_nowrite(void){
    int uva;
    if (argint(0, &uva) < 0) return -1;
    return protect_nowrite(myproc()->pgdir, (void*)uva);
}

[tlb.c]
// tlb.c - 소프트웨어 TLB(집합-연관 캐시) 구현부: 조회/삽입/무효화/통계 제공
#include "types.h"        // <stdint.h> 대신 types.h 사용
#include "defs.h"
#include "mmu.h"
#include "proc.h"
#include "spinlock.h"
#include "tlb.h"

// 컴파일 타임 검사: 세트 수는 반드시 2의 거듭제곱(마스킹 인덱싱을 위해)

```

```

#if (SOFT_TLB_SETS & (SOFT_TLB_SETS - 1)) != 0
#error "SOFT_TLB_SETS must be power of two"
#endif

// 세트 인덱스 해시 함수(간단 혼합 + Knuth 상수 사용). 결과는 [0, SOFT_TLB_SETS)로 마스킹.
static inline uint32_t tlb_index(uint32_t pid, uint32_t va_page) {
    // Knuth multiplicative hash
    uint32_t h = pid ^ (va_page >> 12) ^ (va_page * U32_C(2654435761));
    return h & (uint32_t)(SOFT_TLB_SETS - 1U);
}

// 전역 TLB 인스턴스(락/엔트리/카운터 포함)
struct soft_tlb g_tlb;

// 초기화: 엔트리 invalid 클리어 + 통계 카운터 0 설정
void tlb_init(void){
    initlock(&g_tlb.lock, "softtlb");
    acquire(&g_tlb.lock);
    for (int s = 0; s < SOFT_TLB_SETS; s++)
        for (int w = 0; w < SOFT_TLB_WAYS; w++)
            g_tlb.sets[s][w].valid = (uint8_t)0;
    g_tlb.hits = g_tlb.misses = g_tlb.evicts = g_tlb.invalidations = g_tlb.lookups = U32_C(0);
    release(&g_tlb.lock);
}

// 조회: (pid, va_page)로 탐색. 히트 시 pa/flags 반환, age 증가. 미스 시 0 반환.
int tlb_lookup(uint32_t pid, uint32_t va_page, uint32_t *pa_out, uint16_t *fl_out){
    acquire(&g_tlb.lock);
    g_tlb.lookups++;
    uint32_t s = tlb_index(pid, va_page);
    for (int w = 0; w < SOFT_TLB_WAYS; w++) {
        struct tlb_entry *e = &g_tlb.sets[s][w];
        if (e->valid && e->pid == pid && e->va_page == va_page) {
            g_tlb.hits++;
            e->age++; // 간단 LRU 근사: 접근 시 age 가산
            if (pa_out) *pa_out = e->pa_page;
            if (fl_out) *fl_out = e->flags;
            release(&g_tlb.lock);
            return 1;
        }
    }
    g_tlb.misses++;
    release(&g_tlb.lock);
    return 0;
}

// 교체 대상 선택(락 없이 내부에서만 호출). 빈 슬롯 우선, 없으면 최소 age 선택(LRU 근사).
static int victim_way_nolock(uint32_t s){
    #if SOFT_TLB_WAYS == 1
        return 0;
    #else
        int v = 0;
        uint32_t minage = g_tlb.sets[s][0].age;
        for (int w = 0; w < SOFT_TLB_WAYS; w++) {
            if (!g_tlb.sets[s][w].valid) return w;
            if (g_tlb.sets[s][w].age < minage) {
                minage = g_tlb.sets[s][w].age;
                v = w;
            }
        }
    #endif
}

```

```

    }
}
return v;
#endif
}

// 삽입/갱신: 동일 키 존재 시 갱신, 없으면 victim 선정 후 교체. 교체 시 evicts++
void tlb_insert(uint32_t pid, uint32_t va_page, uint32_t pa_page, uint16_t flags){
    acquire(&g_tlb.lock);
    uint32_t s = tlb_index(pid, va_page);

    // 동일 키 갱신 경로
    for (int w = 0; w < SOFT_TLB_WAYS; w++) {
        struct tlb_entry *e = &g_tlb.sets[s][w];
        if (e->valid && e->pid == pid && e->va_page == va_page) {
            e->pa_page = pa_page;
            e->flags = flags;
            e->age++;
            release(&g_tlb.lock);
            return;
        }
    }

    // 교체 경로
    int wv = victim_way_nolock(s);
    struct tlb_entry *v = &g_tlb.sets[s][wv];
    if (v->valid) g_tlb.evicts++;
    v->pid = pid;
    v->va_page = va_page;
    v->pa_page = pa_page;
    v->flags = flags;
    v->valid = (uint8_t)1;
    v->age = U32_C(1);
    release(&g_tlb.lock);
}

// 단일 엔트리 무효화: (pid, va_page) 일치 항목을 valid=0로 표기, invalidations++
void tlb_invalidate_one(uint32_t pid, uint32_t va_page){
    acquire(&g_tlb.lock);
    uint32_t s = tlb_index(pid, va_page);
    for (int w = 0; w < SOFT_TLB_WAYS; w++) {
        struct tlb_entry *e = &g_tlb.sets[s][w];
        if (e->valid && e->pid == pid && e->va_page == va_page) {
            e->valid = (uint8_t)0;
            g_tlb.invalidations++;
        }
    }
    release(&g_tlb.lock);
}

// 특정 PID 소유 엔트리 전부 무효화(invalidate all for PID)
void tlb_invalidate_pid(uint32_t pid){
    acquire(&g_tlb.lock);
    for (int s = 0; s < SOFT_TLB_SETS; s++)
        for (int w = 0; w < SOFT_TLB_WAYS; w++) {
            struct tlb_entry *e = &g_tlb.sets[s][w];
            if (e->valid && e->pid == pid) {
                e->valid = (uint8_t)0;
            }
        }
}

```

```

        g_tlb.invalidations++;
    }
}
release(&g_tlb.lock);
}

// 전체 무효화(global flush). 한 번의 연산으로 invalidations를 1 증가시켜 이벤트로 집계.
void tlb_invalidate_all(void){
    acquire(&g_tlb.lock);
    for (int s = 0; s < SOFT_TLB_SETS; s++)
        for (int w = 0; w < SOFT_TLB_WAYS; w++)
            g_tlb.sets[s][w].valid = (uint8_t)0;
    g_tlb.invalidations++;
    release(&g_tlb.lock);
}

// 통계 스냅샷 획득: 락 하에 로컬 변수로 복사 후, 락 해제하고 사용자 버퍼로 memmove
int tlb_get_stats(struct tlb_stats *out){
    struct tlb_stats s;
    acquire(&g_tlb.lock);
    s.hits          = g_tlb.hits;
    s.misses        = g_tlb.misses;
    s.evicts        = g_tlb.evicts;
    s.invalidations = g_tlb.invalidations;
    s.lookups       = g_tlb.lookups;
    s.lines         = SOFT_TLB_LINES;
    s.ways          = SOFT_TLB_WAYS;
    s.sets          = SOFT_TLB_SETS;
    release(&g_tlb.lock);
    memmove(out, &s, sizeof(s));
    return 0;
}

// 엔트리 덤프: (옵션) 유효 엔트리만 수집. 최대 max개까지 채움.
int tlb_dump_entries(struct tlb_entry *buf, int max, int only_valid){
    int n = 0;
    acquire(&g_tlb.lock);
    for (int s = 0; s < SOFT_TLB_SETS && n < max; s++)
        for (int w = 0; w < SOFT_TLB_WAYS && n < max; w++) {
            struct tlb_entry e = g_tlb.sets[s][w];
            if (!only_valid || e.valid) buf[n++] = e;
        }
    release(&g_tlb.lock);
    return n;
}

// (일관성 보조) 단일 VA 무효화: 모든 세트를 순회하며 (pid,va_page) 일치 엔트리를 invalid
void tlb_invalidate_va(uint32_t pid, uint32_t va_page) {
    acquire(&g_tlb.lock);
    g_tlb.invalidations++;
    for (int s = 0; s < SOFT_TLB_SETS; s++) {
        for (int w = 0; w < SOFT_TLB_WAYS; w++) {
            struct tlb_entry *e = &g_tlb.sets[s][w];
            if (e->valid && e->pid == pid && e->va_page == va_page) {
                e->valid = (uint8_t)0;
            }
        }
    }
}

```

```

    release(&g_tlb.lock);
}

// (일관성 보조) VA 범위 무효화: [a, b) 페이지 경계로 순회하며 tlb_invalidate_va 호출
void tlb_invalidate_range(uint32_t pid, uint32_t a, uint32_t b) {
    for (uint32_t va = (uint32_t)PGROUNDDOWN(a); va < b; va += (uint32_t)PGSIZE)
        tlb_invalidate_va(pid, va);
}

// 카운터만 0으로 리셋(엔트리는 보존). 측정 구간 초기화용.
void tlb_reset_counters(void) {
    acquire(&g_tlb.lock);
    g_tlb.hits = g_tlb.misses = g_tlb.evicts = g_tlb.invalidations = g_tlb.lookups = 0;
    release(&g_tlb.lock);
}

// 전체 초기화: 모든 엔트리 invalid + 모든 카운터 0 (하드 리셋)
void tlb_reset_all(void) {
    acquire(&g_tlb.lock);
    for (int s=0; s<SOFT_TLB_SETS; s++)
        for (int w=0; w<SOFT_TLB_WAYS; w++)
            g_tlb.sets[s][w].valid = 0;
    g_tlb.hits = g_tlb.misses = g_tlb.evicts = g_tlb.invalidations = g_tlb.lookups = 0;
    release(&g_tlb.lock);
}

[tlb.h]
// tlb.h - 소프트웨어 TLB(집합-연관) 헤더: 엔트리 정의/전역 상태/인터페이스 선언
#pragma once
#include "types.h"
#include "spinlock.h"

// 구성 파라미터: 총 라인수 = lines, 집합 연관도 = ways, 세트수 = lines/ways
// tlb.c에서 SOFT_TLB_SETS가 2의 거듭제곱인지 컴파일 타임 검사함.
#define SOFT_TLB_WAYS 2 // 세트당 way 수(연관도)
#define SOFT_TLB_LINES 64 // 총 캐시 라인 수(= 엔트리 수)
#define SOFT_TLB_SETS (SOFT_TLB_LINES / SOFT_TLB_WAYS) // 세트 개수

// 단일 TLB 엔트리: (pid, va_page) → (pa_page, flags)
// - va_page/pa_page: 페이지 정렬된 VA/PA(페이지 단위 키)
// - flags: PTE 권한 비트(P/W/U 등) 스냅샷
// - valid: 유효 플래그(0/1)
// - age : LRU 근사용 카운터(접근/갱신 시 증가)
struct tlb_entry {
    uint32_t pid;
    uint32_t va_page;
    uint32_t pa_page;
    uint16_t flags;
    uint8_t valid;
    uint32_t age;
};

// 전역 소프트 TLB: 락, 세트-웨이 엔트리 배열, 통계 카운터
struct soft_tlb {
    struct spinlock lock; // 동시성 보호용 락
    struct tlb_entry sets[SOFT_TLB_SETS][SOFT_TLB_WAYS]; // [세트][웨이] 엔트리
    uint32_t hits, misses, evicts, invalidations, lookups; // 통계
};

```

```

// 전역 인스턴스(정의는 tlb.c에 있음)
extern struct soft_tlb g_tlb;

// 초기화/조회/삽입
void tlb_init(void); // 엔트리 invalid, 카운터 0으로 초기화
int tlb_lookup(uint32_t pid, uint32_t va_page, uint32_t *pa_page_out, uint16_t *flags_out); // 히트 시 1, 미스 0
void tlb_insert(uint32_t pid, uint32_t va_page, uint32_t pa_page, uint16_t flags); // 동일 키면 갱신, 없으면 교체

// 무효화(Invalidate)
void tlb_invalidate_one(uint32_t pid, uint32_t va_page); // 특정 (pid,va) 한 건
void tlb_invalidate_pid(uint32_t pid); // 특정 PID 전범위
void tlb_invalidate_all(void); // 전 엔트리

// 카운터/전체 리셋
void tlb_reset_counters(void); // 통계 카운터만 0으로(엔트리는 유지)
void tlb_reset_all(void); // 엔트리까지 전부 무효화 + 카운터 0

// 일관성 보조(Consistency): vm 변경에 연동해 호출
void tlb_invalidate_va(uint32_t pid, uint32_t va_page); // 단일 VA 페이지 무효화
void tlb_invalidate_range(uint32_t pid, uint32_t va_start, uint32_t va_end); // [va_start, va_end) 범위(페이지 단위)

// 통계/디버깅
struct tlb_stats {
    uint32_t hits, misses, evicts, invalidations, lookups; // 동작 통계
    uint32_t lines, ways, sets; // 구성 파라미터 스냅샷
};
int tlb_get_stats(struct tlb_stats *out); // 통계 복사
int tlb_dump_entries(struct tlb_entry *buf, int max, int only_valid); // 엔트리 덤프(유효만 선택 가능)

[tlbstat.c]
// tlbstat.c (fixed) - 소프트웨어 TLB 통계 1줄 요약 출력 유틸리티
#include "types.h"
#include "stat.h"
#include "user.h"

/*-----
 * print_stats: 커널에서 전달된 struct tlb_stats를 사람이 읽기 쉬운 한 줄로 출력
 * - hits/misses/lookups/evicts/invalidations: 동작 카운터
 * - lines/ways/sets: TLB 구성 파라미터(총 라인수, 연관도, 세트수)
 *-----*/
static void print_stats(const struct tlb_stats *s) {
    printf(1,
        "TLB: hits=%d misses=%d lookups=%d evicts=%d invalidations=%d lines=%d ways=%d sets=%d\n",
        s->hits, s->misses, s->lookups, s->evicts, s->invalidations,
        s->lines, s->ways, s->sets);
}

int
main(int argc, char *argv[])
{
    struct tlb_stats st; // 커널이 채워줄 통계 구조체 버퍼

    // tlb_stats_sys: 시스템콜 래퍼(커널의 소프트 TLB 통계를 사용자 공간으로 복사)
    if (tlb_stats_sys(&st) < 0) {
        printf(1, "tlbstat: tlb_stats_sys failed\n");
        exit();
    }
}

```

```

    print_stats(&st);          // 수집한 통계를 한 줄로 출력
    exit();
}

```

[types.h]

// types.h

#pragma once

// ===== 1) 고정폭 정수 타입 (stdint.h 없이) =====

// xv6 32-bit 전제: unsigned int == 32-bit

typedef unsigned int uint32_t; // 32-bit unsigned

typedef int int32_t; // 32-bit signed

typedef unsigned short uint16_t; // 16-bit unsigned

typedef short int16_t; // 16-bit signed

typedef unsigned char uint8_t; // 8-bit unsigned

typedef signed char int8_t; // 8-bit signed

// ===== 2) 폭 검증 (툴체인이 C11 지원 시) =====

#if defined(__STDC_VERSION__) && (__STDC_VERSION__ >= 201112L)

_Static_assert(sizeof(uint32_t) == 4, "uint32_t must be 4 bytes");

_Static_assert(sizeof(uint16_t) == 2, "uint16_t must be 2 bytes");

_Static_assert(sizeof(uint8_t) == 1, "uint8_t must be 1 byte");

#endif

// ===== 3) 32/16/8비트 상수 매크로 (표준 UINT32_C 대체) =====

#define U32_C(x) x##U // 이 커널에서 unsigned int는 32-bit 가정

#define U16_C(x) x##U

#define U8_C(x) x##U

// ===== 4) 기존 별칭 유지용 브리지 =====

// 기존 코드에 남아있는 uint/ushort/uchar를 잠정 호환하기 위해 선언

typedef uint32_t uint;

typedef uint16_t ushort;

typedef uint8_t uchar;

typedef uint pde_t;

[user.h]

#include "phymem.h"

#include "vlist.h"

struct stat;

struct rtcdate;

// system calls

int fork(void);

int exit(void) __attribute__((noreturn));

int wait(void);

int pipe(int*);

int write(int, const void*, int);

int read(int, void*, int);

int close(int);

int kill(int);

int exec(char*, char**);


```

int open(const char*, int);
int mknod(const char*, short, short);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int getpid(void);
char* sbrk(int);
int sleep(int);
int uptime(void);

```

```

// ulib.c
int stat(const char*, struct stat*);
char* strcpy(char*, const char*);
void *memmove(void*, const void*, int);
char* strchr(const char*, char c);
int strcmp(const char*, const char*);
void printf(int, const char*, ...);
char* gets(char*, int max);
uint strlen(const char*);
void* memset(void*, int, uint);
void* malloc(uint);
void free(void*);
int atoi(const char*);

```

```

// #include "physmem.h" // 유저 프로그램, 커널이 모두 같은 구조체 사용하도록 공용 헤더 추가
int dump_physmem_info(void *addr, int max_entries); // 시스템콜 프로토타입

```

```

// 가상주소 → 물리주소 변환(sw_vtop 커널 함수 기반 사용자 버전)
int vtop(void *va, uint32_t *pa_out, uint32_t *flags_out);

```

```

// 소프트웨어 TLB 관련 구조체 및 시스템콜 정의
struct tlb_stats {
    uint32_t hits, misses, evicts, invalidations, lookups; // 동작 통계
    uint32_t lines, ways, sets; // TLB 구성 파라미터
};

```

```

struct tlb_entry; // 내부 구조체 참조용(정의는 커널측 tlb.h에 존재)

```

```

// 소프트웨어 TLB 통계 조회 시스템콜
int tlb_stats_sys(struct tlb_stats *out);

```

```

// 소프트웨어 TLB 리셋: mode=0(카운터만 초기화), 1(엔트리 포함 전체 초기화)
int sw_tlb_reset(int mode);

```

```

// 소프트웨어 TLB 현재 상태를 읽어오는 시스템콜
int sw_tlb_read(struct tlb_stats *out);

```

```

// 물리 ↔ 가상 매핑 관련 시스템콜
int phys2virt(uint pa_page, struct vlist *out, int max); // 물리 페이지 → (pid, va, flags) 역조회
int vm_alias_by_pa(uint pa_page, void *dst_va, uint flags); // 동일 물리 페이지를 새로운 VA에 alias 매핑
int vm_unmap_one(void *va, int free_if_last); // 지정 VA 매핑 해제 (마지막 참조면 프레임 해제)
int vm_map_user_page(uint va_page, uint pa_page, int perm); // 지정 VA에 물리 페이지 수동 매핑
int protect_nowrite(void *va); // VA의 쓰기 권한 제거(RO 보호)

```

```
[usys.S]
#include "syscall.h"
#include "traps.h"

#define SYSCALL(name) \
.globl name; \
name: \
    movl $SYS_ ## name, %eax; \
    int $T_SYSCALL; \
    ret
```

```
SYSCALL(fork)
SYSCALL(exit)
SYSCALL(wait)
SYSCALL(pipe)
SYSCALL(read)
SYSCALL(write)
SYSCALL(close)
SYSCALL(kill)
SYSCALL(exec)
SYSCALL(open)
SYSCALL(mknod)
SYSCALL(unlink)
SYSCALL(fstat)
SYSCALL(link)
SYSCALL(mkdir)
SYSCALL(chdir)
SYSCALL(dup)
SYSCALL(getpid)
SYSCALL(sbrk)
SYSCALL(sleep)
SYSCALL(uptime)
SYSCALL(dump_physmem_info)
SYSCALL(vtop)
SYSCALL(tlb_stats_sys)
SYSCALL(phys2virt)
SYSCALL(vm_alias_by_pa)
SYSCALL(vm_unmap_one)
SYSCALL(sw_tlb_reset)
SYSCALL(sw_tlb_read)
SYSCALL(vm_map_user_page)
SYSCALL(protect_nowrite)
```

```
[vlist.h]
// vlist.h (커널/유저 공용)
#ifndef XV6_VLIST_H
#define XV6_VLIST_H
#include "types.h"
```

```
// 물리 페이지(Physical Page)에 매핑된 가상 주소(Virtual Page)들의 정보를 담는 구조체
// - pid      : 해당 VA를 사용하는 프로세스의 PID
// - va_page   : 페이지 단위로 정렬된 가상 주소 (하위 12비트는 0)
// - flags     : 해당 VA의 PTE 플래그 (P/W/U 등 권한 비트)
```

```

struct vlist {
    int pid;
    uint va_page;
    uint flags;
};
#endif

```

```

[vm.c]
#include "param.h"
#include "types.h"
#include "defs.h"
#include "x86.h"
#include "memlayout.h"
#include "mmu.h"
#include "proc.h"
#include "elf.h"

```

```

// (2) 역페이지 테이블용 헤더 추가
#include "ipt.h"

```

```

// (3) SW TLB
#include "tlb.h"

```

```

extern char data[]; // defined by kernel.ld
pde_t *kpgdir; // for use in scheduler()

```

```

// Set up CPU's kernel segment descriptors.
// Run once on entry on each CPU.

```

```

void
seginit(void)
{
    struct cpu *c;

    // Map "logical" addresses to virtual addresses using identity map.
    // Cannot share a CODE descriptor for both kernel and user
    // because it would have to have DPL_USR, but the CPU forbids
    // an interrupt from CPL=0 to DPL=3.
    c = &cpus[cuid()];
    c->gdt[SEG_KCODE] = SEG(STA_X|STA_R, 0, 0xffffffff, 0);
    c->gdt[SEG_KDATA] = SEG(STA_W, 0, 0xffffffff, 0);
    c->gdt[SEG_UCODE] = SEG(STA_X|STA_R, 0, 0xffffffff, DPL_USER);
    c->gdt[SEG_UDATA] = SEG(STA_W, 0, 0xffffffff, DPL_USER);
    lgdt(c->gdt, sizeof(c->gdt));
}

```

```

// Return the address of the PTE in page table pgdir
// that corresponds to virtual address va. If alloc!=0,
// create any required page table pages.

```

```

static pte_t *
walkpgdir(pde_t *pgdir, const void *va, int alloc)
{
    pde_t *pde;
    pte_t *pgtab;

    pde = &pgdir[PDX(va)];
    if(*pde & PTE_P){

```

```

    pgtab = (pte_t*)P2V(PTE_ADDR(*pde));
} else {
    if(!alloc || (pgtab = (pte_t*)kalloc()) == 0)
        return 0;
    // Make sure all those PTE_P bits are zero.
    memset(pgtab, 0, PGSIZE);
    // The permissions here are overly generous, but they can
    // be further restricted by the permissions in the page table
    // entries, if necessary.
    *pde = V2P(pgtab) | PTE_P | PTE_W | PTE_U;
}
return &pgtab[PTX(va)];
}

// Create PTEs for virtual addresses starting at va that refer to
// physical addresses starting at pa. va and size might not
// be page-aligned.
static int
mappages(pde_t *pgdir, void *va, uint size, uint pa, int perm)
{
    char *a, *last;
    pte_t *pte;

    a = (char*)PGROUNDDOWN((uint)va);
    last = (char*)PGROUNDDOWN(((uint)va) + size - 1);

    for (;;) {
        if ((pte = walkpgdir(pgdir, a, 1)) == 0)
            return -1;
        if (*pte & PTE_P)
            panic("remap");

        // 실제 PTE 기록
        uint32_t pa_page = PTE_ADDR(pa);                // page-aligned PA
        *pte = pa_page | perm | PTE_P;

        // ---- IPT 삽입: 유저 영역만 추적 ----
        if ((uint)a < KERNBASE) {
            int pid = (myproc() && myproc()->pgdir == pgdir)
                ? myproc()->pid
                : ipt_pid_from_pgdir(pgdir);            // 없으면 -1
            if (pid > 0) {                                // pid > 0 일 때만 삽입
                uint32_t va_page = (uint32_t)PGROUNDDOWN((uint)a);
                uint32_t pte_flags = (*pte) & (PTE_P | PTE_W | PTE_U);
                ipt_add(pgdir, (uint32_t)pid, va_page, pa_page, pte_flags);
            }
        }

        // 새 매핑은 remap이 아니므로 SW-TLB invalidation 불필요

        if (a == last)
            break;
        a += PGSIZE;
        pa += PGSIZE;
    }
    return 0;
}

```

```

// There is one page table per process, plus one that's used when
// a CPU is not running any process (kpgdir). The kernel uses the
// current process's page table during system calls and interrupts;
// page protection bits prevent user code from using the kernel's
// mappings.
//
// setupkvm() and exec() set up every page table like this:
//
// 0..KERNBASE: user memory (text+data+stack+heap), mapped to
//                phys memory allocated by the kernel
// KERNBASE..KERNBASE+EXTMEM: mapped to 0..EXTMEM (for I/O space)
// KERNBASE+EXTMEM..data: mapped to EXTMEM..V2P(data)
//                for the kernel's instructions and r/o data
// data..KERNBASE+PHYSTOP: mapped to V2P(data)..PHYSTOP,
//                rw data + free physical memory
// 0xfe000000..0: mapped direct (devices such as ioapic)
//
// The kernel allocates physical memory for its heap and for user memory
// between V2P(end) and the end of physical memory (PHYSTOP)
// (directly addressable from end..P2V(PHYSTOP)).

// This table defines the kernel's mappings, which are present in
// every process's page table.
static struct kmap {
    void *virt;
    uint phys_start;
    uint phys_end;
    int perm;
} kmap[] = {
    { (void*)KERNBASE, 0,          EXTMEM,    PTE_W}, // I/O space
    { (void*)KERNLINK, V2P(KERNLINK), V2P(data), 0},   // kern text+rodata
    { (void*)data,     V2P(data),    PHYSTOP,   PTE_W}, // kern data+memory
    { (void*)DEVSPACE, DEVSPACE,     0,        PTE_W}, // more devices
};

// Set up kernel part of a page table.
pde_t*
setupkvm(void)
{
    pde_t *pgdir;
    struct kmap *k;

    if((pgdir = (pde_t*)kalloc()) == 0)
        return 0;
    memset(pgdir, 0, PGSIZE);
    if (P2V(PHYSTOP) > (void*)DEVSPACE)
        panic("PHYSTOP too high");
    for(k = kmap; k < &kmap[NELEM(kmap)]; k++)
        if(mappages(pgdir, k->virt, k->phys_end - k->phys_start,
            (uint)k->phys_start, k->perm) < 0) {
            freevm(pgdir);
            return 0;
        }
    return pgdir;
}

// Allocate one page table for the machine for the kernel address
// space for scheduler processes.

```

```

void
kvmalloc(void)
{
    kpgdir = setupkvm();
    switchkvm();
}

// Switch h/w page table register to the kernel-only page table,
// for when no process is running.
void
switchkvm(void)
{
    lcr3(V2P(kpgdir)); // switch to the kernel page table
}

// Switch TSS and h/w page table to correspond to process p.
void
switchvm(struct proc *p)
{
    if(p == 0)
        panic("switchvm: no process");
    if(p->kstack == 0)
        panic("switchvm: no kstack");
    if(p->pgdir == 0)
        panic("switchvm: no pgdir");

    pushcli();
    mycpu()->gdt[SEG_TSS] = SEG16(STS_T32A, &mycpu()->ts,
                                sizeof(mycpu()->ts)-1, 0);
    mycpu()->gdt[SEG_TSS].s = 0;
    mycpu()->ts.ss0 = SEG_KDATA << 3;
    mycpu()->ts.esp0 = (uint)p->kstack + KSTACKSIZE;
    // setting IOPL=0 in eflags *and* iomb beyond the tss segment limit
    // forbids I/O instructions (e.g., inb and outb) from user space
    mycpu()->ts.iomb = (ushort) 0xFFFF;
    ltr(SEG_TSS << 3);
    lcr3(V2P(p->pgdir)); // switch to process's address space
    popcli();
}

// Load the initcode into address 0 of pgdir.
// sz must be less than a page.
void
initvm(pde_t *pgdir, char *init, uint sz)
{
    char *mem;

    if(sz >= PGSIZE)
        panic("initvm: more than a page");
    mem = kalloc();
    memset(mem, 0, PGSIZE);
    mappages(pgdir, 0, PGSIZE, V2P(mem), PTE_W|PTE_U);
    memmove(mem, init, sz);
}

// Load a program segment into pgdir. addr must be page-aligned
// and the pages from addr to addr+sz must already be mapped.
int

```

```

loaduvm(pde_t *pgdir, char *addr, struct inode *ip, uint offset, uint sz)
{
    uint i, pa, n;
    pte_t *pte;

    if((uint) addr % PGSIZE != 0)
        panic("loaduvm: addr must be page aligned");
    for(i = 0; i < sz; i += PGSIZE){
        if((pte = walkpgdir(pgdir, addr+i, 0)) == 0)
            panic("loaduvm: address should exist");
        pa = PTE_ADDR(*pte);
        if(sz - i < PGSIZE)
            n = sz - i;
        else
            n = PGSIZE;
        if(readi(ip, P2V(pa), offset+i, n) != n)
            return -1;
    }
    return 0;
}

// Allocate page tables and physical memory to grow process from oldsz to
// newsz, which need not be page aligned. Returns new size or 0 on error.
int
allocuvm(pde_t *pgdir, uint oldsz, uint newsz)
{
    char *mem;
    uint a;

    if(newsz >= KERNBASE)
        return 0;
    if(newsz < oldsz)
        return oldsz;

    a = PGROUNDUP(oldsz);
    for(; a < newsz; a += PGSIZE){
        mem = kalloc();
        if(mem == 0){
            cprintf("allocuvm out of memory\n");
            deallocuvm(pgdir, newsz, oldsz);
            return 0;
        }
        memset(mem, 0, PGSIZE);
        if(mappages(pgdir, (char*)a, PGSIZE, V2P(mem), PTE_W|PTE_U) < 0){
            cprintf("allocuvm out of memory (2)\n");
            deallocuvm(pgdir, newsz, oldsz);
            kfree(mem);
            return 0;
        }
    }
    return newsz;
}

// Deallocate user pages to bring the process size from oldsz to
// newsz. oldsz and newsz need not be page-aligned, nor does newsz
// need to be less than oldsz. oldsz can be larger than the actual
// process size. Returns the new process size.

```

```

int
deallocvm(pde_t *pgdir, uint oldsz, uint newsz)
{
    uint a;

    if (newsz >= oldsz)
        return oldsz;

    a = PGROUNDUP(newsz);
    for (; a < oldsz; a += PGSIZE) {
        int rem;
        // free_if_last=1: 해당 PFN의 마지막 참조일 때만 실제 kfree
        // remaining_out(=rem)은 여기서는 굳이 쓰지 않아도 됩니다.
        (void)vm_unmap_one(pgdir, a, /*free_if_last=*/1, &rem);
    }
    return newsz;
}

// Free a page table and all the physical memory pages
// in the user part.
void
freevm(pde_t *pgdir)
{
    uint i;

    if(pgdir == 0)
        panic("freevm: no pgdir");
    deallocvm(pgdir, KERNBASE, 0);
    for(i = 0; i < NPENTRIES; i++){
        if(pgdir[i] & PTE_P){
            char * v = P2V(PTE_ADDR(pgdir[i]));
            kfree(v);
        }
    }
    kfree((char*)pgdir);
}

// Clear PTE_U on a page. Used to create an inaccessible
// page beneath the user stack.
void
clearpteu(pde_t *pgdir, char *uva)
{
    pte_t *pte = walkpgdir(pgdir, uva, 0);
    if (pte == 0)
        panic("clearpteu");

    // clearpteu()
    uint32_t va_page = (uint32_t)PGROUNDDOWN((uint)uva);
    uint32_t pa_page = PTE_ADDR(*pte);
    int pid = (myproc() && myproc()->pgdir == pgdir) ? myproc()->pid : ipt_pid_from_pgdir(pgdir);
    if (pid > 0 && (pgdir != kpgdir) && (*pte & PTE_U)) {
        ipt_remove(pgdir, (uint32_t)pid, va_page, pa_page);
    }

    // 실제 권한 변경
    *pte &= ~PTE_U;
}

```



```

// --- SW-TLB 정합성: 현재 활성 주소공간이면 해당 엔트리 무효화 ---
if (myproc() && myproc()->pgdir == pgdir) {
    tlb_invalidate_one(myproc()->pid, va_page);
}
}

// Given a parent process's page table, create a copy
// of it for a child.
pde_t*
copyuvm(pde_t *pgdir, uint sz)
{
    pde_t *d;
    pte_t *pte;
    uint pa, i, flags;
    char *mem;

    if((d = setupkvm()) == 0)
        return 0;
    for(i = 0; i < sz; i += PGSIZE){
        if((pte = walkpgdir(pgdir, (void *) i, 0)) == 0)
            panic("copyuvm: pte should exist");
        if(!(*pte & PTE_P))
            panic("copyuvm: page not present");
        pa = PTE_ADDR(*pte);
        flags = PTE_FLAGS(*pte);
        if((mem = kalloc()) == 0)
            goto bad;
        memmove(mem, (char*)P2V(pa), PGSIZE);
        if(mappages(d, (void*)i, PGSIZE, V2P(mem), flags) < 0) {
            kfree(mem);
            goto bad;
        }
    }
}

return d;

bad:
    freevm(d);
    return 0;
}

//PAGEBREAK!
// Map user virtual address to kernel address.
char*
uva2ka(pde_t *pgdir, char *uva)
{
    pte_t *pte;

    pte = walkpgdir(pgdir, uva, 0);
    if (pte == 0)                // 추가: PDE or PTE page 자체가 없음
        return 0;
    if((*pte & PTE_P) == 0)
        return 0;
    if((*pte & PTE_U) == 0)
        return 0;
    return (char*)P2V(PTE_ADDR(*pte));
}

```

```

// Copy len bytes from p to user address va in page table pgdir.
// Most useful when pgdir is not the current page table.
// uva2ka ensures this only works for PTE_U pages.
int
copyout(pde_t *pgdir, uint va, void *p, uint len)
{
    char *buf, *pa0;
    uint n, va0;

    buf = (char*)p;
    while(len > 0){
        va0 = (uint)PGROUNDDOWN(va);
        pa0 = uva2ka(pgdir, (char*)va0);
        if(pa0 == 0)
            return -1;
        n = PGSIZE - (va - va0);
        if(n > len)
            n = len;
        memmove(pa0 + (va - va0), buf, n);
        len -= n;
        buf += n;
        va = va0 + PGSIZE;
    }
    return 0;
}

//PAGEBREAK!
// Blank page.
//PAGEBREAK!
// Blank page.
//PAGEBREAK!
// Blank page.

// 성공 0
// 실패 음수: -1(PDE 없음), -2(PTE 없음), -3(인자 오류)
int
sw_vtop(pde_t *pgdir, const void *va, uint32_t *pa_out, uint32_t *pte_flags_out)
{
    // va==0 은 xv6에서 "정상"일 수 있으므로 예외로 보면 안 됨!
    if (!pgdir || !pa_out || !pte_flags_out)
        return -3;

    uint32_t a = (uint32_t)va;
    uint32_t va_page = PGROUNDDOWN(a);

    // 현재 PID (TLB 태깅용), 커널 초기부팅 구간 안전하게 0 처리
    struct proc *mp = myproc();
    uint32_t pid = (mp ? (uint32_t)mp->pid : 0);

    // --- (A) 소프트 TLB 히트 시 바로 반환 ---
    uint32_t pa_page;
    uint16_t tlb_flags;
    if (tlb_lookup(pid, va_page, &pa_page, &tlb_flags)) {
        *pa_out = pa_page | (a & 0xFFF);
        *pte_flags_out = (uint32_t)tlb_flags; // tlb에는 하위 12비트 전체를 넣어둘 것을 권장
        return 0;
    }
}

```

```

// --- (B) PDE 검사 ---
uint32_t pdx = PDX(a);
pde_t pde = pgdir[pdx];
if ((pde & PTE_P) == 0)
    return -1; // PDE 없음

// --- (C) PTE 접근: PDE의 물리주소 → 커널 가상주소(P2V)로 변환 필수! ---
pte_t *pgtab = (pte_t*)P2V(PTE_ADDR(pde));

uint32_t ptx = PTX(a);
pte_t pte = pgtab[ptx];
if ((pte & PTE_P) == 0)
    return -2; // PTE 없음

// --- (D) 최종 물리주소 조합 ---
uint32_t pa = PTE_ADDR(pte) | (a & 0xFFF);

// --- (E) 플래그: 하위 12비트 전체를 반환(A/D 포함) → hw와 일치 ---
uint32_t flags = (pte & 0xFFF);

// --- (F) TLB 적재 ---
tlb_insert(pid, va_page, PTE_ADDR(pte), (uint16_t)flags);

*pa_out = pa;
*pte_flags_out = flags;
return 0;
}

// 디버그용 walkpgdir 래퍼
int hw_walk_probe(pde_t *pgdir, const void *va, uint *pa_out, uint *fl_out) {
    uint v = (uint)va;
    pde_t pde = pgdir[PDX(v)];
    if ((pde & PTE_P) == 0) return 1; // PDE 없음

    pte_t *pt = (pte_t*)P2V(PTE_ADDR(pde));
    pte_t pte = pt[PTX(v)];
    if ((pte & PTE_P) == 0) return 2; // PTE 없음

    *pa_out = (PTE_ADDR(pte) | (v & 0xFFF));
    *fl_out = (pte & 0xFFF);
    return 0;
}

int vm_map_user_page(pde_t *pgdir, uint va_page, uint pa_page, int perm) {
    // 1) 페이지 정렬 검증
    if ((va_page & (PGSIZE-1)) || (pa_page & (PGSIZE-1)))
        return -1;

    // 2) 사용자 영역만 허용 (필요 시 정책에 맞게 조정)
    if (va_page >= KERNBASE) return -1;
    if ((perm & PTE_U) == 0) return -1;

    // 3) 한 페이지만 매핑 (중복 매핑이면 mappages가 panic("remap"))
    return mappages(pgdir, (void*)va_page, PGSIZE, pa_page, perm);
}

```

```

// vm.c
/*int
protect_nowrite(pde_t *pgdir, void *uva)
{
    pte_t *pte = walkpgdir(pgdir, uva, 0);
    if (!pte || (*pte & PTE_P) == 0)
        return -1;

    uint32_t va_page = PGROUNDDOWN((uint)uva);

    // 현재/비현재 pgdir 모두에서 pid를 계산해 두되,
    // 실제 TLB invalidate는 '현재 주소공간'일 때만 수행.
    int pid = (myproc() && myproc()->pgdir == pgdir)
        ? myproc()->pid
        : ipt_pid_from_pgdir(pgdir);

    // 쓰기 권한 제거
    *pte &= ~PTE_W;

    // 현재 활성 주소공간인 경우에만 SW-TLB invalidate
    if (myproc() && myproc()->pgdir == pgdir) {
        tlb_invalidate_one(pid, va_page); // ← pid를 실제로 사용
    }
    return 0;
} */

int
protect_nowrite(pde_t *pgdir, void *uva)
{
    if (!pgdir || !uva) return -1;

    pte_t *pte = walkpgdir(pgdir, uva, 0);
    if (!pte || (*pte & PTE_P) == 0)
        return -1;

    uint va_page = PGROUNDDOWN((uint)uva);
    uint pa_page = PTE_ADDR(*pte);
    uint old_flags = (*pte) & 0xFFF;
    uint new_flags = old_flags & ~PTE_W; // W 비트만 제거

    // 이미 W가 꺼져있으면 그대로 종료
    if (new_flags == old_flags)
        return 0;

    // PTE를 새 플래그로 갱신 (주소 비트 보존)
    *pte = pa_page | new_flags;

    // 현재/비현재 모두에서 pid 산출 (IPT 갱신용)
    int pid = (myproc() && myproc()->pgdir == pgdir)
        ? myproc()->pid
        : ipt_pid_from_pgdir(pgdir);

    // ---- IPT 플래그 동기화 (유저 매핑만) ----
    if ((old_flags & PTE_U) && pid > 0 && pgdir != kpgdir) {
        // 1) 기존 (pid, va) 엔트리 제거
        (void)ipt_remove(pgdir, (uint32_t)pid, va_page, pa_page);
        // 2) 새 플래그로 재삽입
        ipt_add(pgdir, (uint32_t)pid, va_page, pa_page, new_flags);
    }
}

```

```

}

// ---- SW-TLB invalidate: 현재 활성 주소공간이면 ----
if (myproc() && myproc()->pgdir == pgdir) {
    tlb_invalidate_one(pid, va_page);
}

return 0;
}

// vm.c
// vm.c
int
vm_unmap_one(pde_t *pgdir, uint va_any, int free_if_last, int *remaining_out)
{
    uint a = PGROUNDDOWN(va_any);

    pte_t *pte = walkpgdir(pgdir, (void*)a, 0);
    if (!pte) return -1; // PDE 없음
    if (!(*pte & PTE_P)) return -2; // PTE 없음

    uint pa_page = PTE_ADDR(*pte);
    uint32_t pfn = PFN(pa_page); // 추가
    int remaining = -1; // 모르면 kfree 안 함

    // 유저 매핑이면 IPT에서 먼저 제거
    if (a < KERNBASE && (*pte & PTE_U)) {
        int pid = (myproc() && myproc()->pgdir == pgdir)
            ? myproc()->pid
            : ipt_pid_from_pgdir(pgdir);

        if (pid > 0 && pgdir != kpgdir) {
            int r = ipt_remove(pgdir, (uint32_t)pid, a, pa_page);
            if (r >= 0) {
                remaining = r;
            } else {
                remaining = ipt_remove_fallback((uint32_t)pid, a, pfn);
            }
        }
    }

    // 실제 언맵
    *pte = 0;

    // 현재 주소공간이면 TLB invalidate
    if (myproc() && myproc()->pgdir == pgdir)
        tlb_invalidate_one(myproc()->pid, a);

    // 마지막 참조면 물리 프레임 해제
    if (free_if_last && remaining == 0)
        kfree((char*)P2V(pa_page));

    if (remaining_out) *remaining_out = remaining;
    return 0;
}

```

```

// vtop.c
#include "types.h"
#include "stat.h"
#include "user.h"

// 공백 판별
static int is_space(char c) {
    return c==' ' || c=='\t' || c=='\n' || c=='\r' || c=='\v' || c=='\f';
}

// 입력 전체에 A-F/a-f가 하나라도 있으면 16진으로 간주
static int has_hex_alpha(const char *s) {
    for (; *s; s++) {
        if (*s=='x' || *s=='X') continue;
        if ((*s>='a' && *s<='f') || (*s>='A' && *s<='F')) return 1;
    }
    return 0;
}

// 자동 진수 판별 파서: 0x... → 16진, 그 외에 A-F/a-f 포함 → 16진, 아니면 10진
static int parse_u32_autobase(const char *s, uint *out) {
    int base = 10;
    uint val = 0;
    int i = 0, n_digits = 0;

    while (is_space(s[i])) i++;

    if (s[i]=='0' && (s[i+1]=='x' || s[i+1]=='X')) { base = 16; i += 2; }
    else if (has_hex_alpha(s + i)) { base = 16; }

    for (; s[i]; i++) {
        char c = s[i];
        if (is_space(c)) break;
        int d = -1;
        if (c >= '0' && c <= '9') d = c - '0';
        else if (base == 16 && c >= 'a' && c <= 'f') d = c - 'a' + 10;
        else if (base == 16 && c >= 'A' && c <= 'F') d = c - 'A' + 10;
        else return -1; // 유효하지 않은 문자
        if (d >= base) return -1;
        val = val * base + (uint)d;
        n_digits++;
    }

    while (is_space(s[i])) i++;
    if (s[i] != 0) return -1; // 뒷부분에 쓰레기 문자가 남음
    if (n_digits == 0) return -1; // 숫자를 하나도 못 읽음

    *out = val;
    return 0;
}

int
main(int argc, char *argv[])
{
    if (argc != 2) {
        printf(1, "usage: vtop <va>\n");
        printf(1, " accepts: decimal (e.g., 36864), hex with 0x (e.g., 0x9000), or bare hex (e.g., 9FFF)\n");
        exit();
    }

```

```

}

uint va_u = 0;
if (parse_u32_autobase(argv[1], &va_u) < 0) {
    printf(1, "vtop: invalid address '%s'\n", argv[1]);
    exit();
}

void *va = (void*)va_u;
uint pa = 0, flags = 0;
int rc = vtop(va, &pa, &flags);
if (rc == 0)
    printf(1, "VA=0x%x -> PA=0x%x flags=0x%x\n", va_u, pa, flags);
else
    printf(1, "vtop failed rc=%d\n", rc);
exit();
}

```

[vtop_proof.c]

```

#include "types.h"
#include "stat.h"
#include "user.h"
#include "mmu.h" // PTE_* bits

static void show(const char *tag, uint va){
    uint pa, fl;
    int rc = vtop((void*)va, &pa, &fl);
    if(rc==0)
        printf(1, "%s va=0x%x -> pa=0x%x flags=0x%x (P=%d W=%d U=%d)\n",
            tag, va, pa, fl, !(fl&PTE_P), !(fl&PTE_W), !(fl&PTE_U));
    else
        printf(1, "%s va=0x%x -> rc=%d\n", tag, va, rc);
}

static void check_offset(uint va_base) {
    uint pa, fl;
    int rc = vtop((void*)(va_base + 0x123), &pa, &fl);
    printf(1, "[offset] va=0x%x rc=%d pa=0x%x (pa&0xFFF=0x%x expect 0x123)\n",
        va_base+0x123, rc, pa, pa & 0xFFF);
}

static void check_unmapped(uint va) {
    uint pa, fl;
    int rc = vtop((void*)va, &pa, &fl);
    printf(1, "[unmapped] va=0x%x rc=%d (expect -1 or -2)\n", va, rc);
}

int
main(void)
{
    // 1) 사용자 힙에서 정확히 1페이지만 확보
    char *p = sbrk(4096);
    if((int)p < 0){ printf(1, "sbrk fail\n"); exit(); }
    uint va0 = ((uint)p) & ~0xFFF;

    // 2) 동일 VA 두 번 조회 → 2번째는 SW-TLB hit 기대(커널 로그로 확인)

```

```

show("[vtop]", va0);
show("[vtop]", va0);

// 3) 오프셋 보존 확인
check_offset(va0);

// 4) 원본의 PA/flags
uint pa0, fl0;
if(vtop((void*)va0, &pa0, &fl0) != 0){ printf(1,"vtop base fail\n"); exit(); }
uint pa_page = pa0 & ~0xFFF;

// 5) alias 만들기: va1은 아직 미할당 영역으로 설정
uint va1 = va0 + 0x1000; // sbrk(4096)만 했으니 va1은 unmapped 상태
if(vm_map_user_page(va1, pa_page, PTE_U|PTE_W) < 0){
    printf(1,"vm_map_user_page RWU fail\n"); exit();
}
show("[alias RWU]", va1);

// 6) 권한 변경: W 제거 → 플래그 0x7 → 0x5 기대 + TLB invalidation 반영
if (protect_nowrite((void*)va1) < 0) {
    printf(1,"protect_nowrite fail\n"); exit();
}
show("[alias RO ]", va1);

// 7) unmap → vtop이 미매핑 에러(-2) 반환 기대
if (vm_unmap_one((void*)va1, 0) < 0) {
    printf(1,"unmap fail\n"); exit();
}
show("[after unmap]", va1); // rc 음수 기대
check_unmapped(va1);

// 8) 다시 remap (RO) → flags 0x5 확인
if (vm_map_user_page(va1, pa_page, PTE_U /*RO*/) < 0) {
    printf(1,"remap RO fail\n"); exit();
}
show("[remap RO]", va1);

printf(1,"[vtop_proof] done\n");
exit();
}

```